Báo cáo thực hành Data Engineering — Lab L01–L06

Trương Minh Tiến

30/10/2026

Mục lục

L01 — HDFS: blocks, replicas và sự cố DataNode **1**
1–2. Khởi động cluster và `hdfs dfs` shell . . . 2
3. Block mặc định (128 MB) và block 16 MB . . . 3
4. Replication factor . . . 4
5. Tắt DataNode → under-replication → re-replication . . . 5
6–7. Safe mode và fsimage . . . 9
Trả lời checkpoint . . . 9

L02 — MapReduce với Hadoop Streaming (Python) **11**
File nộp . . . 11
1–2. Word count: test local và chạy trên YARN . . . 13
3–4. Dữ liệu taxi CSV và map-side join . . . 16
5. Combiner và không combiner . . . 16
6. YARN và JobHistory UI . . . 18
Stretch (bonus): giá cước trung bình theo zone với combiner đúng . . . 22
Trả lời checkpoint . . . 23

L03 — Hive: SQL trên HDFS, partitioning, định dạng file, bucketing **24**
File SQL đã hoàn thành . . . 24
1–2. Khởi động stack và đưa CSV lên HDFS . . . 28
3. External tables . . . 29
4. Bảng partition ORC và Parquet . . . 29
Bảng kích thước . . . 33
5. CSV, ORC và Parquet: bảng thời gian (lần chạy thứ 2) . . . 33
6. Partition pruning . . . 35
7. Bucketing . . . 35
Stretch . . . 36
Trả lời checkpoint . . . 38

L04 — Ingestion: Sqoop và Spark JDBC từ PostgreSQL vào HDFS **40**
File nộp . . . 40
1. Nguồn dữ liệu . . . 42
2. Sqoop full import và cách chia split . . . 42
TODO 1.2: `customers --split-by customer_zip_code_prefix -m 4` . . . 44
3. Sqoop incremental: append, lastmodified, saved job, merge, export . . . 45
4. Spark JDBC: đọc song song và ghi Parquet (TODO 1–3) . . . 47
So sánh Sqoop và Spark JDBC (orders) . . . 50

5. Spark incremental: watermark, change log, snapshot (TODO 4–5) 51
Trả lời checkpoint 53

L05 — PySpark cơ bản: DataFrame, Spark SQL và Spark UI 55
File nộp 55
Kết quả chính (khớp đáp án) 57
Plan và Spark UI 62
Ghi Parquet phân vùng (TODO 7) 66
Trả lời checkpoint 66

L06 — Spark performance: shuffle, join, skew, AQE, partitioning, cache 69
File nộp 69
Bảng kết quả (lần 2) 71
Bằng chứng trên Spark UI 72
Skew: Summary Metrics của stage join 79
Trả lời checkpoint 83

L01 — HDFS: blocks, replicas và sự cố DataNode

Môi trường: WSL2 (x86_64, 12 CPU, Docker 31.9 GB), Hadoop 3.4.3 — 1 NameNode, 3 DataNode, RM, NM, JobHistory. Log gốc của mọi bước nằm trong các file `0*.txt` ở thư mục này; ảnh ở `img/`.

Ánh xạ IP → hostname (từ `hdfs dfsadmin -report`):

IP	Host
172.29.0.5	datanode1
172.29.0.2	datanode2
172.29.0.3	datanode3

1–2. Khởi động cluster và `hdfs dfs` shell

```
root@namenode: hdfs dfsadmin -report / hdfs dfs ... / TODO 2.1

Safe mode is OFF
### 1. dfsadmin -report
Live datanodes (3):
Name: 172.29.0.2:9866 (l01_hdfs-datanode2-1.l01_hdfs_default)
Hostname: datanode2
Name: 172.29.0.3:9866 (l01_hdfs-datanode3-1.l01_hdfs_default)
Hostname: datanode3
Name: 172.29.0.5:9866 (l01_hdfs-datanode1-1.l01_hdfs_default)
Hostname: datanode1
### 2. dfs shell
Found 1 items
-rw-r--r--   3 root supergroup      12331 2026-10-30 10:14 /data/taxi/taxi_zone_lookup.csv
Manhattan rows: 70
-rw-r--r-- 1 root root 12331 Oct 30 10:15 /tmp/zones_copy.csv
Deleted /user/root/zones_copy.csv
12.0 K  36.1 K  /data/taxi/taxi_zone_lookup.csv
### TODO 2.1
name=taxi_zone_lookup.csv blocksize=134217728 repl=3 size=12331
```

Hadoop | Overview | Datanodes | Datanode Volume Failures | Snapshot | Startup Progress | Utilities

Overview 'namenode:8020' (✔active)

Started:	Fri Oct 30 17:19:21 +0700 2026
Version:	3.4.3, r9d50c6884666e794e45102260a4017bb31802e1b
Compiled:	Fri Feb 13 21:23:00 +0700 2026 by stevel from branch-3.4.3
Cluster ID:	de-labs-cluster
Block Pool ID:	BP-361839436-172.29.0.7-1793355266756

Summary

Security is off.

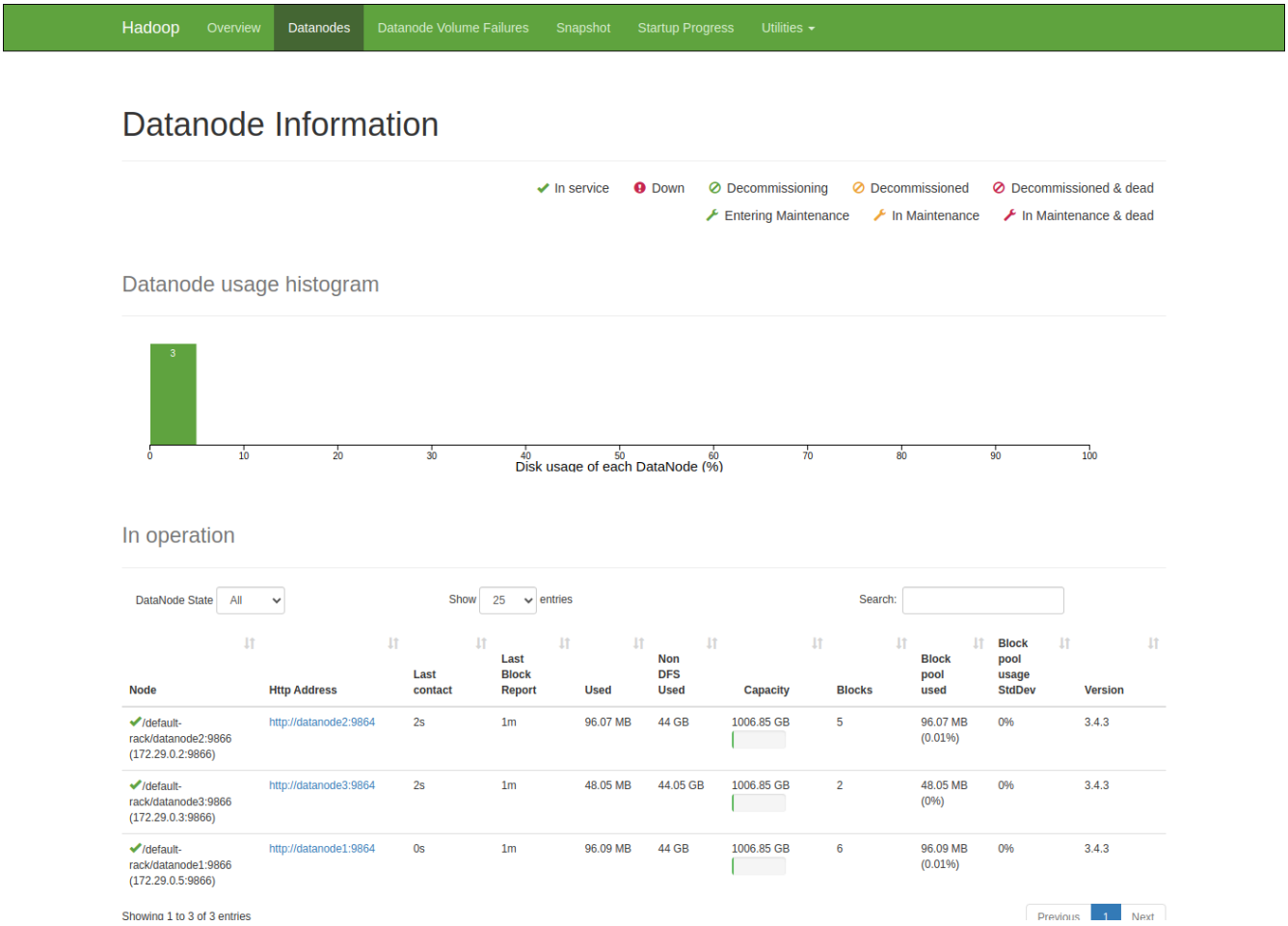
Safemode is off.

17 files and directories, 6 blocks (6 replicated blocks, 0 erasure coded block groups) = 23 total filesystem object(s).

Heap Memory used 81.64 MB of 384 MB Heap Memory. Max Heap Memory is 384 MB.

Non Heap Memory used 63.74 MB of 66.31 MB Commited Non Heap Memory. Max Non Heap Memory is <unbounded>.

Configured Capacity:	2.95 TB
Configured Remote Capacity:	0 B
DFS Used:	240.21 MB (0.01%)
Non DFS Used:	132.06 GB
DFS Remaining:	2.67 TB (90.53%)
Block Pool Used:	240.21 MB (0.01%)
DataNodes usages% (Min/Median/Max/stdDev):	0.00% / 0.01% / 0.01% / 0.00%
Live Nodes	3 (Decommissioned: 0, In Maintenance: 0)
Dead Nodes	0 (Decommissioned: 0, In Maintenance: 0)
Decommissioning Nodes	0
Entering Maintenance Nodes	0
Total Datanode Volume Failures	0 (0 B)
Number of Under-Replicated Blocks	0
Number of Blocks Pending Deletion (including replicas)	0
Block Deletion Start Time	Fri Oct 30 17:19:21 +0700 2026
Last Checkpoint Time	Fri Oct 30 17:19:37 +0700 2026



TODO 2.1

```
hdfs dfs -stat "name=%n blocksize=%o repl=%r size=%b" /data/taxi/taxi_zone_lookup.csv
# name=taxi_zone_lookup.csv blocksize=134217728 repl=3 size=12331
```

3. Block mặc định (128 MB) và block 16 MB

Upload	Số block	Kích thước block
mặc định 128 MB	1	49,961,641
`-D dfs.blocksize=16m`	3	16,777,216 + 16,777,216 + 16,407,209

```
hdfs fsck -files -blocks -locations  (128 MB vs 16 MB blocks)

### 3. blocks
yellow_tripdata_2024-01.parquet blocksize=134217728 repl=3 size=49961641
yellow_tripdata_2024-01.parquet blocksize=16777216 repl=3 size=49961641
--- fsck default_blocks
/data/taxi/default_blocks <dir>
/data/taxi/default_blocks/yellow_tripdata_2024-01.parquet 49961641 bytes, replicated: replication=3, 1 block(s):  OK
0. BP-361839436-172.29.0.7-1793355266756:blk_1073741827_1003 len=49961641 Live_repl=3
  [DatanodeInfoWithStorage[172.29.0.3:9866,DS-40f5e434-77f4-46f4-b595-2cea416937fe,DISK],
  DatanodeInfoWithStorage[172.29.0.5:9866,DS-a1311a4-4968-49b1-af9d-4b33712c6021,DISK], DatanodeInfoWithStorage[172.29.0.2:9866,DS-
  abdc23f4-4959-47fa-b34b-0829bd838ae0,DISK]]
Status: HEALTHY

### fsck small_blocks rep3 (settled)
/data/taxi/small_blocks <dir>
/data/taxi/small_blocks/yellow_tripdata_2024-01.parquet 49961641 bytes, replicated: replication=3, 3 block(s):  OK
0. BP-361839436-172.29.0.7-1793355266756:blk_1073741828_1004 len=16777216 Live_repl=3  [DatanodeInfoWithStorage[172.29.0.2:9866,DS-
  abdc23f4-4959-47fa-b34b-0829bd838ae0,DISK], DatanodeInfoWithStorage[172.29.0.3:9866,DS-40f5e434-77f4-46f4-b595-2cea416937fe,DISK],
  DatanodeInfoWithStorage[172.29.0.5:9866,DS-a1311a4-4968-49b1-af9d-4b33712c6021,DISK]]
1. BP-361839436-172.29.0.7-1793355266756:blk_1073741829_1005 len=16777216 Live_repl=3
  [DatanodeInfoWithStorage[172.29.0.3:9866,DS-40f5e434-77f4-46f4-b595-2cea416937fe,DISK],
  DatanodeInfoWithStorage[172.29.0.5:9866,DS-a1311a4-4968-49b1-af9d-4b33712c6021,DISK], DatanodeInfoWithStorage[172.29.0.2:9866,DS-
  abdc23f4-4959-47fa-b34b-0829bd838ae0,DISK]]
2. BP-361839436-172.29.0.7-1793355266756:blk_1073741830_1006 len=16407209 Live_repl=3
  [DatanodeInfoWithStorage[172.29.0.3:9866,DS-40f5e434-77f4-46f4-b595-2cea416937fe,DISK],
  DatanodeInfoWithStorage[172.29.0.5:9866,DS-a1311a4-4968-49b1-af9d-4b33712c6021,DISK], DatanodeInfoWithStorage[172.29.0.2:9866,DS-
  abdc23f4-4959-47fa-b34b-0829bd838ae0,DISK]]
Status: HEALTHY
 Under-replicated blocks:       0 (0.0 %)
```

Hình 1: fsck 128m vs 16m

Các file block vật lý trên datanode1 (`blk_<id>` + `blk_<id>_<genstamp>.meta`); block cuối chỉ 16,407,209 byte, không bị pad lên 16 MB:

```
datanode1: find /hadoop/dfs/data -name 'blk_*'

-rw-r--r-- 1 root root 128191 Oct 30 10:15 blk_1073741830_1006.meta
-rw-r--r-- 1 root root 16407209 Oct 30 10:15 blk_1073741830
-rw-r--r-- 1 root root 16777216 Oct 30 10:15 blk_1073741829
-rw-r--r-- 1 root root 131079 Oct 30 10:15 blk_1073741829_1005.meta
-rw-r--r-- 1 root root 107 Oct 30 10:14 blk_1073741825_1001.meta
-rw-r--r-- 1 root root 49961641 Oct 30 10:15 blk_1073741827
-rw-r--r-- 1 root root 12331 Oct 30 10:14 blk_1073741825
-rw-r--r-- 1 root root 16777216 Oct 30 10:15 blk_1073741828
-rw-r--r-- 1 root root 390335 Oct 30 10:15 blk_1073741827_1003.meta
-rw-r--r-- 1 root root 131079 Oct 30 10:15 blk_1073741828_1004.meta
```

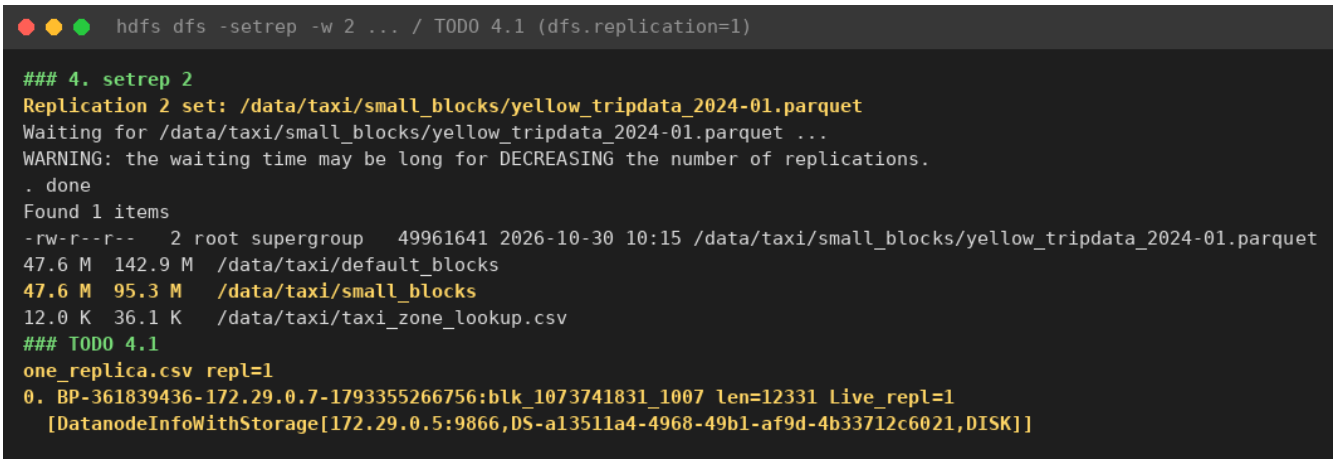
Hình 2: block files

4. Replication factor

`setrep -w 2` → `Live_repl=2` cho cả 3 block; dung lượng raw của `small_blocks` giảm từ 142.9 M xuống **95.3 M**.

TODO 4.1

```
hdfs dfs -D dfs.replication=1 -put /datasets/data/taxi/taxi_zone_lookup.csv
 ↪  /tmp/one_replica.csv
# one_replica.csv repl=1   (replica duy nhất nằm trên 172.29.0.5 = datanode1)
```

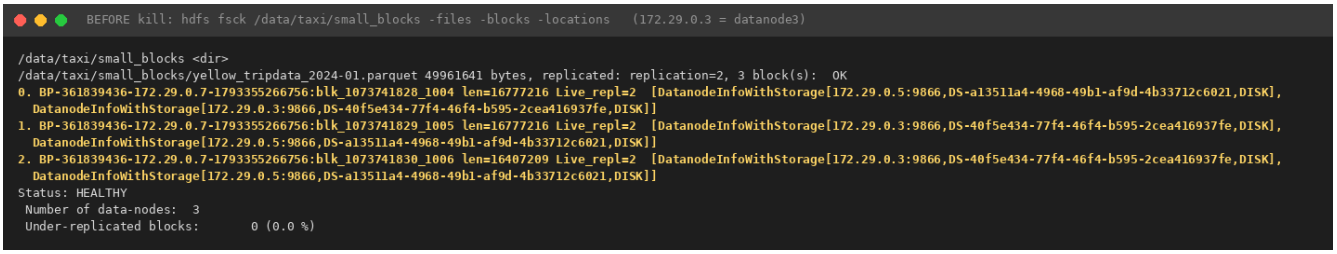


Hình 3: setrep

5. Tắt DataNode → under-replication → re-replication

Tắt **datanode3** (đang giữ replica của cả 3 block của file `small_blocks`).

(a) `fsck` của file block 16 MB TRƯỚC khi tắt (replica nằm trên `.5` datanode1 và `.3` datanode3):



Hình 4: before

(b) Thời gian đến khi node bị coi là chết: 90 s (`Dead datanodes (1)` lúc t+90 s). Lúc t+94 s (10:17:40) NameNode log `removeDeadDatanode: lost heartbeat from 172.29.0.3`; từ lần poll t+98 s trở đi `Under replicated blocks: 2`. Con số 90 s khớp với lý thuyết 2 × recheck-interval (30 s) + 10 × heartbeat (3 s).

```
docker compose stop datanode3  +  watch hdfs dfsadmin -report

datanode3 stopped at 10:16:06 UTC
t+1s    Under replicated blocks: 0 Live datanodes (3):
t+9s    Under replicated blocks: 0 Live datanodes (3):
t+17s   Under replicated blocks: 0 Live datanodes (3):
t+26s   Under replicated blocks: 0 Live datanodes (3):
t+34s   Under replicated blocks: 0 Live datanodes (3):
t+42s   Under replicated blocks: 0 Live datanodes (3):
t+50s   Under replicated blocks: 0 Live datanodes (3):
t+58s   Under replicated blocks: 0 Live datanodes (3):
t+66s   Under replicated blocks: 0 Live datanodes (3):
t+75s   Under replicated blocks: 0 Live datanodes (3):
t+82s   Under replicated blocks: 0 Live datanodes (3):
t+90s   Under replicated blocks: 0 Live datanodes (2): Dead datanodes (1):
>>> declared DEAD after 90s
t+98s   Under replicated blocks: 2 Live datanodes (2): Dead datanodes (1):
t+105s          Under replicated blocks: 2 Live datanodes (2): Dead datanodes (1):
t+113s          Under replicated blocks: 2 Live datanodes (2): Dead datanodes (1):
t+121s          Under replicated blocks: 2 Live datanodes (2): Dead datanodes (1):
t+129s          Under replicated blocks: 2 Live datanodes (2): Dead datanodes (1):
t+137s          Under replicated blocks: 2 Live datanodes (2): Dead datanodes (1):
namenode-1  | 2026-10-30 10:17:40,079 INFO hdfs.StateChange: BLOCK* removeDeadDatanode: lost heartbeat from 172.29.0.3:9866, removeBlocksFromBlockMap true
```

Hadoop | Overview | Datanodes | Datanode Volume Failures | Snapshot | Startup Progress | Utilities

Overview 'namenode:8020' (✔active)

Started:	Fri Oct 30 17:19:21 +0700 2026
Version:	3.4.3, r9d50c6884666e794e45102260a4017bb31802e1b
Compiled:	Fri Feb 13 21:23:00 +0700 2026 by stevel from branch-3.4.3
Cluster ID:	de-labs-cluster
Block Pool ID:	BP-361839436-172.29.0.7-1793355266756

Summary

Security is off.

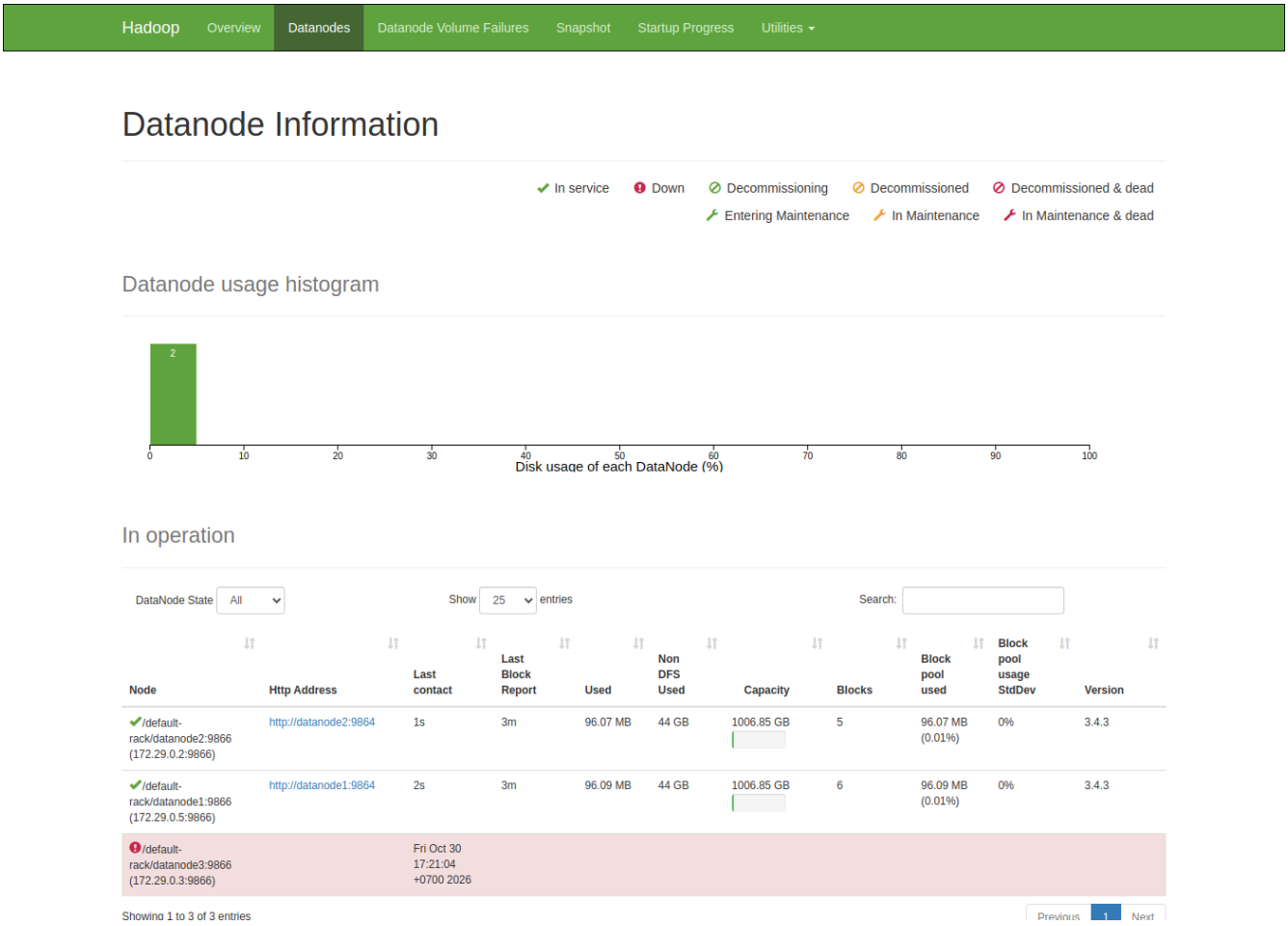
Safemode is off.

17 files and directories, 6 blocks (6 replicated blocks, 0 erasure coded block groups) = 23 total filesystem object(s).

Heap Memory used 206.64 MB of 384 MB Heap Memory. Max Heap Memory is 384 MB.

Non Heap Memory used 75.84 MB of 78.63 MB Commited Non Heap Memory. Max Non Heap Memory is <unbounded>.

Configured Capacity:	1.97 TB
Configured Remote Capacity:	0 B
DFS Used:	192.16 MB (0.01%)
Non DFS Used:	88.01 GB
DFS Remaining:	1.78 TB (90.53%)
Block Pool Used:	192.16 MB (0.01%)
DataNodes usages% (Min/Median/Max/stdDev):	0.01% / 0.01% / 0.01% / 0.00%
Live Nodes	2 (Decommissioned: 0, In Maintenance: 0)
Dead Nodes	1 (Decommissioned: 0, In Maintenance: 0)
Decommissioning Nodes	0
Entering Maintenance Nodes	0
Total Datanode Volume Failures	0 (0 B)
Number of Under-Replicated Blocks	2
Number of Blocks Pending Deletion (including replicas)	0
Block Deletion Start Time	Fri Oct 30 17:19:21 +0700 2026
Last Checkpoint Time	Fri Oct 30 17:19:37 +0700 2026



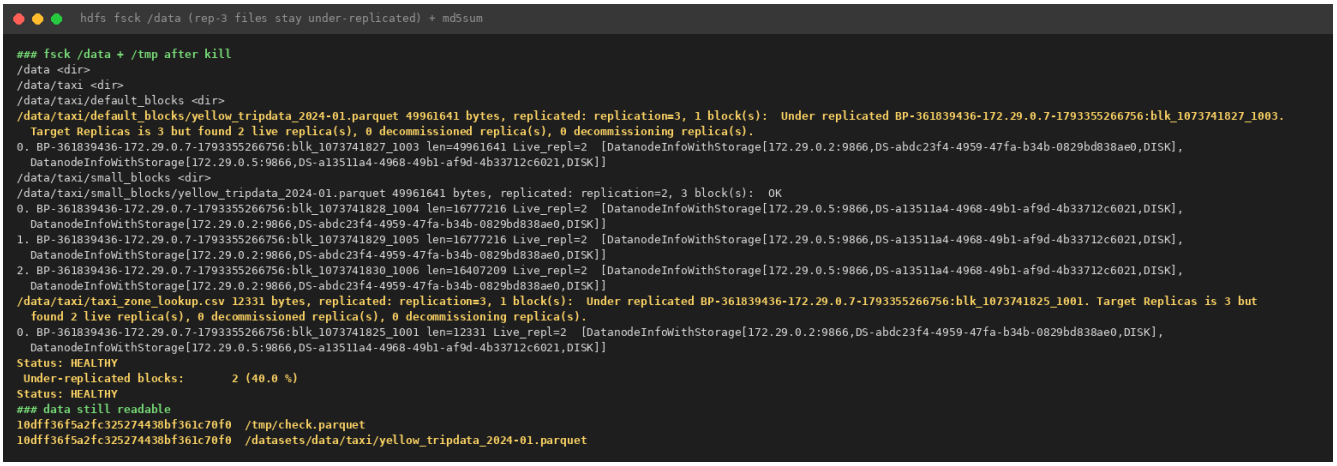
(c) `fsck` của file block 16 MB SAU khi tắt: cả 3 block vẫn `Live_repl=2`, nhưng replica trên `.3` đã được **chép lại sang `.2` (datanode2)** → file trở lại `OK`.

```
AFTER kill (re-replicated to 172.29.0.2 = datanode2): hdfs fsck /data/taxi/small_blocks

/data/taxi/small_blocks <dir>
/data/taxi/small_blocks/yellow_tripdata_2024-01.parquet 49961641 bytes, replicated: replication=2, 3 block(s):  OK
0. BP-361839436-172.29.0.7-1793355266756:blk_1073741828_1004 len=16777216 Live_repl=2  [DatanodeInfoWithStorage[172.29.0.5:9866,DS-a13511a4-4968-49b1-af9d-4b33712c6021,DISK],
  DatanodeInfoWithStorage[172.29.0.2:9866,DS-abdc23f4-4959-47fa-b34b-0829bd838ae0,DISK]]
1. BP-361839436-172.29.0.7-1793355266756:blk_1073741829_1005 len=16777216 Live_repl=2  [DatanodeInfoWithStorage[172.29.0.5:9866,DS-a13511a4-4968-49b1-af9d-4b33712c6021,DISK],
  DatanodeInfoWithStorage[172.29.0.2:9866,DS-abdc23f4-4959-47fa-b34b-0829bd838ae0,DISK]]
2. BP-361839436-172.29.0.7-1793355266756:blk_1073741830_1006 len=16407209 Live_repl=2  [DatanodeInfoWithStorage[172.29.0.5:9866,DS-a13511a4-4968-49b1-af9d-4b33712c6021,DISK],
  DatanodeInfoWithStorage[172.29.0.2:9866,DS-abdc23f4-4959-47fa-b34b-0829bd838ae0,DISK]]
Status: HEALTHY
 Number of data-nodes:  2
 Under-replicated blocks:       0 (0.0 %)
```

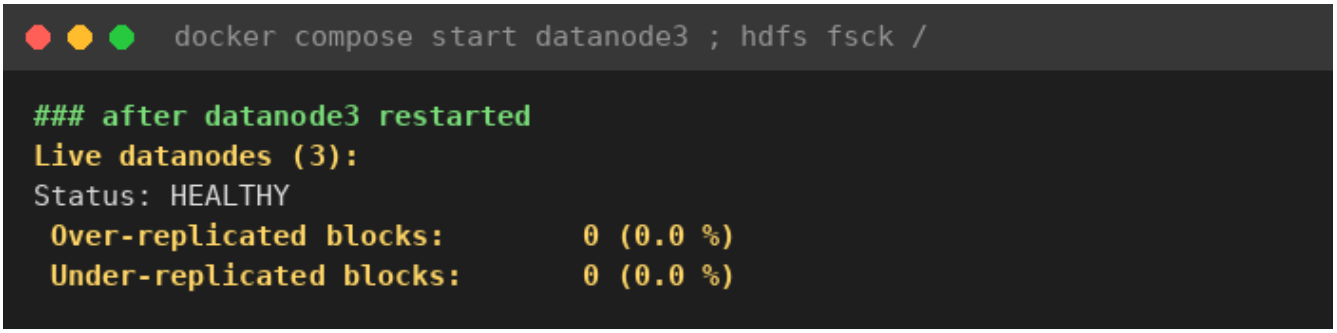
Hình 5: after

Hai file replication 3 (`default_blocks/...parquet`, `taxi_zone_lookup.csv`) vẫn **under-replicated** (2 block, 40 %) nhưng `Status: HEALTHY`; md5 của file tải về trùng với bản gốc → không mất dữ liệu. (`/tmp/one_replica.csv` nằm trên datanode1 nên không bị MISSING.)



Hình 6: under-replicated + md5

Bật lại datanode3 → `Under-replicated 0`, `Over-replicated 0` (replica thừa đã bị xoá):



Hình 7: back

6–7. Safe mode và fsimage

safe mode (manual + at restart) / fsimage via hdfs oiv

```
### 6. safe mode manual
Safe mode is OFF
Safe mode is ON
put: Cannot create file/tmp/x.csv._COPYING_. Name node is in safe mode.
Found 3 items
drwxr-xr-x   - root supergroup          0 2026-10-30 10:15 /data/taxi/default_blocks
drwxr-xr-x   - root supergroup          0 2026-10-30 10:15 /data/taxi/small_blocks
Safe mode is OFF
### automatic safe mode at restart
Safe mode is ON
Safe mode is OFF
### 7. fsimage
Safe mode is ON
Save namespace successful
Safe mode is OFF
-rw-r--r-- 1 root root    1574 Oct 30 10:19 fsimage_0000000000000000059
-rw-r--r-- 1 root root      62 Oct 30 10:19 fsimage_0000000000000000059.md5
-rw-r--r-- 1 root root       3 Oct 30 10:19 seen_txid
-rw-r--r-- 1 root root     189 Oct 30 10:19 VERSION
<name></name>
<name>tmp</name>
<name>hadoop-yarn</name>
<name>staging</name>
<name>mr-history</name>
<name>user</name>
<name>root</name>
<name>done</name>
<name>tmp</name>
<name>data</name>
<name>taxi</name>
<name>taxi_zone_lookup.csv</name>
block location info in fsimage? 0
```

Hình 8: safemode

Trong safe mode lệnh ghi bị từ chối (`Name node is in safe mode`), lệnh đọc vẫn chạy. Sau `restart namenode` safe mode tự bật vài giây. fsimage (`hdfs oiv`) chứa tên file/thư mục và block ID nhưng **không có vị trí block** (grep thông tin DataNode/port 9866 trả về 0 dòng).

Trả lời checkpoint

Q1. File 300 MB, block mặc định, replication 3? [300/128] = **3 block** (128 + 128 + 44 MB) → **9 block replica** → **~900 MB** đĩa raw (cộng thêm file `.meta` checksum rất nhỏ). NameNode chỉ giữ 3 đối tượng block trong RAM, không phụ thuộc replication.

Q2. Vì sao block cuối nhỏ hơn 16 MB, có lãng phí 16 MB không? File kết thúc ở đó:

49,961,641 − 2 × 16,777,216 = 16,407,209 byte. Block size chỉ là kích thước **tối đa**; file block trên DataNode đúng bằng lượng dữ liệu thực (thấy rõ ở ảnh 03), nên không lãng phí đĩa.

Q3. Danh sách vị trí block được lưu ở đâu? Restart NameNode thì sao? Chỉ nằm trong **RAM của NameNode**, không có trong fsimage hay edit log (ảnh 10). Khi restart, NameNode dựng lại block map từ **block report** của các DataNode, vì thế nó phải nằm trong safe mode đến khi nhận đủ report.

Q4. Vì sao file replication 2 được chữa còn file replication 3 thì không? Re-replication cần một DataNode **chưa có** replica của block đó. Khi chỉ còn 2 node sống: block rep-2 mất một bản trên datanode3 → chép được sang datanode2 (node còn trống). Block rep-3 thì đã có bản trên **cả hai** node còn lại, không còn node thứ ba để đặt bản sao, và HDFS không bao giờ đặt 2 replica của cùng một block trên một DataNode. Vì vậy nó vẫn under-replicated cho đến khi datanode3 quay lại.

Q5. `fsck` báo gì khi under-replicated, vì sao không phải lỗi? `Status: HEALTHY` kèm `Under-replicated blocks: 2 (40.0 %)`. HEALTHY nghĩa là block nào cũng có ít nhất `dfs.namenode.replication.min` (=1) replica sống, nên toàn bộ dữ liệu vẫn đọc được (md5 khớp). Under-replication chỉ là **rủi ro** (giảm độ dư thừa); lỗi thật là MISSING/CORRUPT.

Q6. Với `recheck-interval` mặc định (5 phút) thì sao? Vì sao timeout dài là mặc định tốt? Node bị coi là chết sau 2 × 5 phút + 10 × 3 s = **10.5 phút** (lab: 90 s). Timeout dài tránh **replication storm**: một node reboot hay chập mạng ngắn sẽ khiến cluster chép hàng TB block, rồi các bản đó lại thành thừa khi node quay lại sau một phút, tốn băng thông và I/O. Trong lúc chờ, client vẫn tránh các node stale (30 s không có heartbeat) khi đọc/ghi.

Q7. Vì sao NameNode khởi động trong safe mode, vì sao phải chặn ghi? Lúc khởi động NameNode biết namespace (fsimage + edits) nhưng chưa biết block nằm ở đâu. Trước khi ≥ 99.9 % block được báo cáo, nó không phân biệt được block "bị mất" với block "chưa báo cáo". Nếu cho phép ghi/xoá hoặc bắt đầu re-replication thì nó sẽ quyết định sai, ví dụ lên lịch chép hàng loạt các block chỉ đơn giản là chưa được báo cáo. Safe mode = chỉ đọc cho tới khi block map đáng tin cậy.

Q8. 1 file 1 KB và 1 triệu file 1 KB: vì sao trường hợp sau là vấn đề? Mỗi file, thư mục và block là một object trong heap NameNode (~150 B). 1 triệu file nhỏ = ≥ 2 triệu object ≈ 300 MB heap cho vỏn vẹn 1 GB dữ liệu, cộng thêm 1 triệu RPC và sau này 1 triệu map task (mỗi file một split). Đây là **small files problem**. Cách xử lý: gộp file, dùng định dạng container (Parquet/ORC, SequenceFile, HAR).

L02 — MapReduce với Hadoop Streaming (Python)

Cluster: dùng lại cluster L01 (1 NN, 3 DN, RM, NM, JobHistory). Log gốc trong các file `0*.txt` và `*combiner.log`; ảnh ở `img/`.

File nộp

File	Nội dung
`wc_mapper.py`, `wc_reducer.py`	TODO 1–3 (word count)
`zones_mapper.py`	TODO 4–6 (map-side join với `taxi_zone_lookup.csv`, counter `BAD_RECORDS`)
`stretch_avg_fare/`	Stretch: giá cước trung bình theo zone với combiner đúng `(sum, count)`

wc_mapper.py

```
#!/usr/bin/env python3
"""Word count -- MAPPER (Hadoop Streaming).

Hadoop feeds the input split to this script line by line on STDIN.
Whatever you print to STDOUT is the map output: one  key<TAB>value  per line.

Goal: for every word in the line, emit  "<word>\t1".
Normalise words to lower case and keep only letters and apostrophes,
e.g. "Hadoop's HDFS, hadoop!" -> hadoop's 1 / hdfs 1 / hadoop 1
"""
import re
import sys

WORD = re.compile(r"[a-z][a-z']*")

for line in sys.stdin:
    for word in WORD.findall(line.lower()):
        print(f"{word}\t1")
```

wc_reducer.py

```
#!/usr/bin/env python3
"""Word count -- REDUCER (Hadoop Streaming).

Hadoop SORTS the map output by key before calling the reducer, so all lines
with the same word arrive one after another:
    apache\t1
    apache\t1
    hadoop\t1
Unlike the Java API you do NOT get (key, [values]) -- you must detect the
key change yourself.

Goal: print "<word>\t<total>" once per word.
"""
import sys

current_word, current_count = None, 0

for line in sys.stdin:
    word, _, count = line.rstrip("\n").partition("\t")
    count = int(count)
    if word == current_word:
        current_count += count
    else:
        if current_word is not None:
            print(f"{current_word}\t{current_count}")
        current_word, current_count = word, count

if current_word is not None:  # the last word's total
    print(f"{current_word}\t{current_count}")
```

zones_mapper.py

```
#!/usr/bin/env python3
"""Trips per pickup zone -- MAPPER (Hadoop Streaming).

Input lines are headerless taxi CSV rows (labs/datasets/parquet_to_csv.py):
  0 VendorID, 1 tpep_pickup_datetime, 2 tpep_dropoff_datetime, 3 passenger_count,
  4 trip_distance, 5 RatecodeID, 6 store_and_fwd_flag, 7 PULocationID,
  8 DOLocationID, 9 payment_type, 10 fare_amount, ... 16 total_amount, ...

Goal: emit "<zone>\t1" per trip.  If the file taxi_zone_lookup.csv is present in
the task's working directory (ship it with  -files .../taxi_zone_lookup.csv),
translate the numeric PULocationID into "Borough/Zone"  -> a MAP-SIDE JOIN.
"""
import csv
import os
import sys

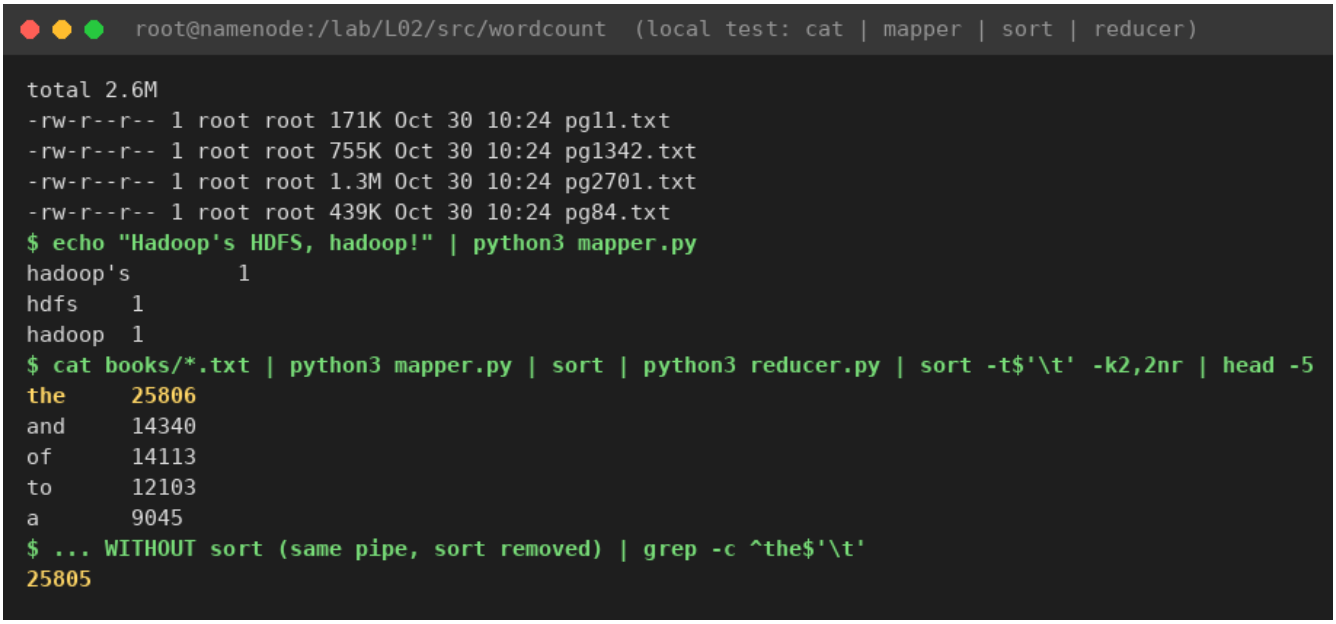
PU_COL = 7

zones = {}
if os.path.exists("taxi_zone_lookup.csv"):
    with open("taxi_zone_lookup.csv", newline="") as f:
        for row in csv.DictReader(f):  # header: LocationID,Borough,Zone,service_zone
            zones[row["LocationID"]] = f"{row['Borough']}/{row['Zone']}"

for line in sys.stdin:
    fields = line.rstrip("\n").split(",")
    if len(fields) <= PU_COL or not fields[PU_COL].strip().isdigit():
        sys.stderr.write("reporter:counter:TaxiLab,BAD_RECORDS,1\n")
        continue
    pu = fields[PU_COL].strip()
    print(f"{zones.get(pu, pu)}\t1")
```

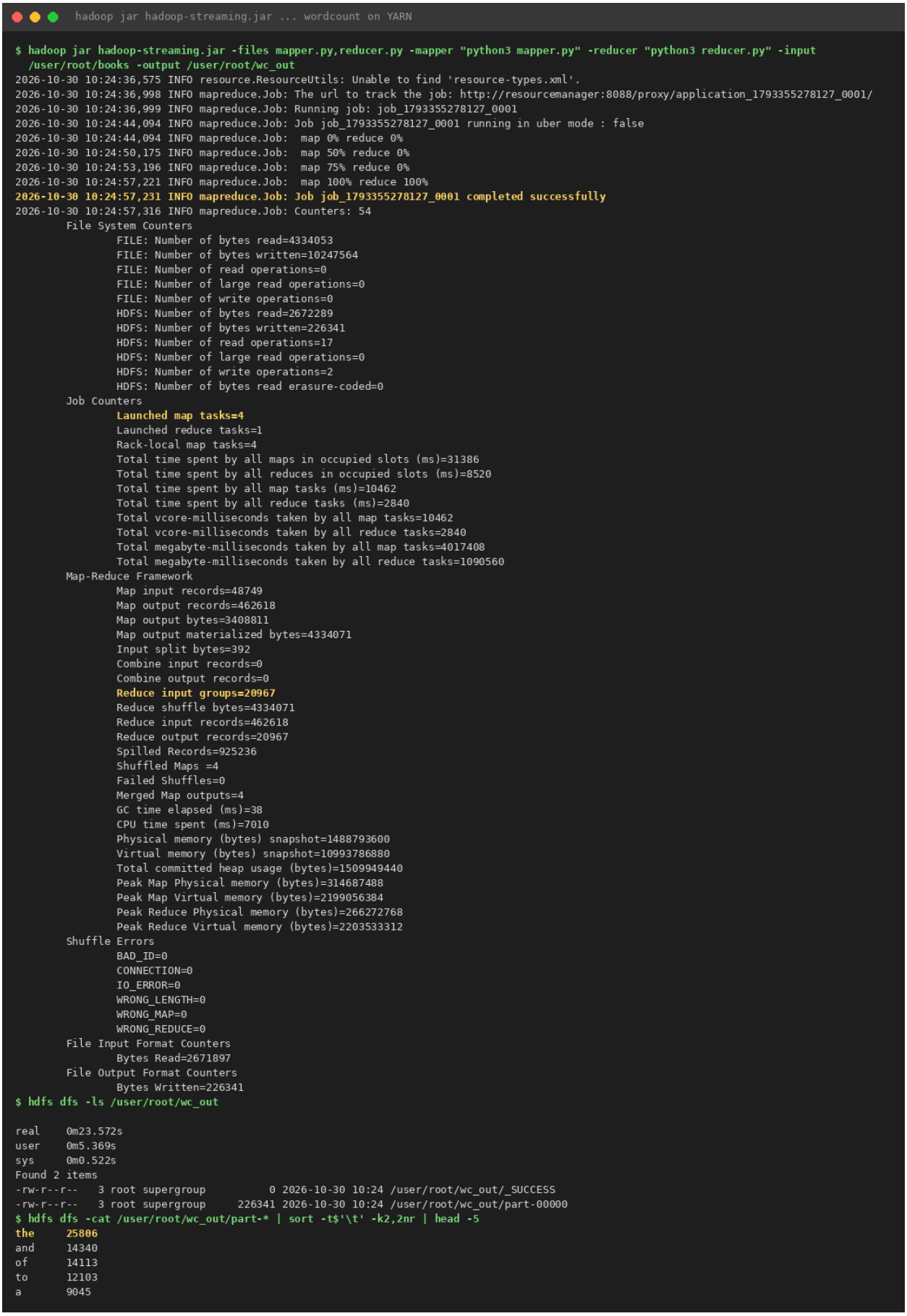
1-2. Word count: test local và chạy trên YARN

Test local (`cat | mapper | sort | reducer`) cho top 5 `the 25806, and 14340, of 14113, to 12103, a 9045`. Khi **bỏ `sort`**, reducer in `the` ra **25,805 dòng** (mỗi lần xuất hiện gần như là một "nhóm" riêng) thay vì 1 dòng tổng.



Hình 9: local pipe

Trên YARN: `Launched map tasks=4` (mỗi cuốn sách < 1 block là 1 split), `Reduce input groups=20967`; top 5 giống hệt bản local.



Hình 10: wordcount yarn

3–4. Dữ liệu taxi CSV và map-side join

1,000,000 dòng (92.8 MB) đưa lên HDFS với block 16 MB → **6 block → 6 map task**. Test local trên 1,000 dòng: `Manhattan/East Village 59`, `Manhattan/Lincoln Square East 55`. Dòng hỏng được đếm qua stderr `reporter:counter:TaxiLab,BAD_RECORDS,1` (dữ liệu thật có 0 dòng hỏng nên counter không xuất hiện trong job).

```
parquet_to_csv (1M rows) -> HDFS 16 MB blocks; zones mapper local test

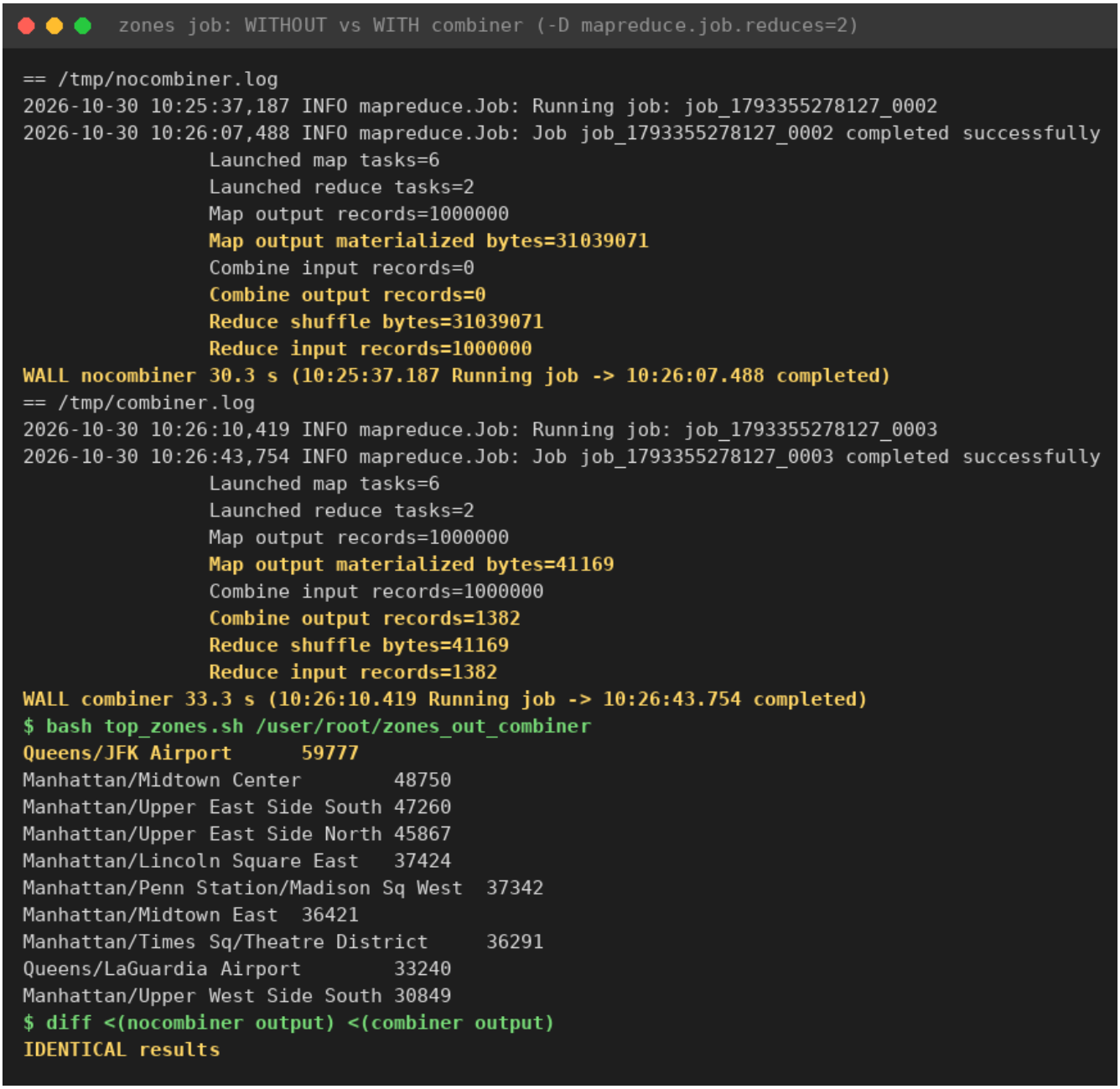
wrote 1,000,000 rows -> /lab/L02/work/yellow_2024-01_1M.csv (92.8 MB)
$ head -2 yellow_2024-01_1M.csv
2,2024-01-01 00:57:55,2024-01-01 01:17:43,1,1.72,1,N,186,79,2,17.7,1,0.5,0,0,1,22.7,2.5,0
1,2024-01-01 00:03:00,2024-01-01 00:09:36,1,1.8,1,N,140,236,1,10,3.5,0.5,3.75,0,1,18.75,2.5,0
$ hdfs fsck /user/root/taxi_csv -files -blocks | grep "block(s)"
/user/root/taxi_csv/yellow_2024-01_1M.csv 92847108 bytes, replicated: replication=3, 6 block(s):  OK
$ head -1000 yellow_2024-01_1M.csv | python3 mapper.py | sort | python3 reducer.py | sort -t$'\t' -k2,2nr | head -3
Manhattan/East Village  59
Manhattan/Lincoln Square East   55
Manhattan/Upper West Side South 45
$ printf "garbage\nx,y,z,a,b,c,d,NOTNUM,1\n" | python3 mapper.py   # bad records -> stderr counter
reporter:counter:TaxiLab,BAD_RECORDS,1
reporter:counter:TaxiLab,BAD_RECORDS,1
```

Hình 11: taxi csv

5. Combiner và không combiner

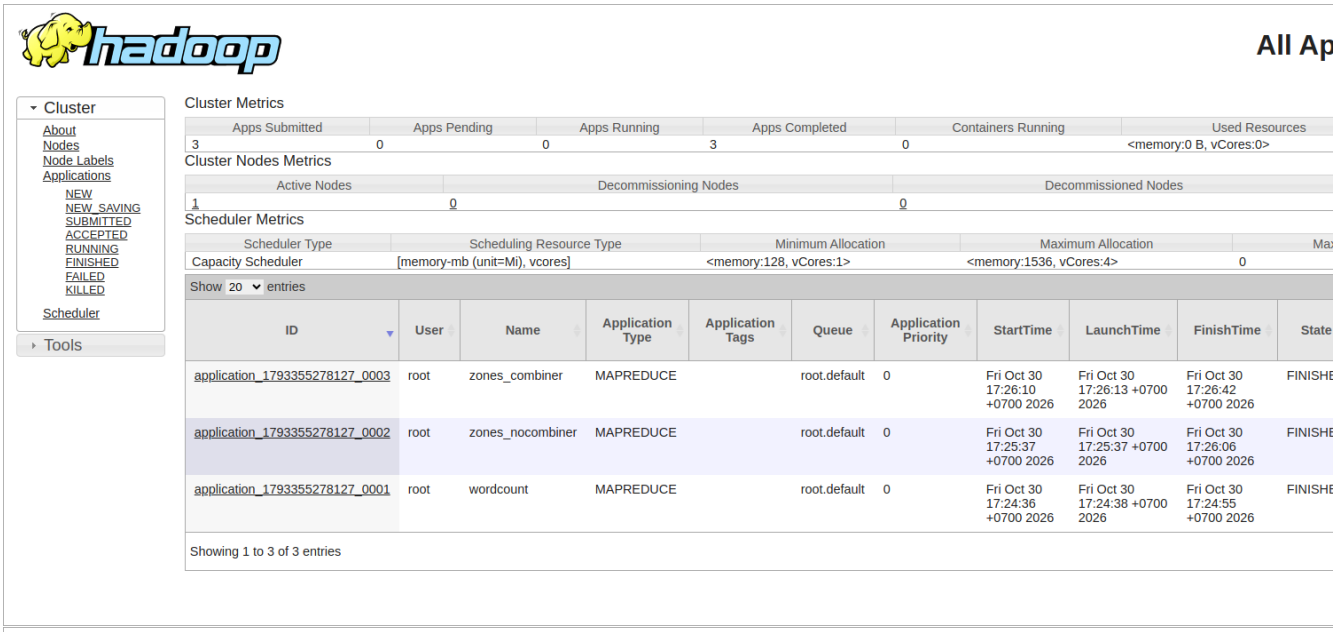
Counter	Không combiner	Có combiner
Launched map tasks	6	6
Map output records	1,000,000	1,000,000
Combine output records	0	1,382
Map output materialized bytes	31,039,071	41,169
Reduce shuffle bytes	31,039,071	41,169
Reduce input records	1,000,000	1,382
Wall-clock time	30.3 s	33.3 s

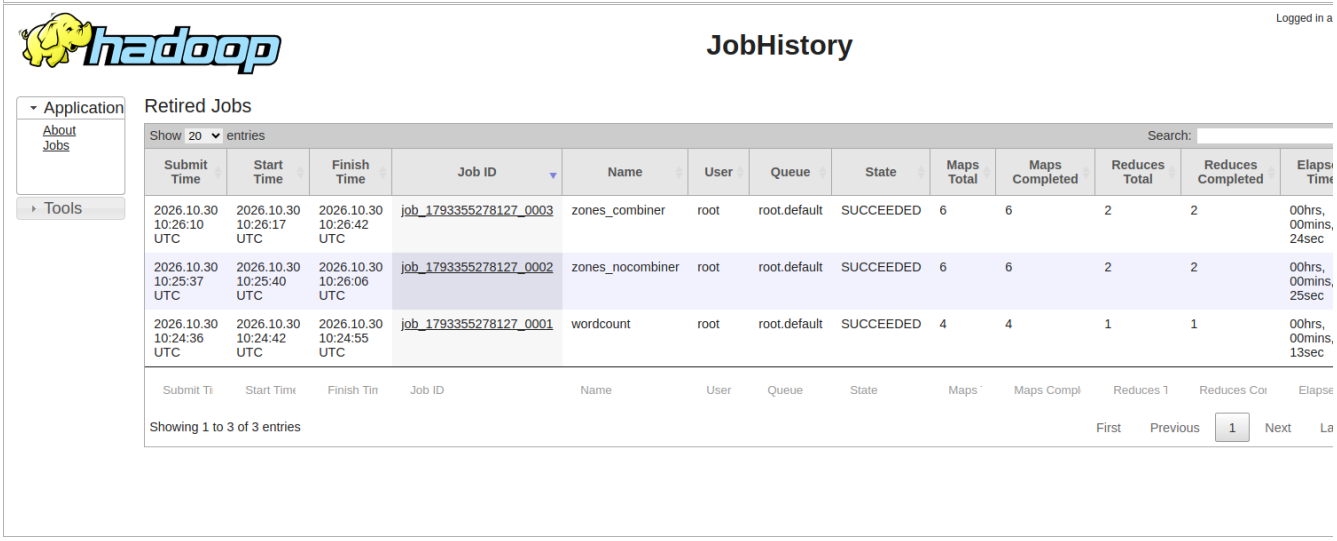
Kết quả hai lần chạy **giống hệt** (`diff` rỗng). Top zone: `Queens/JFK Airport 59777`, `Manhattan/Midtown Center 48750`.

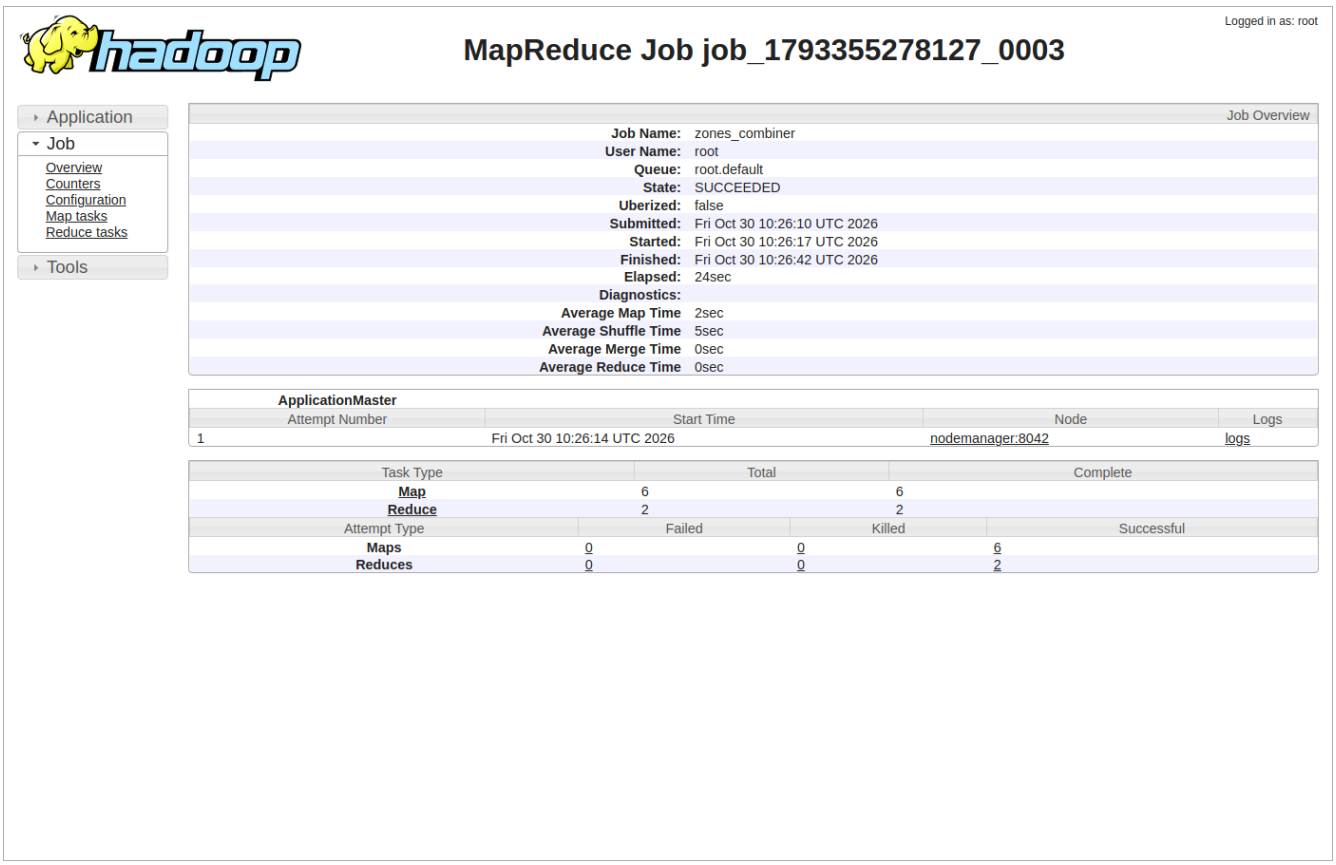


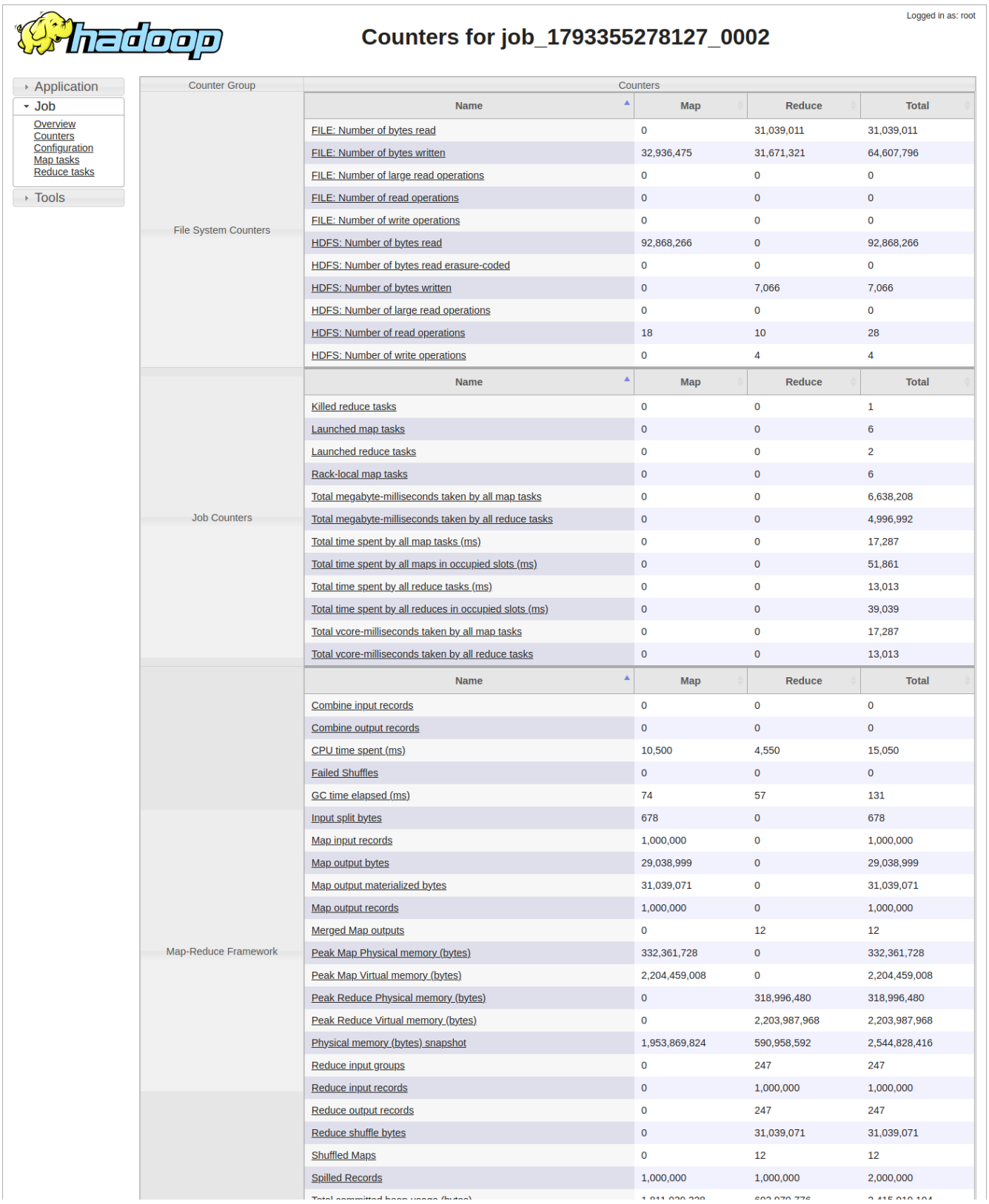
Hình 12: combiner table

6. YARN và JobHistory UI






Logged in as: root
MapReduce Job job_1793355278127_0003
Application
Job
Overview
Counters
Configuration
Map tasks
Reduce tasks
Tools
Job Overview
Job Name: zones_combiner
User Name: root
Queue: root.default
State: SUCCEEDED
Uberized: false
Submitted: Fri Oct 30 10:26:10 UTC 2026
Started: Fri Oct 30 10:26:17 UTC 2026
Finished: Fri Oct 30 10:26:42 UTC 2026
Elapsed: 24sec
Diagnostics:
Average Map Time 2sec
Average Shuffle Time 5sec
Average Merge Time 0sec
Average Reduce Time 0sec
ApplicationMaster
Attempt Number Start Time Node Logs
1 Fri Oct 30 10:26:14 UTC 2026 nodemanager:8042 logs
Task Type Total Complete
Map 6 6
Reduce 2 2
Attempt Type Failed Killed Successful
Maps 0 0 6
Reduces 0 0 2



Logged in as: root

Counters for job_1793355278127_0002

- Application
- Job
  - Overview
  - Counters
  - Configuration
  - Map tasks
  - Reduce tasks
- Tools

Counter Group	Counters			
	Name	Map	Reduce	Total
File System Counters	FILE: Number of bytes read	0	31,039,011	31,039,011
	FILE: Number of bytes written	32,936,475	31,671,321	64,607,796
	FILE: Number of large read operations	0	0	0
	FILE: Number of read operations	0	0	0
	FILE: Number of write operations	0	0	0
	HDFS: Number of bytes read	92,868,266	0	92,868,266
	HDFS: Number of bytes read erasure-coded	0	0	0
	HDFS: Number of bytes written	0	7,066	7,066
	HDFS: Number of large read operations	0	0	0
	HDFS: Number of read operations	18	10	28
	HDFS: Number of write operations	0	4	4
	Name	Map	Reduce	Total
Job Counters	Killed reduce tasks	0	0	1
	Launched map tasks	0	0	6
	Launched reduce tasks	0	0	2
	Rack-local map tasks	0	0	6
	Total megabyte-milliseconds taken by all map tasks	0	0	6,638,208
	Total megabyte-milliseconds taken by all reduce tasks	0	0	4,996,992
	Total time spent by all map tasks (ms)	0	0	17,287
	Total time spent by all maps in occupied slots (ms)	0	0	51,861
	Total time spent by all reduce tasks (ms)	0	0	13,013
	Total time spent by all reduces in occupied slots (ms)	0	0	39,039
	Total vcore-milliseconds taken by all map tasks	0	0	17,287
	Total vcore-milliseconds taken by all reduce tasks	0	0	13,013
	Name	Map	Reduce	Total
Map-Reduce Framework	Combine input records	0	0	0
	Combine output records	0	0	0
	CPU time spent (ms)	10,500	4,550	15,050
	Failed Shuffles	0	0	0
	GC time elapsed (ms)	74	57	131
	Input split bytes	678	0	678
	Map input records	1,000,000	0	1,000,000
	Map output bytes	29,038,999	0	29,038,999
	Map output materialized bytes	31,039,071	0	31,039,071
	Map output records	1,000,000	0	1,000,000
	Merged Map outputs	0	12	12
	Peak Map Physical memory (bytes)	332,361,728	0	332,361,728
	Peak Map Virtual memory (bytes)	2,204,459,008	0	2,204,459,008
	Peak Reduce Physical memory (bytes)	0	318,996,480	318,996,480
	Peak Reduce Virtual memory (bytes)	0	2,203,987,968	2,203,987,968
	Physical memory (bytes) snapshot	1,953,869,824	590,958,592	2,544,828,416
	Reduce input groups	0	247	247
	Reduce input records	0	1,000,000	1,000,000
	Reduce output records	0	247	247
	Reduce shuffle bytes	0	31,039,071	31,039,071
	Shuffled Maps	0	12	12
	Spilled Records	1,000,000	1,000,000	2,000,000
	Total committed heap usage (bytes)	1,811,939,328	603,979,776	2,415,919,104

Logged in as: root

Counters for job_1793355278127_0003

- Application
- Job
  - Overview
  - Counters
  - Configuration
  - Map tasks
  - Reduce tasks
- Tools

Counter Group	Counters			
	Name	Map	Reduce	Total
File System Counters	FILE: Number of bytes read	0	41,109	41,109
	FILE: Number of bytes written	1,940,751	674,145	2,614,896
	FILE: Number of large read operations	0	0	0
	FILE: Number of read operations	0	0	0
	FILE: Number of write operations	0	0	0
	HDFS: Number of bytes read	92,868,266	0	92,868,266
	HDFS: Number of bytes read erasure-coded	0	0	0
	HDFS: Number of bytes written	0	7,066	7,066
	HDFS: Number of large read operations	0	0	0
	HDFS: Number of read operations	18	10	28
	HDFS: Number of write operations	0	4	4
Job Counters	Name	Map	Reduce	Total
	Launched map tasks	0	0	6
	Launched reduce tasks	0	0	2
	Rack-local map tasks	0	0	6
	Total megabyte-milliseconds taken by all map tasks	0	0	6,680,832
	Total megabyte-milliseconds taken by all reduce tasks	0	0	4,359,552
	Total time spent by all map tasks (ms)	0	0	17,398
	Total time spent by all maps in occupied slots (ms)	0	0	52,194
	Total time spent by all reduce tasks (ms)	0	0	11,353
	Total time spent by all reduces in occupied slots (ms)	0	0	34,059
	Total vcore-milliseconds taken by all map tasks	0	0	17,398
	Total vcore-milliseconds taken by all reduce tasks	0	0	11,353
Map-Reduce Framework	Name	Map	Reduce	Total
	Combine input records	1,000,000	0	1,000,000
	Combine output records	1,382	0	1,382
	CPU time spent (ms)	12,380	1,640	14,020
	Failed Shuffles	0	0	0
	GC time elapsed (ms)	72	26	98
	Input split bytes	678	0	678
	Map input records	1,000,000	0	1,000,000
	Map output bytes	29,038,999	0	29,038,999
	Map output materialized bytes	41,169	0	41,169
	Map output records	1,000,000	0	1,000,000
	Merged Map outputs	0	12	12
	Peak Map Physical memory (bytes)	328,896,512	0	328,896,512
	Peak Map Virtual memory (bytes)	2,201,120,768	0	2,201,120,768
	Peak Reduce Physical memory (bytes)	0	251,559,936	251,559,936
	Peak Reduce Virtual memory (bytes)	0	2,205,458,432	2,205,458,432
	Physical memory (bytes) snapshot	1,954,152,448	501,567,488	2,455,719,936
	Reduce input groups	0	247	247
	Reduce input records	0	1,382	1,382
	Reduce output records	0	247	247
	Reduce shuffle bytes	0	41,169	41,169
	Shuffled Maps	0	12	12
	Spilled Records	1,382	1,382	2,764
	Total committed heap usage (bytes)	1,811,939,328	603,979,776	2,415,919,104
	Virtual memory (bytes) snapshot	13,186,910,208	4,407,558,144	17,594,477,440

yarn application -list / mapred job -list all

```
$ yarn application -list -appStates FINISHED
Total number of applications (application-types: [], states: [FINISHED] and tags: []):3
                Application-Id      Application-Name        Application-Type          User           Queue                   State             Final-State             Progress
  Tracking-URL
application_1793355278127_0003        zones_combiner               MAPREDUCE          root      root.default             FINISHED               SUCCEEDED                100%
  http://historyserver:19888/jobhistory/job/job_1793355278127_0003
application_1793355278127_0001             wordcount               MAPREDUCE          root      root.default             FINISHED               SUCCEEDED                100%
  http://historyserver:19888/jobhistory/job/job_1793355278127_0001
application_1793355278127_0002      zones_nocombiner               MAPREDUCE          root      root.default             FINISHED               SUCCEEDED                100%
  http://historyserver:19888/jobhistory/job/job_1793355278127_0002
$ mapred job -list all
Total jobs:3
                  JobId                                JobName            State           StartTime      UserName           Queue      Priority      UsedContainers  RsvdC
 job_1793355278127_0003                        zones_combiner         SUCCEEDED       1793355970353          root    root.default       DEFAULT                 N/A
 job_1793355278127_0001                             wordcount         SUCCEEDED       1793355876697          root    root.default       DEFAULT                 N/A
 job_1793355278127_0002                       zones_nocombiner         SUCCEEDED       1793355937120          root    root.default       DEFAULT                 N/A
```

Stretch (bonus): giá cước trung bình theo zone với combiner đúng

Mapper emit `zone \t fare,1`; combiner cộng dồn `(sum, count)` và xuất **cùng định dạng** với input; reducer cộng tiếp rồi mới chia `sum/count`. Kết quả JFK `59.71` trên `59,777` chuyến, trùng với bản chạy local không combiner, nghĩa là combiner không làm sai kết quả. Shuffle chỉ còn 56,289 byte.

stretch: average fare per zone with a correct (sum,count) combiner

```
$ hadoop jar streaming -mapper mapper.py -combiner combiner.py -reducer reducer.py -input /user/root/taxi_csv -output /user/root/avg_fare_out
2026-10-30 10:28:00,220 INFO mapreduce.Job: Job job_1793355278127_0004 completed successfully
                Combine input records=1000000
                Combine output records=1382
                Reduce shuffle bytes=56289
$ top 10 zones by trips: zone / avg_fare / trips
Queens/JFK Airport      59.71   59777
Manhattan/Midtown Center        15.33   48750
Manhattan/Upper East Side South 11.91   47260
Manhattan/Upper East Side North 12.49   45867
Manhattan/Lincoln Square East   13.30   37424
Manhattan/Penn Station/Madison Sq West  15.72   37342
Manhattan/Midtown East  14.71   36421
Manhattan/Times Sq/Theatre District     18.20   36291
Queens/LaGuardia Airport        40.87   33240
Manhattan/Upper West Side South 13.36   30849
$ check vs local (no combiner) for JFK:
Queens/JFK Airport      59.71   59777
```

stretch_avg_fare/combiner.py

```
#!/usr/bin/env python3
"""Average fare -- COMBINER: adds up (sum, count) pairs per key. Output has the same shape as its
input, so it may run 0, 1 or many times without changing the final result."""
import sys

current_key, total, count = None, 0.0, 0

for line in sys.stdin:
    key, _, value = line.rstrip("\n").partition("\t")
    s, _, c = value.partition(",")
    if key == current_key:
        total, count = total + float(s), count + int(c)
    else:
        if current_key is not None:
            print(f"{current_key}\t{total},{count}")
        current_key, total, count = key, float(s), int(c)

if current_key is not None:
    print(f"{current_key}\t{total},{count}")
```

Trả lời checkpoint

Q1. Lệnh `sort` đóng vai trò **shuffle & sort**: nó gom các key giống nhau nằm liền nhau. Thiếu nó, reducer thấy cùng một từ ở nhiều đoạn không liền nhau nên in ra nhiều lần với số đếm cục bộ (thực nghiệm: `the` bị in 25,805 lần).

Q2. Streaming chỉ đưa cho reducer một **luồng dòng đã sort**; khái niệm "nhóm" `(key, Iterable<values>)` nằm trong framework Java. Reducer Python phải tự phát hiện ranh giới key (`current_key`) và nhớ in key cuối cùng.

Q3. 6 map task = 6 input split = 6 block HDFS 16 MB của file CSV 92.8 MB (mặc định mỗi block là một split). Wordcount có 4 map vì mỗi cuốn sách nhỏ hơn 1 block nên là 1 split. **2 reduce task** vì đặt `-D mapreduce.job.reduces=2` (mặc định là 1): số reducer do người dùng chọn, không suy ra từ dữ liệu.

Q4. Shuffle giảm **~754 lần** (31,039,071 → 41,169 byte). Mỗi map task chỉ xuất một bản ghi cho mỗi **zone khác nhau mà nó gặp** (~230–260 zone) × 6 map ≈ 1,382. Chỉ các tổng cục bộ này đi qua mạng.

Q5. Combiner không nhanh hơn (33.3 s so với 30.3 s) vì trên một máy, shuffle chỉ là copy qua đĩa/loopback, gần như miễn phí, trong khi combiner thêm một process Python phải đọc, parse và sort lại 1 triệu bản ghi ở mỗi map. Trên cluster thật, shuffle đi qua mạng và reducer phải merge-sort hàng GB từ hàng trăm mapper; khi đó giảm 99.9 % lượng shuffle sẽ áp đảo chi phí trên và combiner thắng rõ.

Q6. Không. Trung bình của các trung bình là sai vì mỗi partition có số bản ghi khác nhau. Combiner phải emit cặp `(sum, count)` cục bộ và reducer mới tính `sum/count` (đã cài đặt ở phần stretch). Combiner phải có tính kết hợp và giao hoán, output cùng định dạng với input, vì Hadoop có thể chạy nó 0, 1 hoặc nhiều lần.

Q7. Mỗi mapper nạp file nhỏ `taxi_zone_lookup.csv` (265 dòng, phân phối bằng distributed cache `-files`) vào một dict và thay ID bằng tên zone, nên join không cần shuffle. Cách này hết hợp lý khi bảng "nhỏ" không còn vừa thoải mái trong bộ nhớ của từng task; khi đó phải dùng reduce-side join (shuffle cả hai tập theo key). Spark gọi ý tưởng này là **broadcast join** (L06).

Q8. `BAD_RECORDS` xuất hiện trong job counters (console cuối job và JobHistory UI → Counters → nhóm `TaxiLab`); chỉ xuất hiện khi có ít nhất một dòng hỏng. Vài dòng lỗi không nên làm chết cả job xử lý hàng tỷ dòng; đếm chúng giúp lộ ra vấn đề chất lượng dữ liệu và cho phép đặt ngưỡng cảnh báo.

L03 — Hive: SQL trên HDFS, partitioning, định dạng file, bucketing

Môi trường: Hive 4.2.1 (HiveServer2 + Metastore độc lập, Tez local mode), Metastore DB PostgreSQL 18, HDFS 3.4.3. Dữ liệu: **cả tháng 01/2024** (2,964,624 dòng, CSV 274.2 MB), không dùng mẫu 1 triệu dòng. Log gốc: các file `0*.txt`, `1*.txt` (đọc nhanh bằng `./clean.sh <file>`); file SQL đã hoàn thành nằm trong thư mục này; ảnh ở `img/`.

File SQL đã hoàn thành

File	TODO
`01_external_tables.sql`	TODO 1: `zones` dùng `OpenCSVSerde`, `LOCATION '/data/zones'`, `skip.header.line.count=1`
`02_partitioned_tables.sql`	TODO 2: `hive.exec.dynamic.partition.mode=nonstrict;` TODO 3: `date_format(tpep_pickup_datetime,'yyyy-MM');` TODO 4: `trips_parquet` (Parquet + Snappy)
`04_partition_pruning.sql`	TODO 5: `EXPLAIN DEPENDENCY` với filter trên `tpep_pickup_datetime`
`05_bucketing.sql`	TODO 6: `CLUSTERED BY (pulocationid) SORTED BY (pulocationid) INTO 4 BUCKETS`

sql/01_external_tables.sql (đã hoàn thành)

```
-- L03 step 2: external tables over files that already sit in HDFS.
-- Hive only stores METADATA (schema + location) in the metastore; DROP TABLE keeps the files.
CREATE DATABASE IF NOT EXISTS taxi;
USE taxi;

DROP TABLE IF EXISTS trips_csv;
CREATE EXTERNAL TABLE trips_csv (
  vendorid              INT,
  tpep_pickup_datetime  TIMESTAMP,
  tpep_dropoff_datetime TIMESTAMP,
  passenger_count       BIGINT,
  trip_distance         DOUBLE,
  ratecodeid            BIGINT,
  store_and_fwd_flag    STRING,
  pulocationid          INT,
  dolocationid          INT,
  payment_type          BIGINT,
  fare_amount           DOUBLE,
  extra                 DOUBLE,
  mta_tax               DOUBLE,
  tip_amount            DOUBLE,
  tolls_amount          DOUBLE,
  improvement_surcharge DOUBLE,
  total_amount          DOUBLE,
  congestion_surcharge  DOUBLE,
  airport_fee           DOUBLE
)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
STORED AS TEXTFILE
LOCATION '/data/taxi_csv';

-- The zone lookup has a header line and "quoted","values" -> OpenCSVSerde
-- (OpenCSVSerde reads every column as STRING; we cast when querying).
DROP TABLE IF EXISTS zones;
CREATE EXTERNAL TABLE zones (
  locationid   STRING,
  borough      STRING,
  zone         STRING,
  service_zone STRING
)
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'
STORED AS TEXTFILE
LOCATION '/data/zones'
TBLPROPERTIES ('skip.header.line.count' = '1');

SHOW TABLES;
SELECT * FROM trips_csv LIMIT 3;
SELECT * FROM zones LIMIT 3;
SELECT count(*) AS csv_rows FROM trips_csv;
```

sql/02_partitioned_tables.sql (đã hoàn thành)

```
-- L03 step 3: managed, columnar, partitioned copies of the CSV data.
USE taxi;
SET hive.exec.dynamic.partition = true;
-- TODO 2: dynamic partitioning is "strict" by default (at least one static partition value
--         required). Set hive.exec.dynamic.partition.mode so ALL partition values can be dynamic.
SET hive.exec.dynamic.partition.mode = nonstrict;


DROP TABLE IF EXISTS trips_orc;
CREATE TABLE trips_orc (
  vendorid INT, tpep_pickup_datetime TIMESTAMP, tpep_dropoff_datetime TIMESTAMP,
  passenger_count BIGINT, trip_distance DOUBLE, ratecodeid BIGINT, store_and_fwd_flag STRING,
  pulocationid INT, dolocationid INT, payment_type BIGINT, fare_amount DOUBLE, extra DOUBLE,
  mta_tax DOUBLE, tip_amount DOUBLE, tolls_amount DOUBLE, improvement_surcharge DOUBLE,
  total_amount DOUBLE, congestion_surcharge DOUBLE, airport_fee DOUBLE
)
PARTITIONED BY (pickup_month STRING)
STORED AS ORC
TBLPROPERTIES ('orc.compress' = 'ZLIB');

-- Dynamic partitioning: the LAST column of the SELECT feeds the partition column.
INSERT OVERWRITE TABLE trips_orc PARTITION (pickup_month)
SELECT t.*,
       date_format(t.tpep_pickup_datetime, 'yyyy-MM') AS pickup_month
FROM trips_csv t;

-- TODO 4: create trips_parquet: same columns and partitioning as trips_orc, STORED AS PARQUET
--         (TBLPROPERTIES ('parquet.compression' = 'SNAPPY')), and fill it the same way.
DROP TABLE IF EXISTS trips_parquet;
CREATE TABLE trips_parquet (
  vendorid INT, tpep_pickup_datetime TIMESTAMP, tpep_dropoff_datetime TIMESTAMP,
  passenger_count BIGINT, trip_distance DOUBLE, ratecodeid BIGINT, store_and_fwd_flag STRING,
  pulocationid INT, dolocationid INT, payment_type BIGINT, fare_amount DOUBLE, extra DOUBLE,
  mta_tax DOUBLE, tip_amount DOUBLE, tolls_amount DOUBLE, improvement_surcharge DOUBLE,
  total_amount DOUBLE, congestion_surcharge DOUBLE, airport_fee DOUBLE
)
PARTITIONED BY (pickup_month STRING)
STORED AS PARQUET
TBLPROPERTIES ('parquet.compression' = 'SNAPPY');

INSERT OVERWRITE TABLE trips_parquet PARTITION (pickup_month)
SELECT t.*, date_format(t.tpep_pickup_datetime, 'yyyy-MM') AS pickup_month
FROM trips_csv t;


SHOW PARTITIONS trips_orc;
SELECT pickup_month, count(*) AS trips FROM trips_orc GROUP BY pickup_month ORDER BY pickup_month;
```

sql/04_partition_pruning.sql (đã hoàn thành)

```
-- L03 step 5: does Hive read all partitions or only the ones we need?
USE taxi;

-- (a) filter on the PARTITION column -> only 1 partition is an input
EXPLAIN DEPENDENCY
SELECT count(*) FROM trips_orc WHERE pickup_month = '2024-01';

-- (b) filter on a normal column -> every partition is an input
-- TODO 5: write the same count for January 2024, but filter on tpep_pickup_datetime
--         instead of pickup_month, prefixed with EXPLAIN DEPENDENCY. How many partitions are read?
EXPLAIN DEPENDENCY
SELECT count(*) FROM trips_orc
WHERE tpep_pickup_datetime >= '2024-01-01' AND tpep_pickup_datetime < '2024-02-01';


-- (c) the full plan: look for "filterExpr", "Statistics: Num rows" and the partition list
EXPLAIN EXTENDED
SELECT count(*) FROM trips_orc WHERE pickup_month = '2024-01';

-- (d) where does a partition live?
DESCRIBE FORMATTED trips_orc PARTITION (pickup_month = '2024-01');
```

sql/05_bucketing.sql (đã hoàn thành)

```
-- L03 step 6: bucketing = hash(column) mod N -> a fixed number of files per partition/table.
USE taxi;

DROP TABLE IF EXISTS trips_bucketed;
CREATE TABLE trips_bucketed (
  tpep_pickup_datetime TIMESTAMP,
  pulocationid INT,
  dolocationid INT,
  fare_amount DOUBLE,
  total_amount DOUBLE
)
-- TODO 6: bucket by pulocationid into 4 buckets, sorted by pulocationid
CLUSTERED BY (pulocationid) SORTED BY (pulocationid) INTO 4 BUCKETS
STORED AS ORC;

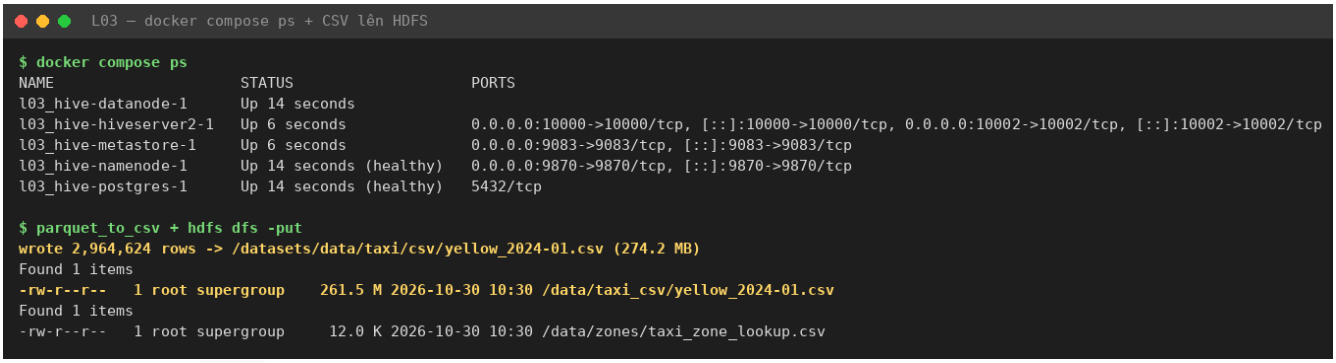
INSERT OVERWRITE TABLE trips_bucketed
SELECT tpep_pickup_datetime, pulocationid, dolocationid, fare_amount, total_amount
FROM trips_orc WHERE pickup_month = '2024-01';

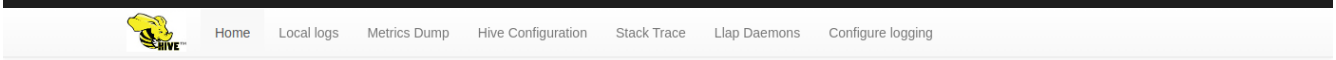
DESCRIBE FORMATTED trips_bucketed;

-- Sampling reads just one bucket (~1/4 of the data):
SELECT count(*) FROM trips_bucketed TABLESAMPLE (BUCKET 1 OUT OF 4 ON pulocationid);
SELECT count(*) FROM trips_bucketed;

-- Every row of zone 132 (JFK) is in exactly one bucket file:
SELECT INPUT__FILE__NAME AS file, count(*) AS rows_for_jfk
FROM trips_bucketed WHERE pulocationid = 132 GROUP BY INPUT__FILE__NAME;
```

1-2. Khởi động stack và đưa CSV lên HDFS





HiveServer2

Active Sessions

User Name	IP Address	Operation Count	Active Time (s)	Idle Time (s)

Total number of sessions: 0

Open Queries

User Name	Query	Execution Engine	State	Opened Timestamp	Opened (s)	Latency (s)	Drilldown Link

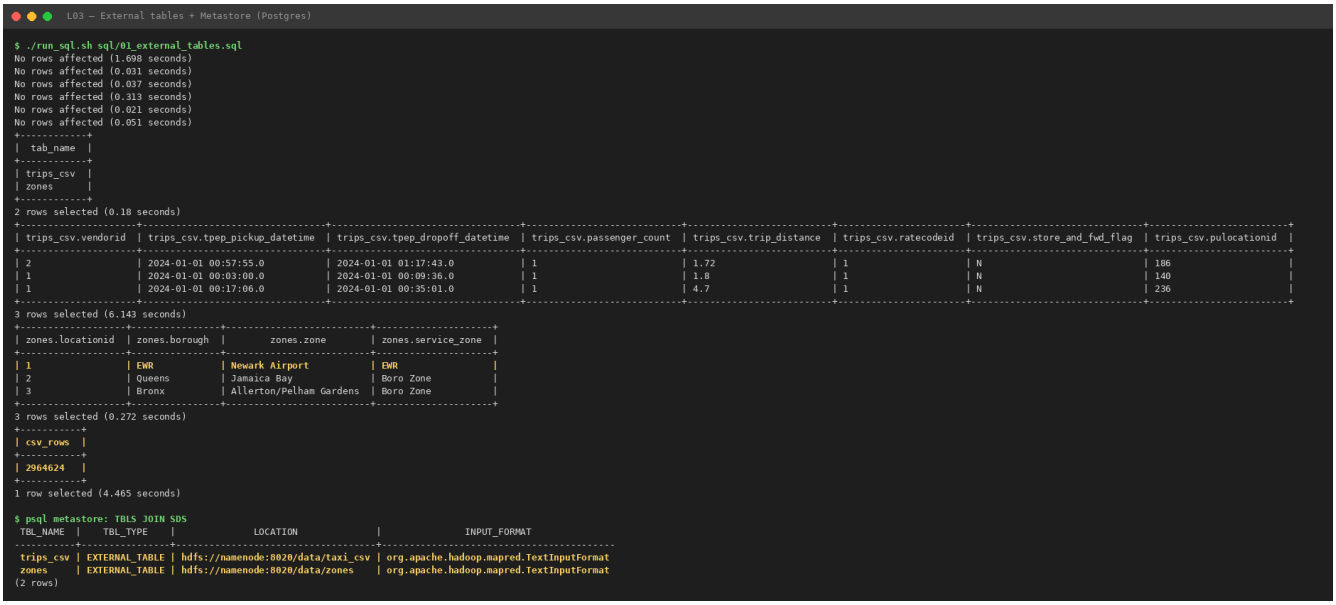
Total number of queries: 0

Last Max 25 Closed Queries

User Name	Query	Execution Engine	State	Opened (s)	Closed Timestamp	Latency (s)	Drilldown Link
hive	SELECT count(*) AS stats_count FROM trips_orc	tez	FINISHED	2	Fri Oct 30 10:36:42 GMT 2026	2	Drilldown
hive	EXPLAIN SELECT count(*) FROM trips_orc	tez	FINISHED	0	Fri Oct 30 10:36:40 GMT 2026	0	Drilldown
hive	USE taxi	tez	FINISHED	0	Fri Oct 30 10:36:40 GMT 2026	0	Drilldown
hive	DESCRIBE FORMATTED zone_notes	tez	FINISHED	0	Fri Oct 30 10:36:22 GMT 2026	0	Drilldown
hive	SELECT * FROM zone_notes ORDER BY locationid	tez	FINISHED	1	Fri Oct 30 10:36:22 GMT 2026	1	Drilldown
hive	DELETE FROM zone_notes WHERE locationid = 161	tez	FINISHED	3	Fri Oct 30 10:36:20 GMT 2026	3	Drilldown
hive	UPDATE zone_notes SET note = 'JFK Airport' WHERE locationid = 132	tez	FINISHED	4	Fri Oct 30 10:36:17 GMT 2026	4	Drilldown
hive	INSERT INTO zone_notes VALUES (132, 'JFK'), (138, 'LaGuardia'), (161, 'Midtown')	tez	FINISHED	3	Fri Oct 30 10:36:13 GMT 2026	3	Drilldown
hive	CREATE TABLE zone_notes (locationid INT, note STRING) STORED AS ORC TBLPROPERTIES ('transactional'='true')	tez	FINISHED	0	Fri Oct 30 10:36:09 GMT 2026	0	Drilldown
hive	DROP TABLE IF EXISTS zone_notes	tez	FINISHED	0	Fri Oct 30 10:36:09 GMT 2026	0	Drilldown
hive	SELECT count(*) AS stats_count FROM trips_orc	tez	FINISHED	1	Fri Oct 30 10:36:09 GMT 2026	1	Drilldown

3. External tables

3 dòng mẫu, 3 zone (`1 | EWR | Newark Airport | EWR...`), `csv_rows = 2,964,624`. Metastore (Postgres) chỉ lưu tên bảng, kiểu bảng, `LOCATION` và `INPUT_FORMAT`:



Hình 13: external + metastore

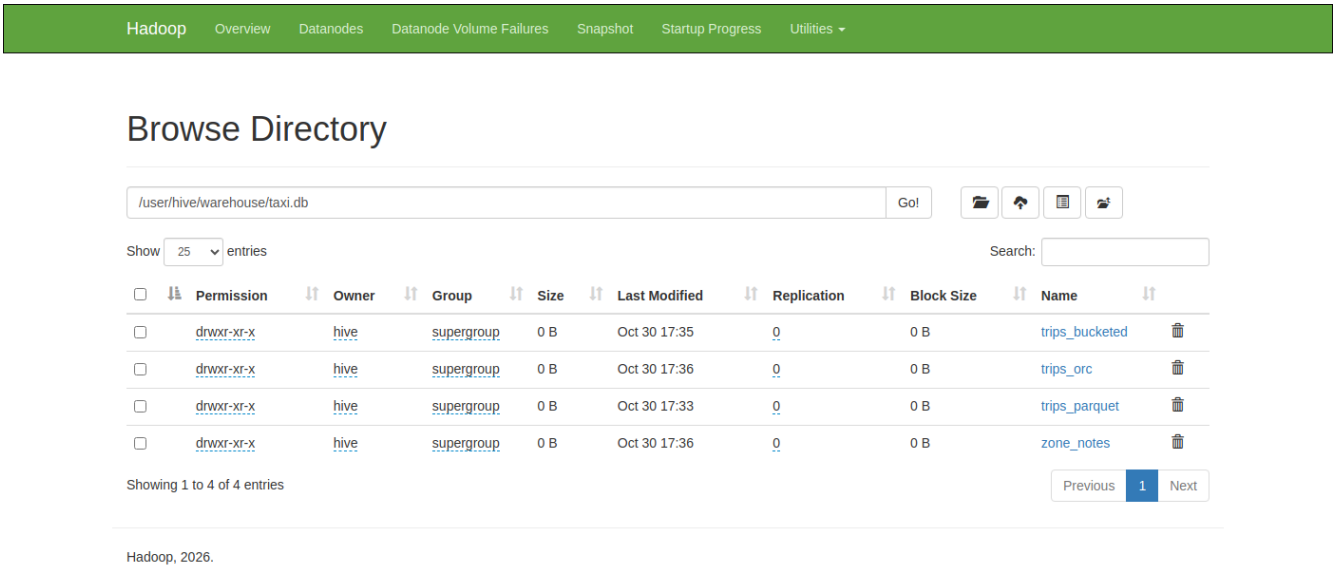
4. Bảng partition ORC và Parquet

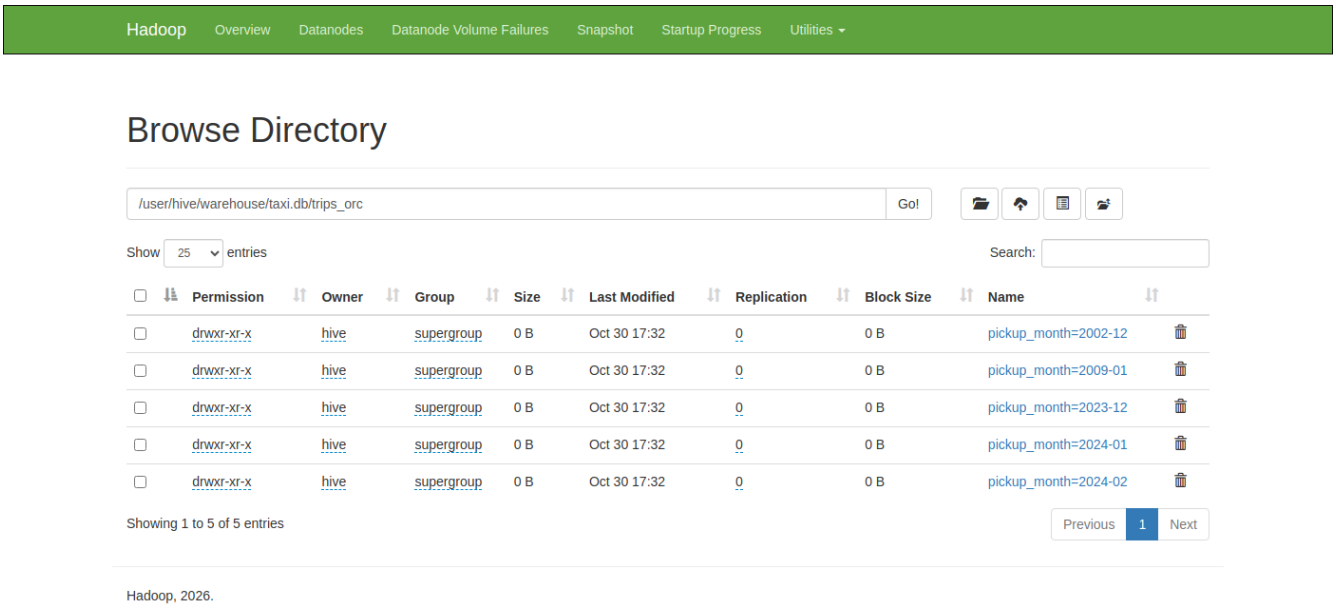
INSERT OVERWRITE: ORC 42.8 s, Parquet 45.1 s (2,964,624 dòng mỗi bảng). `SHOW PARTITIONS` cho **5 partition** (dữ liệu cả tháng có thêm 2024-02 so với mẫu 1 triệu dòng):

pickup_month	trips
2002-12	2
2009-01	3
2023-12	10
2024-01	2,964,606
2024-02	3

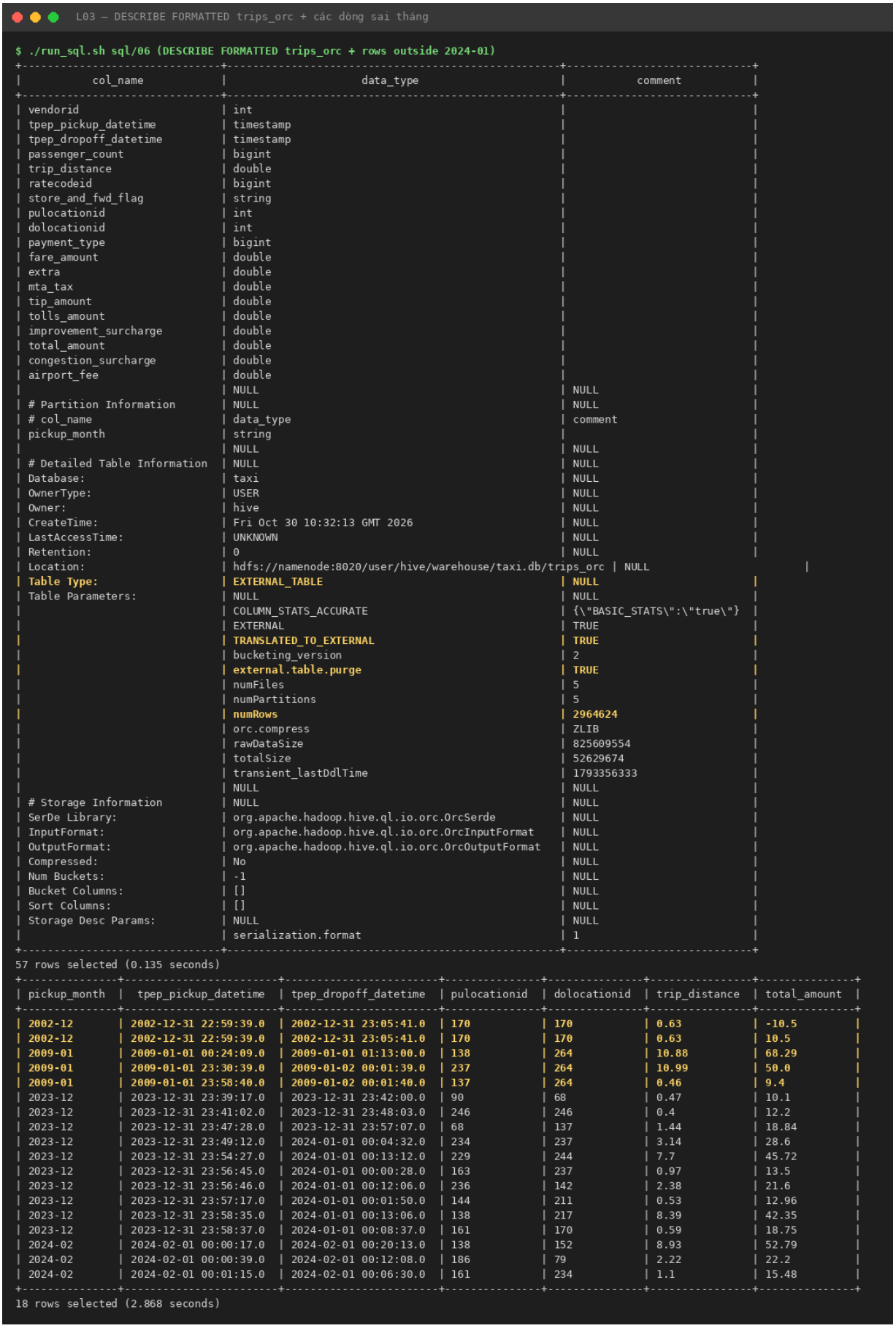
L03 — Dynamic partitioning ORC + Parquet

```
$ ./run_sql.sh sql/02_partitioned_tables.sql
No rows affected (0.06 seconds)
No rows affected (0.006 seconds)
No rows affected (0.004 seconds)
No rows affected (0.017 seconds)
No rows affected (0.07 seconds)
2,964,624 rows affected (42.761 seconds)
No rows affected (0.017 seconds)
No rows affected (0.052 seconds)
2,964,624 rows affected (45.134 seconds)
+------------------------+
|       partition        |
+------------------------+
| pickup_month=2002-12   |
| pickup_month=2009-01   |
| pickup_month=2023-12   |
| pickup_month=2024-01   |
| pickup_month=2024-02   |
+------------------------+
5 rows selected (0.077 seconds)
+---------------+----------+
| pickup_month  |  trips   |
+---------------+----------+
| 2002-12       | 2        |
| 2009-01       | 3        |
| 2023-12       | 10       |
| 2024-01       | 2964606  |
| 2024-02       | 3        |
+---------------+----------+
5 rows selected (2.24 seconds)
```





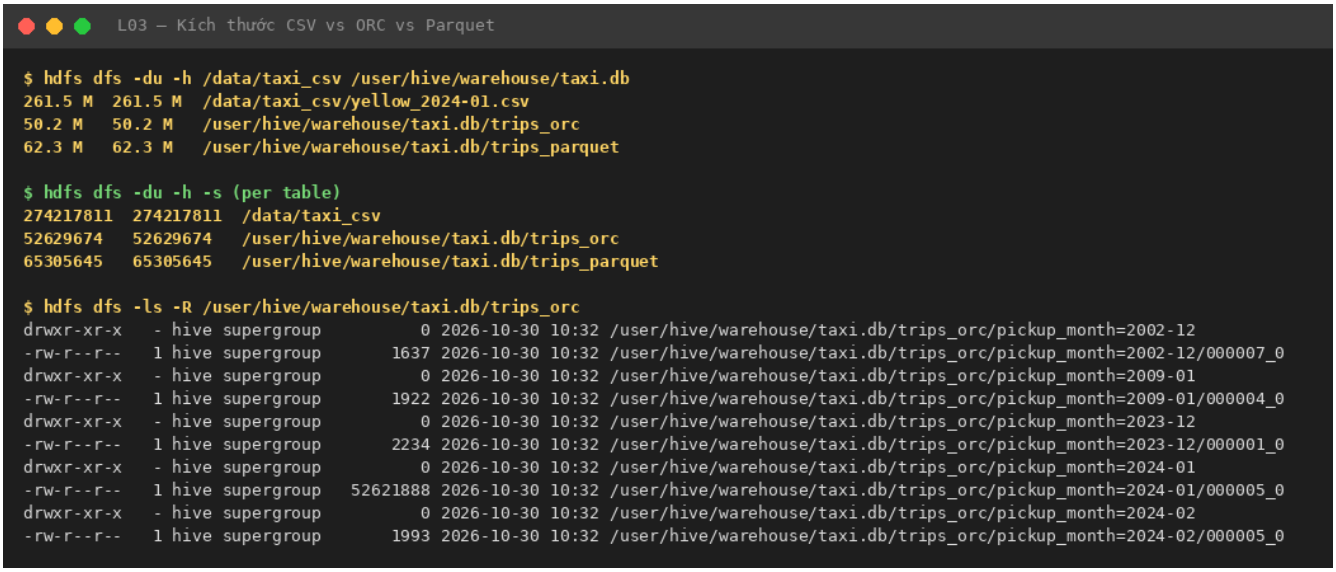
`DESCRIBE FORMATTED trips_orc` cho thấy `Table Type: EXTERNAL_TABLE`, `TRANSLATED_TO_EXTERNAL=TRUE`, `external.table.purge=TRUE` (đúng hành vi Hive 4 với bảng "managed" không phải ACID).



Hình 14: describe + bad rows

Bảng kích thước

Định dạng	Kích thước (cả tháng, 2.96 triệu dòng)	So với CSV
CSV (text)	274,217,811 B (261.5 MiB)	1×
ORC + ZLIB	52,629,674 B (50.2 MiB)	**5.2× nhỏ hơn**
Parquet + Snappy	65,305,645 B (62.3 MiB)	**4.2× nhỏ hơn**
ORC bucketed (chỉ 5 cột)	25,271,072 B (24.1 MiB)	10.9×



Hình 15: sizes

5. CSV, ORC và Parquet: bảng thời gian (lần chạy thứ 2)

Query	CSV	ORC	Parquet
Q1 top pickup zones tháng 01/2024	**8.53 s**	**1.89 s**	**1.62 s**
Q2 join zones theo borough (ORC)		2.66 s	

(Lần chạy 1: 9.22 / 1.99 / 1.77 s.) Q1 dòng đầu: zone 132 (JFK) có **145,240 chuyến**, avg fare **59.40**; ba nguồn cho kết quả giống hệt nhau. Q2: Manhattan 2,646,933 chuyến (**89.3 %**), Queens 273,125, Brooklyn 25,258…

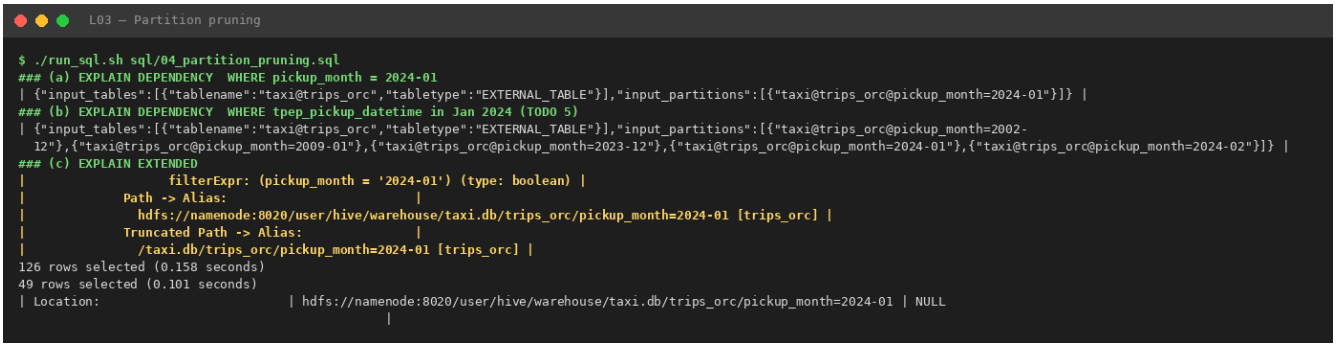
L03 — Q1/Q2 trên CSV vs ORC vs Parquet (lần chạy 2)

```
$ ./run_sql.sh sql/03_compare_formats.sql   # RUN 2 (CSV, ORC, Parquet, join)
+---------------+---------+-----------+
| pulocationid  |  trips  | avg_fare  |
+---------------+---------+-----------+
| 132           | 145240  | 59.4      |
| 161           | 143469  | 15.21     |
| 237           | 142707  | 12.18     |
| 236           | 136464  | 12.71     |
| 162           | 106717  | 14.79     |
+---------------+---------+-----------+
5 rows selected (8.528 seconds)
+---------------+---------+-----------+
| pulocationid  |  trips  | avg_fare  |
+---------------+---------+-----------+
| 132           | 145240  | 59.4      |
| 161           | 143469  | 15.21     |
| 237           | 142707  | 12.18     |
| 236           | 136464  | 12.71     |
| 162           | 106717  | 14.79     |
+---------------+---------+-----------+
5 rows selected (1.885 seconds)
+---------------+---------+-----------+
| pulocationid  |  trips  | avg_fare  |
+---------------+---------+-----------+
| 132           | 145240  | 59.4      |
| 161           | 143469  | 15.21     |
| 237           | 142707  | 12.18     |
| 236           | 136464  | 12.71     |
| 162           | 106717  | 14.79     |
+---------------+---------+-----------+
5 rows selected (1.618 seconds)
+----------------+----------+-----------+
|   z.borough    |  trips   |  revenue  |
+----------------+----------+-----------+
| Manhattan      | 2646933  | 59247250  |
| Queens         | 273125   | 18639629  |
| Brooklyn       | 25258    | 828808    |
| Unknown        | 10360    | 304152    |
| Bronx          | 6905     | 246496    |
| N/A            | 1658     | 154901    |
| EWR            | 295      | 30738     |
| Staten Island  | 72       | 3966      |
+----------------+----------+-----------+
8 rows selected (2.655 seconds)
```

Hình 16: compare

6. Partition pruning

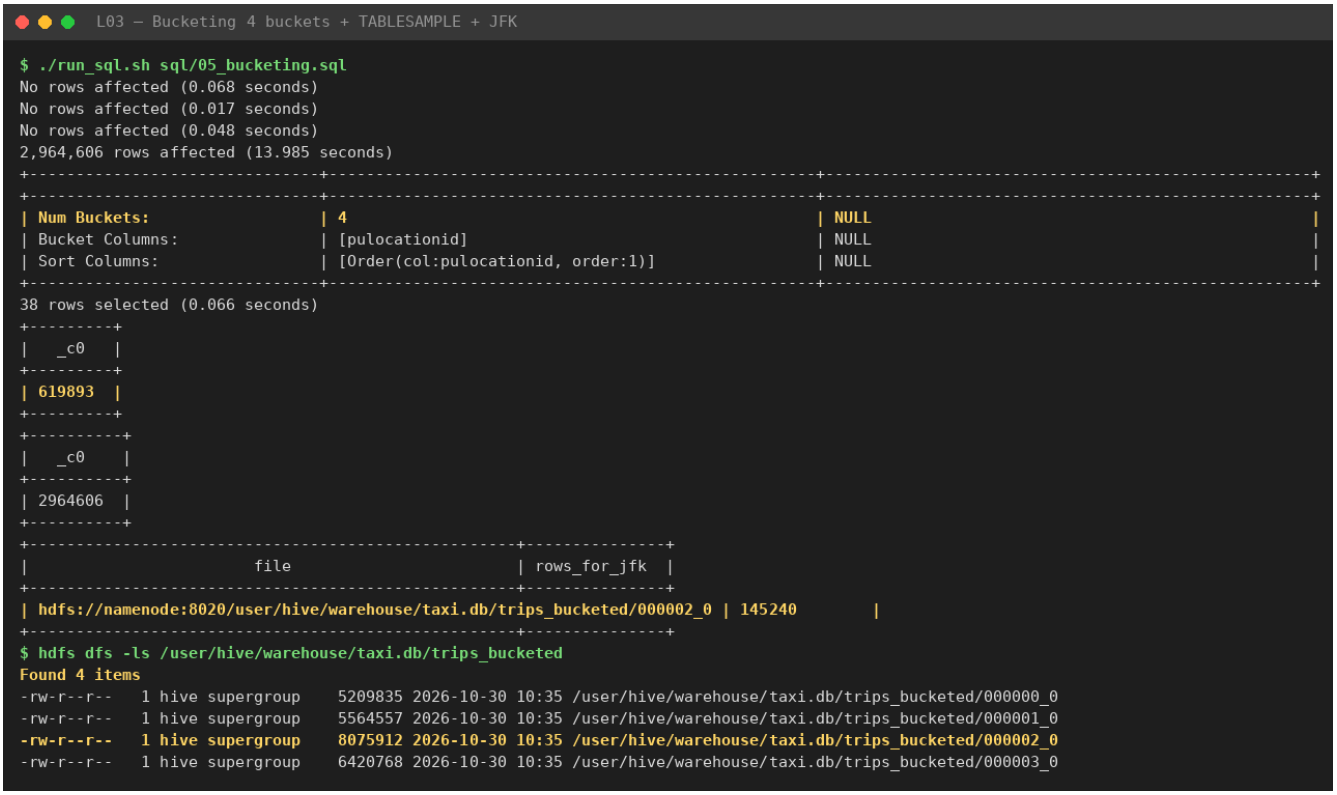
- (a) `WHERE pickup_month = '2024-01'` → `input_partitions` chỉ có **1** partition.
- (b) TODO 5, `WHERE tpep_pickup_datetime` trong tháng 1 → **cả 5** partition là input (Hive không suy được partition từ cột thường).
- (c) `EXPLAIN EXTENDED: filterExpr: (pickup_month = '2024-01'), Path -> Alias` chỉ có `.../trips_orc/pickup_month=2024-01`.

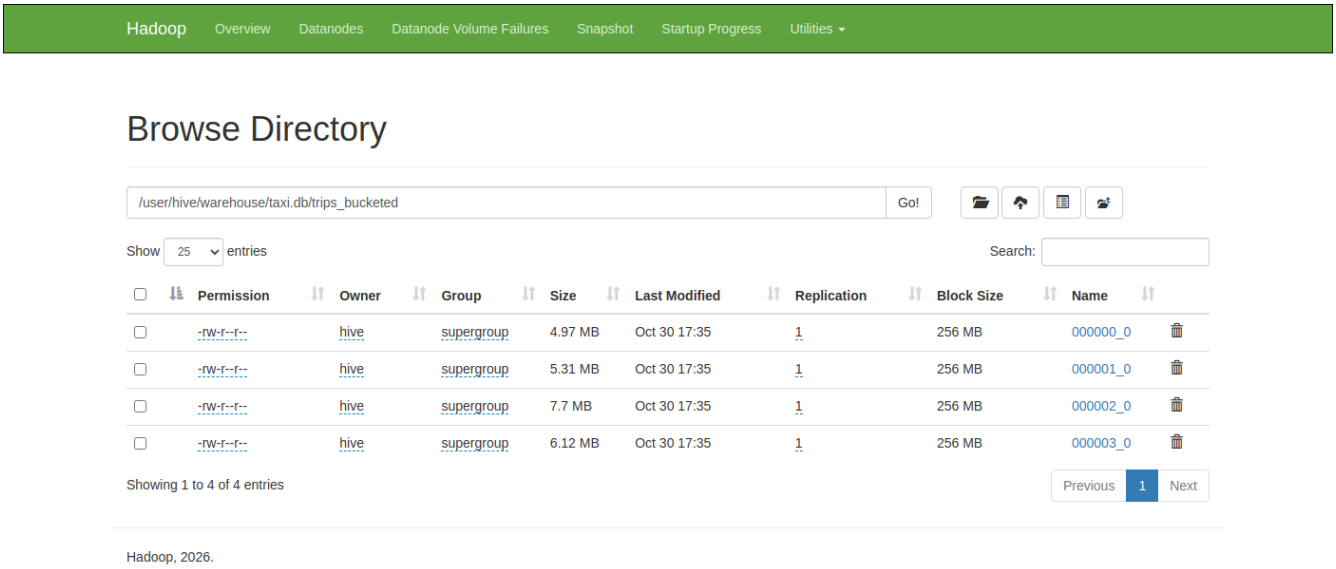


Hình 17: pruning

7. Bucketing

Đúng **4 file** `000000_0 … 000003_0`. `TABLESAMPLE(BUCKET 1 OUT OF 4)` = 619,893 dòng (~21 %, vì bucket theo hash nên không đều). Cả **145,240 dòng JFK** đều nằm trong **một** file `000002_0` (vì vậy file này lớn nhất, 8.1 MB).

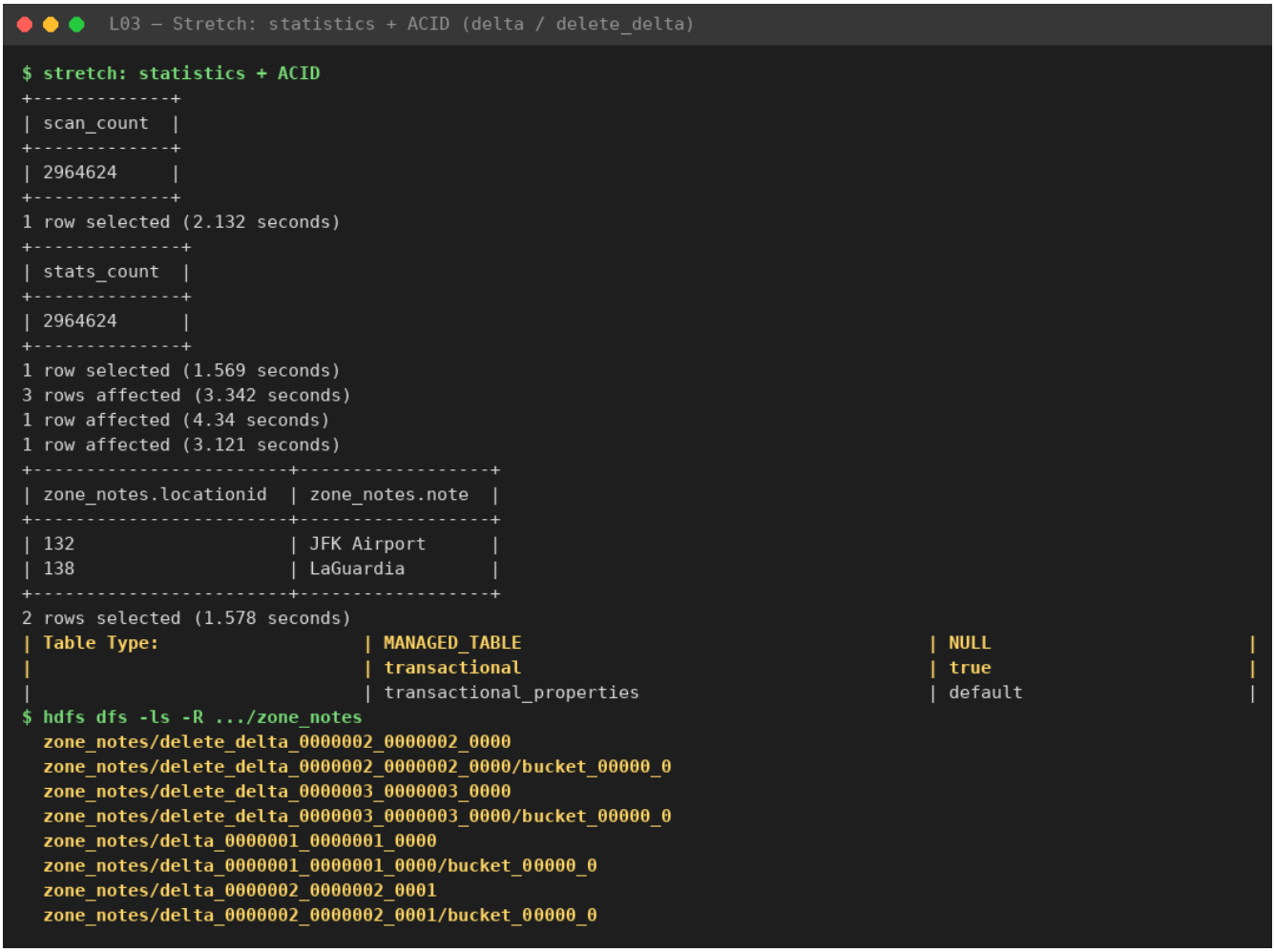


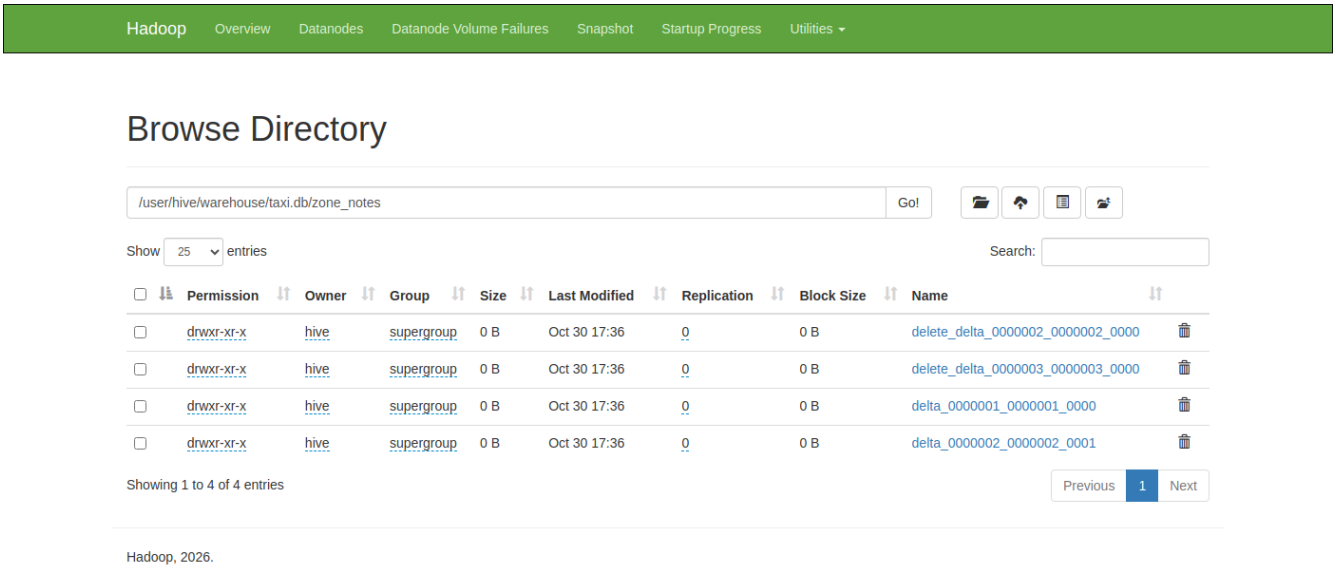


Stretch

ACID. `zone_notes` với `transactional=true` là `MANAGED_TABLE` thật. Sau INSERT, UPDATE rồi DELETE, HDFS có `delta_0000001` (insert), `delete_delta_0000002` + `delta_0000002` (UPDATE = xoá + ghi lại) và `delete_delta_0000003` (DELETE). Không file nào bị sửa tại chỗ; reader merge các delta khi đọc, compaction gộp chúng về sau.

Statistics. `ANALYZE ... COMPUTE STATISTICS FOR COLUMNS` mất 18.4 s. Sau đó, với `hive.compute.query.using.stats=true`, `count(*)` **vẫn chạy DAG Tez** (EXPLAIN vẫn có `TableScan`, ~1.6–2.6 s). Lý do: `trips_orc` thực chất là bảng **EXTERNAL** (translated), và Hive không tin stats của bảng external để trả lời thay cho việc quét, vì file có thể bị thay đổi ngoài Hive. Với bảng managed/ACID, câu trả lời sẽ lấy thẳng từ metastore (chỉ còn Fetch, không có job).


L03 — Stretch: statistics + ACID (delta / delete_delta)
$ stretch: statistics + ACID
+-------------+
| scan_count |
+-------------+
| 2964624 |
+-------------+
1 row selected (2.132 seconds)
+--------------+
| stats_count |
+--------------+
| 2964624 |
+--------------+
1 row selected (1.569 seconds)
3 rows affected (3.342 seconds)
1 row affected (4.34 seconds)
1 row affected (3.121 seconds)
+------------------------+-------------------+
| zone_notes.locationid | zone_notes.note |
+------------------------+-------------------+
| 132 | JFK Airport |
| 138 | LaGuardia |
+------------------------+-------------------+
2 rows selected (1.578 seconds)
Table Type:	MANAGED_TABLE	NULL
	transactional	true
	transactional_properties	default
$ hdfs dfs -ls -R .../zone_notes
zone_notes/delete_delta_0000002_0000002_0000
zone_notes/delete_delta_0000002_0000002_0000/bucket_00000_0
zone_notes/delete_delta_0000003_0000003_0000
zone_notes/delete_delta_0000003_0000003_0000/bucket_00000_0
zone_notes/delta_0000001_0000001_0000
zone_notes/delta_0000001_0000001_0000/bucket_00000_0
zone_notes/delta_0000002_0000002_0001
zone_notes/delta_0000002_0000002_0001/bucket_00000_0



Trả lời checkpoint

Q1. Metastore lưu gì, HDFS lưu gì? DROP TABLE thì sao? Metastore (bảng `TBLS`, `SDS`, `PARTITIONS`… trong Postgres): database, tên bảng, cột và kiểu, SerDe, input/output format, kiểu bảng, **LOCATION** của bảng và của từng partition, giá trị partition, thống kê (numRows, totalSize…), trạng thái ACID/transaction. **HDFS** chỉ chứa file dữ liệu. `DROP TABLE trips_csv` (external): chỉ xoá metadata, file trong `/data/taxi_csv` vẫn còn. `DROP TABLE trips_orc` (external + `external.table.purge=TRUE`): xoá metadata **và** thư mục `/user/hive/warehouse/taxi.db/trips_orc`.

Q3. Vì sao ORC/Parquet nhỏ hơn CSV 4–5 lần? (thực đo 5.2× và 4.2×) (a) **Lưu theo cột**: giá trị cùng kiểu, cùng cột nằm cạnh nhau nên các encoding dictionary, RLE, delta, bit-packing rất hiệu quả (`vendorid`, `payment_type`, `ratecodeid` chỉ có vài giá trị khác nhau). (b) **Kiểu nhị phân + nén** (ZLIB/Snappy) thay cho chữ số dạng text và dấu phân cách: một timestamp là 8 byte số thay vì chuỗi 19 ký tự. ORC + ZLIB nén mạnh hơn Parquet + Snappy (Snappy ưu tiên tốc độ hơn tỉ lệ nén).

Q4. Vì sao Q1 nhanh hơn trên ORC/Parquet? (8.53 s → 1.89 / 1.62 s, nhanh ~4.5–5×) * **Column pruning**: Q1 chỉ cần 2–3 trong 19 cột (`pulocationid`, `fare_amount`, cột partition). ORC/Parquet chỉ đọc các column chunk đó, còn CSV phải đọc và parse **toàn bộ** 274 MB, từng dòng. * **Partition pruning**: với ORC/Parquet, filter nằm trên `pickup_month`, nên Hive chỉ mở thư mục `pickup_month=2024-01` (đã chứng minh bằng EXPLAIN DEPENDENCY, 1 so với 5 partition). Bảng CSV không có partition nên phải quét hết rồi lọc theo timestamp. * **Predicate pushdown / min-max statistics**: mỗi stripe/row group lưu min/max và có row index, nên reader bỏ qua được các khối không thể khớp điều kiện. * Ngoài ra, reader ORC/Parquet được **vectorized** (xử lý theo batch 1,024 dòng) và không tốn công parse text

thành số.

Q6. Partition theo `pulocationid` + ngày? Theo `tpep_pickup_datetime`? `pulocationid` × ngày ≈ 265 × 31 ≈ **8,000 partition mỗi tháng**, mỗi partition chỉ có một file vài KB đến vài trăm KB. Đây là **small files problem**: NameNode phải giữ hàng chục nghìn object trong heap, **Metastore** phải lưu hàng nghìn dòng `PARTITIONS`/`SDS` (query planning chậm vì phải liệt kê và lấy metadata partition), và mỗi file nhỏ thành một split/task riêng nên overhead lấn át thời gian xử lý thật. Partition theo timestamp gốc thì gần như **mỗi giây một partition**: hàng triệu thư mục, file vài trăm byte, metastore và NameNode quá tải. Nguyên tắc: partition theo cột có cardinality thấp mà query hay lọc, mỗi partition nên chứa từ hàng trăm MB đến khoảng 1 GB.

Q7. Partitioning hay bucketing? Vì sao bucketing có số file cố định? Partitioning tạo thư mục theo **giá trị** cột, dùng cho cột cardinality thấp mà query hay lọc (ngày, tháng, quốc gia), để bỏ qua cả thư mục khi đọc. **Bucketing** chia dữ liệu theo `hash(col) mod N` vào N file, dùng cho cột cardinality cao (id, `pulocationid`). Nó cho phép sampling (`TABLESAMPLE`), bucket map join hoặc sort-merge-bucket join, và tránh shuffle khi hai bảng được bucket giống nhau trên khoá join. Số file cố định vì **N được chốt trong DDL** (`INTO 4 BUCKETS`): mỗi dòng đi vào đúng bucket `hash mod 4` bất kể dữ liệu nhiều hay ít (thực nghiệm: đúng 4 file, mọi dòng JFK nằm trong `000002_0`).

Q2. Trong file trips, `ROW FORMAT DELIMITED` (LazySimpleSerDe) chỉ tách theo dấu phẩy, đủ dùng vì không có ngoặc kép hay header. File zones có header và `"giá trị trong ngoặc kép"` (ví dụ `"Allerton/Pelham Gardens"`), nên cần `OpenCSVSerde`. Nhược điểm của nó: **mọi cột đều là STRING** (phải tự CAST, như `CAST(z.locationid AS INT)` trong Q2), parse chậm hơn, không xử lý NULL đặc biệt.

Q5. Partition tĩnh ghi giá trị ngay trong câu lệnh (`PARTITION (pickup_month='2024-01')`). Partition động lấy giá trị từ dữ liệu (cột cuối của SELECT). Mặc định là `strict` (bắt buộc ít nhất một giá trị tĩnh) để một câu query lỗi không vô tình tạo hàng nghìn partition, file và object trong metastore. Lab đã thấy điều này ở quy mô nhỏ: các dòng 2002/2009 sinh ra partition "lạ".

Q8. Có 18 dòng ngoài tháng 01/2024: 2 dòng 2002-12-31 (một cặp +10.5/−10.5, tức một chuyến bị hoàn tiền), 3 dòng 2009-01-01, 10 dòng 2023-12-31 23:39–23:58 (chuyến bắt đầu trước nửa đêm giao thừa) và 3 dòng 2024-02-01 00:00 (chuyến bắt đầu ngay sau nửa đêm). Các dòng năm 2002/2009 là lỗi đồng hồ thiết bị, còn các dòng 2023-12 và 2024-02 là chuyến thật nằm ở biên tháng. Trong pipeline thật: kiểm tra event time so với tháng của file nguồn; đưa dòng ngoài khoảng hợp lệ vào bảng **quarantine/rejected** kèm lý do và đếm/cảnh báo; các dòng biên tháng hợp lệ thì ghi vào đúng partition (hoặc giữ lại theo tháng của file); không để dynamic partitioning âm thầm tạo partition rác.

L04 — Ingestion: Sqoop và Spark JDBC từ PostgreSQL vào HDFS

Stack L04 gồm PostgreSQL (nguồn Olist, 6 bảng), HDFS, YARN, JobHistory, Sqoop 1.4.7 và Spark. Log gốc nằm trong các file `0*.txt`, ảnh ở `img/`.

File nộp

File	Nội dung
`sqoop_part1.sh`	Các lệnh Sqoop phần 1: explore, full import, TODO 1.2 (customers), append, lastmodified, saved job
`spark_jdbc_ingest.py`	TODO 1–5: đọc JDBC song song, ghi Parquet phân vùng, incremental theo watermark, dedup snapshot
`01-09_*.txt`	Log từng bước, kể cả log query phía PostgreSQL



```
src/spark_jdbc_ingest.py (đã hoàn thành TODO 1–5)

"""L04 Part 2 -- Spark JDBC ingestion from Postgres to Parquet on HDFS (STARTER -- complete the TODOs).

Run inside the spark container:
    docker compose exec spark spark-submit --jars /jars/postgresql.jar \
        /lab/src/spark_jdbc_ingest.py --step full
    docker compose exec spark spark-submit --jars /jars/postgresql.jar \
        /lab/src/spark_jdbc_ingest.py --step incremental

--step full         single-partition read vs. parallel read (partitionColumn/lowerBound/upperBound/
                    numPartitions), rows per partition, Parquet partitioned by purchase month.
--step incremental  watermark on updated_at: read only rows changed since the last run, append them
                    to a change log, rebuild the "current" snapshot, save the new watermark.
"""
import argparse
import time

from pyspark.sql import SparkSession, Window
from pyspark.sql import functions as F

JDBC_URL = "jdbc:postgresql://postgres:5432/olist"
JDBC_OPTS = {"url": JDBC_URL, "user": "olist", "password": "olist", "driver": "org.postgresql.Driver"}
BASE = "hdfs://namenode:8020/data/olist/spark"


def jdbc(spark, **options):
    """spark.read.format('jdbc') with the connection options pre-filled."""
    reader = spark.read.format("jdbc").options(**JDBC_OPTS)
    for k, v in options.items():
        reader = reader.option(k, v)
    return reader.load()


def timed(label, fn):
    t0 = time.time()
    result = fn()
    print(f"[time] {label}: {time.time() - t0:.2f} s")
    return result


def step_full(spark):
    # (1) naive read: ONE partition -> one JDBC connection, one task does all the work
    naive = jdbc(spark, dbtable="orders")
    print("naive partitions:", naive.rdd.getNumPartitions())
    timed("naive count", naive.count)

    # (2) the bounds, like Sqoop's BoundingValsQuery
    b = jdbc(spark, query="SELECT min(order_seq) AS lo, max(order_seq) AS hi FROM orders").first()
    print(f"order_seq bounds: {b.lo} .. {b.hi}")

    # (3) parallel read: Spark generates numPartitions range queries on partitionColumn.
    #     lowerBound/upperBound only decide the STRIDE -- they do NOT filter rows.
    # TODO 1: read "orders" in 4 parallel partitions split on order_seq.
    #         Use the options partitionColumn, lowerBound, upperBound (from b.lo / b.hi, as strings),
    #         numPartitions and fetchsize="1000".
    orders = jdbc(spark, dbtable="orders", partitionColumn="order_seq",
                  lowerBound=str(b.lo), upperBound=str(b.hi), numPartitions="4", fetchsize="1000")
    print("parallel partitions:", orders.rdd.getNumPartitions())
    timed("parallel count", orders.count)
    orders.groupBy(F.spark_partition_id().alias("partition")).count().orderBy("partition").show()

    # (4) same with a timestamp partition column (also allowed: numeric, date, timestamp)
    # TODO 2: same read, but split on the timestamp column order_purchase_timestamp,
    #         lowerBound "2017-01-01 00:00:00", upperBound "2018-09-01 00:00:00", 4 partitions.
    by_ts = jdbc(spark, dbtable="orders", partitionColumn="order_purchase_timestamp",
                 lowerBound="2017-01-01 00:00:00", upperBound="2018-09-01 00:00:00", numPartitions="4")
    print("rows per partition when splitting on order_purchase_timestamp (volume grows over time):")
    by_ts.groupBy(F.spark_partition_id().alias("partition")).count().orderBy("partition").show()

    # (5) write Parquet, partitioned by purchase month (Hive-style directories)
    # TODO 3: add a column purchase_month = 'yyyy-MM' of order_purchase_timestamp and write the
    #         DataFrame as Parquet to f"{BASE}/orders", partitioned by purchase_month, mode overwrite.
    out = orders.withColumn("purchase_month", F.date_format("order_purchase_timestamp", "yyyy-MM"))
    timed("write parquet", lambda: out.write.mode("overwrite").partitionBy("purchase_month")
          .parquet(f"{BASE}/orders"))
    back = spark.read.parquet(f"{BASE}/orders")
    print("rows written:", back.count(), "| months:", back.select("purchase_month").distinct().count())
    back.filter("purchase_month = '2018-08'").explain()   # look for PartitionFilters

    # (6) the other tables, one call each (small tables: 1 partition is fine)
    for table in ["customers", "order_items", "order_payments", "products", "sellers"]:
        df = jdbc(spark, dbtable=table)
        df.write.mode("overwrite").parquet(f"{BASE}/{table}")
        print(f"{table}: {df.count()} rows")


def path_exists(spark, path):
    """Ask the Hadoop FileSystem API (through py4j) whether an HDFS path exists."""
    jpath = spark._jvm.org.apache.hadoop.fs.Path(path)
    return jpath.getFileSystem(spark._jsc.hadoopConfiguration()).exists(jpath)


def read_watermark(spark):
    state = f"{BASE}/_state/orders_watermark"
    if not path_exists(spark, state):          # first run: no state yet -> load everything
        return "1970-01-01 00:00:00"
    return spark.read.json(state).first()["watermark"]


def step_incremental(spark):
    wm_old = read_watermark(spark)
    # upper bound = DB clock NOW, fixed before reading, so the next run starts exactly here
    wm_new = jdbc(spark, query="SELECT to_char(now(), 'YYYY-MM-DD HH24:MI:SS.US') AS ts").first().ts
    print(f"watermark: ({wm_old}, {wm_new}]")

    # TODO 4: read ONLY the rows with  wm_old < updated_at <= wm_new.
    #         Hint: dbtable accepts a sub-query with an alias: "(SELECT ... WHERE ...) AS changed_orders"
    #         Then add a column _ingested_at = wm_new cast to timestamp.
    changed_sql = (f"(SELECT * FROM orders WHERE updated_at > timestamp '{wm_old}'"
                   f" AND updated_at <= timestamp '{wm_new}') AS changed_orders")
    changes = jdbc(spark, dbtable=changed_sql).withColumn("_ingested_at", F.lit(wm_new).cast("timestamp"))
    changes = changes.cache()
    n = changes.count()
    print(f"changed rows since last run: {n}")
    changes.select("order_seq", "order_id", "order_status", "updated_at").orderBy("updated_at").show(10, False)

    # append-only change log (bronze-style), one directory per ingestion
    changes.write.mode("append").parquet(f"{BASE}/orders_changes")

    # current snapshot = latest version of every order_id across the whole change log
    log = spark.read.parquet(f"{BASE}/orders_changes")
    # TODO 5: keep only the LATEST version of every order_id (row_number over a window partitioned by
    #         order_id, ordered by updated_at desc, _ingested_at desc; keep row 1).
    w = Window.partitionBy("order_id").orderBy(F.col("updated_at").desc(), F.col("_ingested_at").desc())
    current = log.withColumn("_rn", F.row_number().over(w)).filter("_rn = 1").drop("_rn")
    current.write.mode("overwrite").parquet(f"{BASE}/orders_current")
    print("change-log rows:", log.count(), "| current snapshot rows:",
  spark.read.parquet(f"{BASE}/orders_current").count())

    # persist the new watermark ONLY after the data was written successfully
    spark.createDataFrame([(wm_new,)], "watermark string").coalesce(1) \
        .write.mode("overwrite").json(f"{BASE}/_state/orders_watermark")
    print("saved watermark", wm_new)

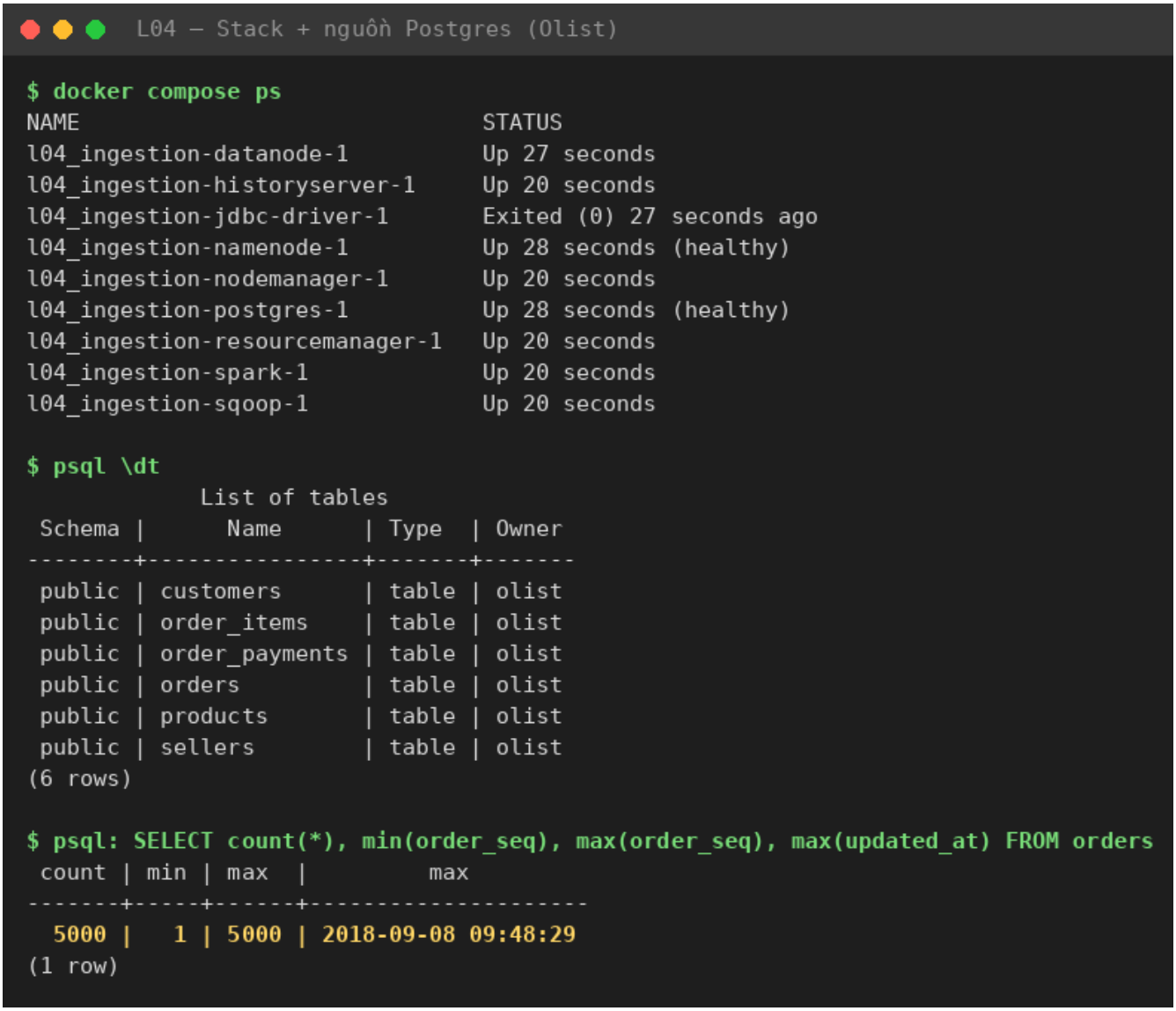

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--step", choices=["full", "incremental"], required=True)
    args = ap.parse_args()
    spark = (SparkSession.builder.appName(f"L04-jdbc-{args.step}")
             .config("spark.hadoop.dfs.replication", "1")      # single-DataNode lab HDFS
             .config("spark.sql.session.timeZone", "UTC")
             .getOrCreate())
    spark.sparkContext.setLogLevel("WARN")
    {"full": step_full, "incremental": step_incremental}[args.step](spark)
    spark.stop()


if __name__ == "__main__":
    main()
```

Hình 18: spark_jdbc_ingest.py

1. Nguồn dữ liệu

Có 6 bảng. `orders` có 5,000 dòng, `order_seq` chạy từ 1 đến 5000 và `max(updated_at)` là `2018-09-08 09:48:29`. Theo trạng thái: delivered 4,793, shipped 102, ...



Hình 19: stack and source

2. Sqoop full import và cách chia split

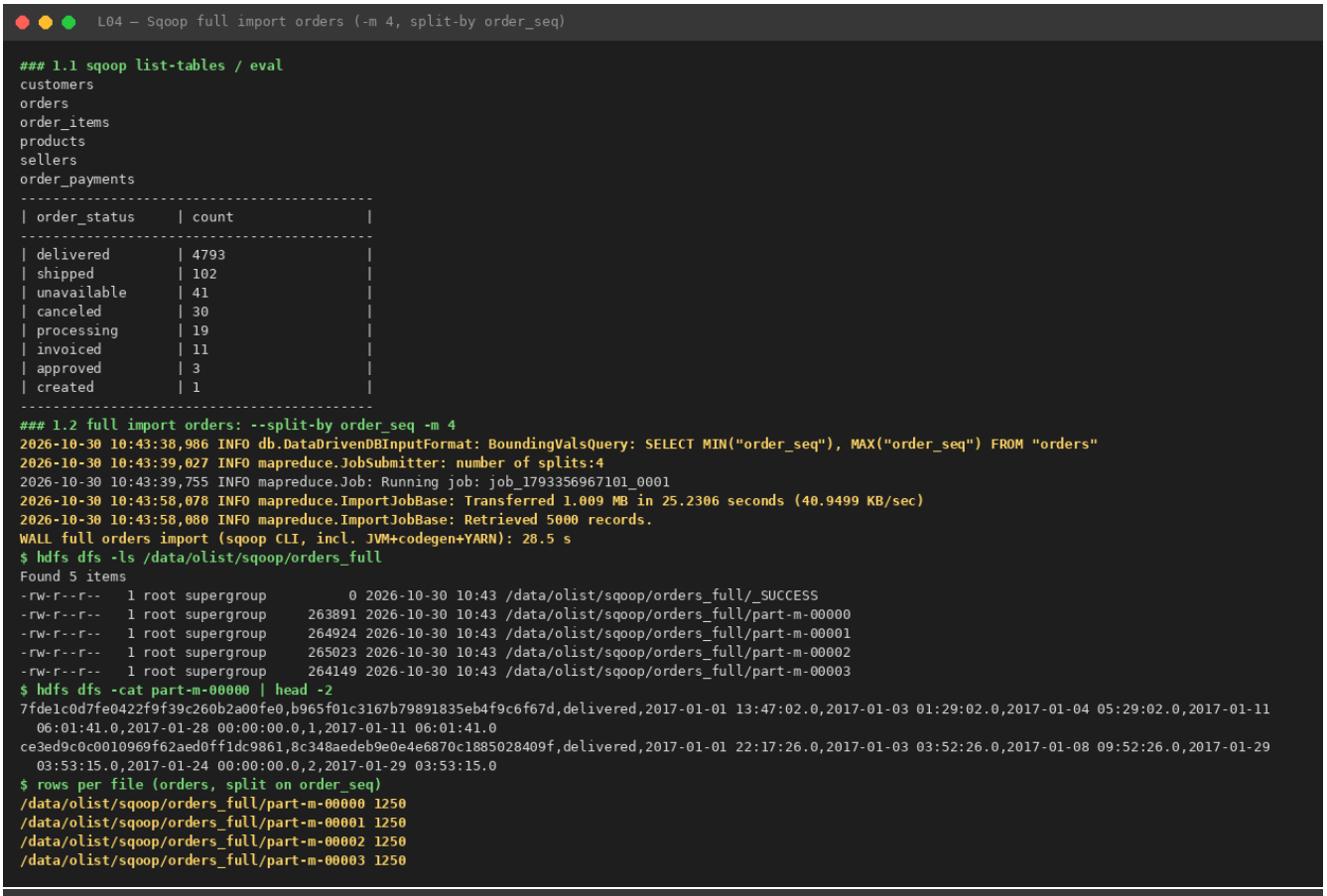
`sqoop import --table orders --split-by order_seq -m 4`:

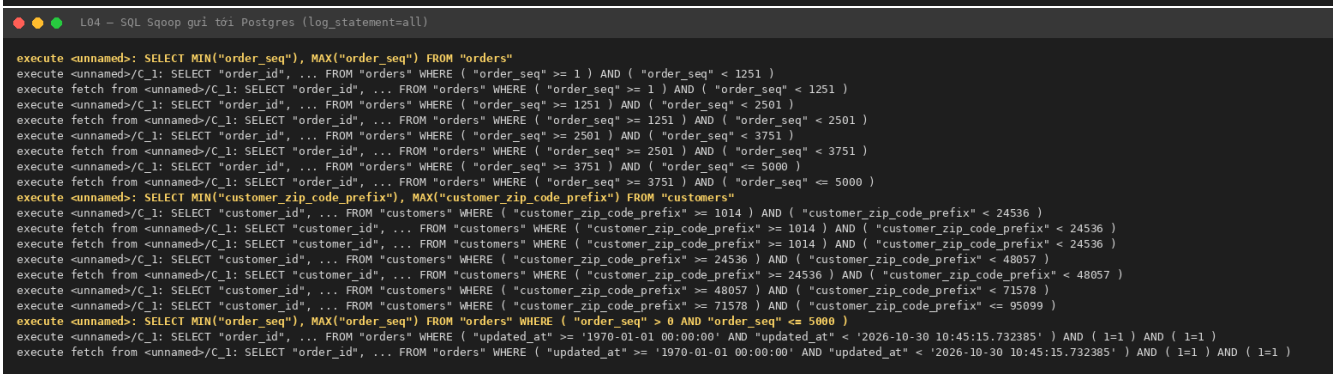
- Sqoop chạy `BoundingValsQuery` (`SELECT MIN(order_seq), MAX(order_seq) FROM orders`), được (1, 5000), rồi chia thành 4 khoảng đều nhau.
- Log PostgreSQL ghi lại đúng 4 câu query:

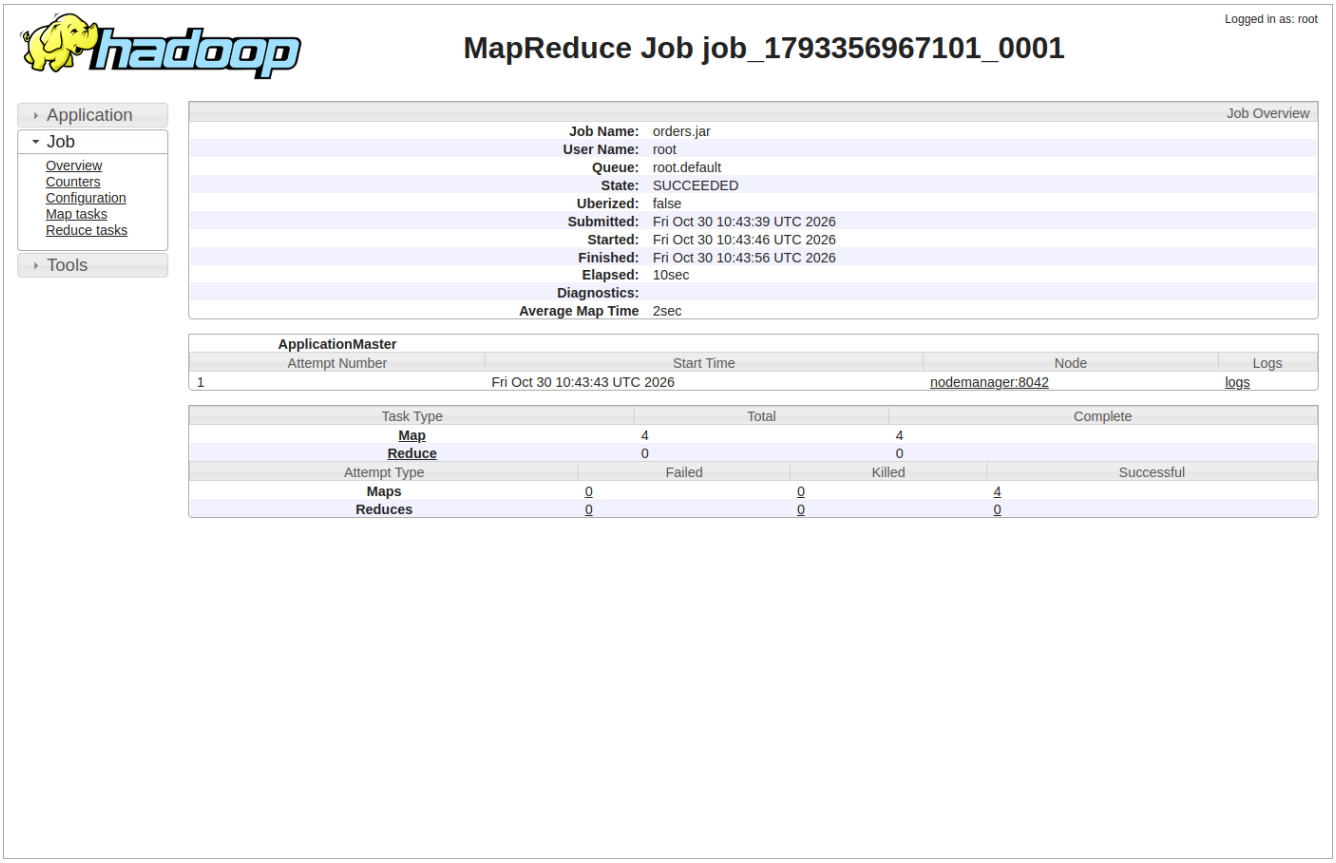
Map	Điều kiện WHERE	Số dòng
0	`order_seq >= 1 AND order_seq < 1251`	1,250
1	`order_seq >= 1251 AND order_seq < 2501`	1,250

Map	Điều kiện WHERE	Số dòng
2	`order_seq >= 2501 AND order_seq < 3751`	1,250
3	`order_seq >= 3751 AND order_seq <= 5000`	1,250

- Đã chuyển 1.009 MB trong 25.23 s (thời gian MR); tổng thời gian lệnh CLI là 28.5 s.
- 4 file `part-m-0000x` chia đều nhau.



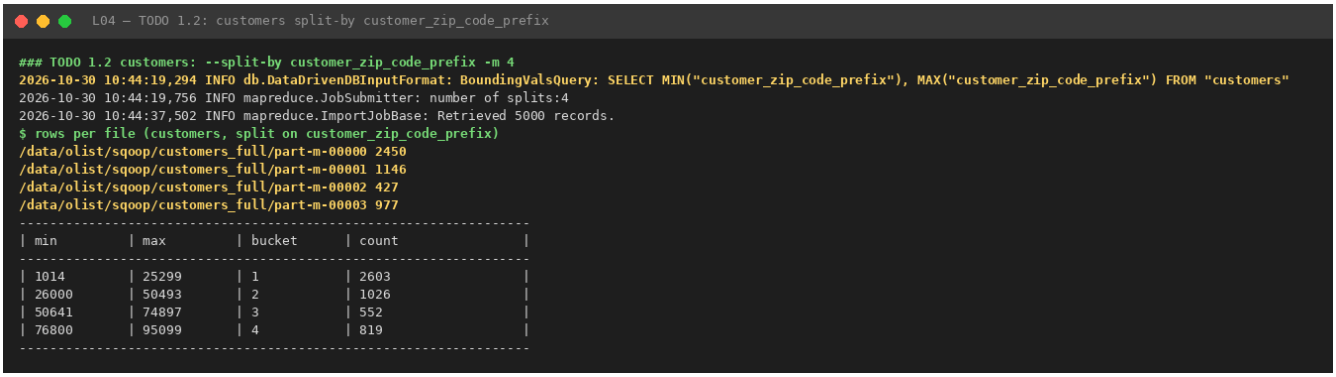




TODO 1.2: `customers --split-by customer_zip_code_prefix -m 4`

Bảng / cột split	part-m-00000	part-m-00001	part-m-00002	part-m-00003
`orders` / `order_seq` (1–5000)	1,250	1,250	1,250	1,250
`customers` / `customer_zip_code_prefix` (1014–95099)	**2,450**	1,146	**427**	977

Khoảng zip được chia đều tại 24536, 48057 và 71578, nhưng zip code tập trung ở vùng São Paulo (01xxx–19xxx). Vì vậy split đầu nhận gần một nửa số dòng, còn split thứ ba chỉ nhận 427 dòng: **data skew**.



Hình 20: customers skew

3. Sqoop incremental: append, lastmodified, saved job, merge, export

Bước	Kết quả
Append, lần nạp đầu	lower 0, upper 5000 → 5,000 dòng; Sqoop gợi ý `--last-value 5000`
Lastmodified, baseline	5,000 dòng; last-value `2026-10-30 10:45:15.732385`
Saved job `orders_incr`	`incremental.last.value = 5000` lưu trong metastore của Sqoop
`simulate_changes` #1	tạo `order_seq` 5001–5003; cập nhật đơn 139 và 278 thành `delivered`
`sqoop job --exec orders_incr`	lower 5000, upper 5003 → lấy **3** dòng; `last.value` thành 5003; tổng 5,003 dòng
Lastmodified + `--merge-key order_id`	lấy **5** dòng (3 mới + 2 cập nhật); merge job → một `part-r-00000` gồm 5,003 dòng, không trùng
`sqoop export` ngược về PostgreSQL	5,000 record

Saved job tự lưu `last.value`, nên lần chạy sau chỉ đọc phần mới. Append chỉ nhìn thấy 3 dòng INSERT. Lastmodified thấy thêm 2 dòng UPDATE, và merge-key gộp mỗi `order_id` về bản mới nhất.

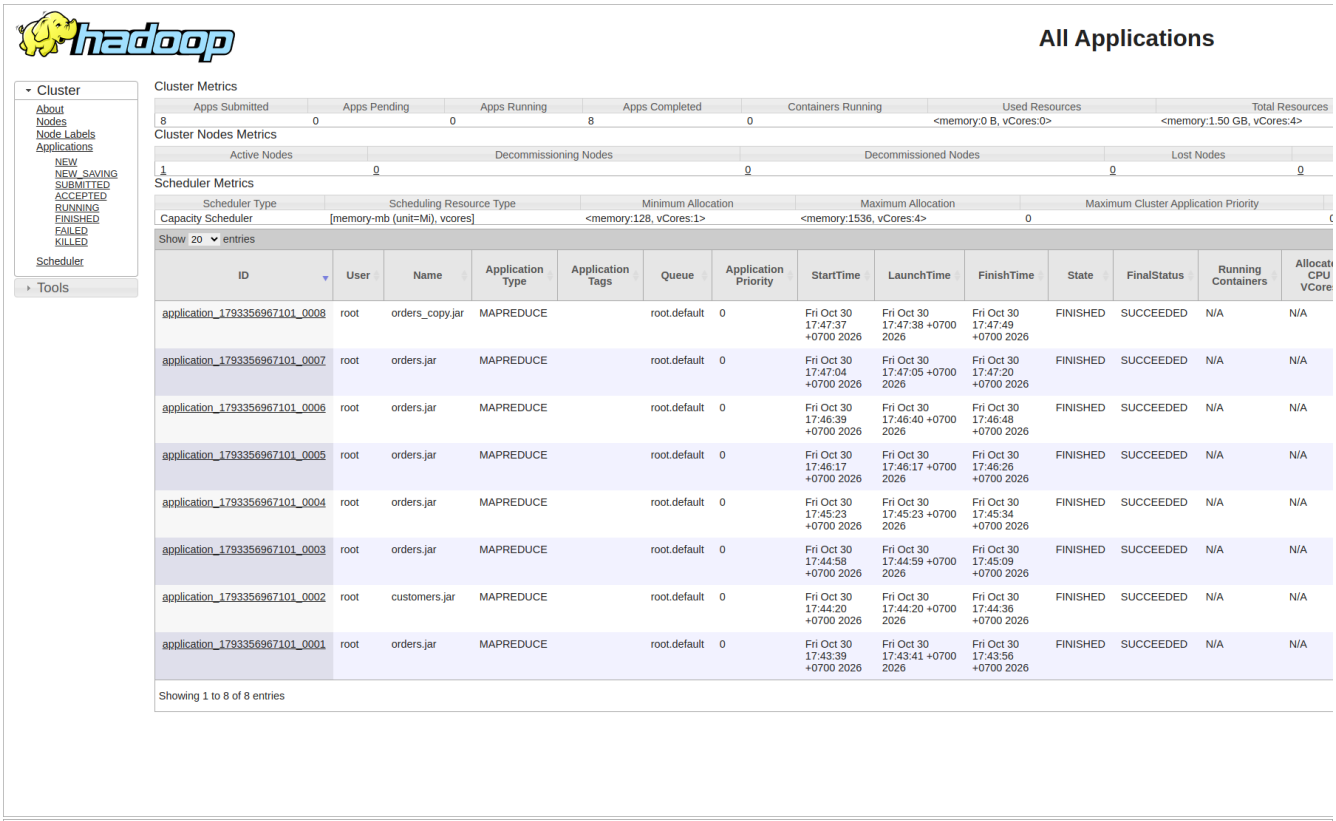
L04 — Sqoop incremental append / lastmodified baseline / saved job

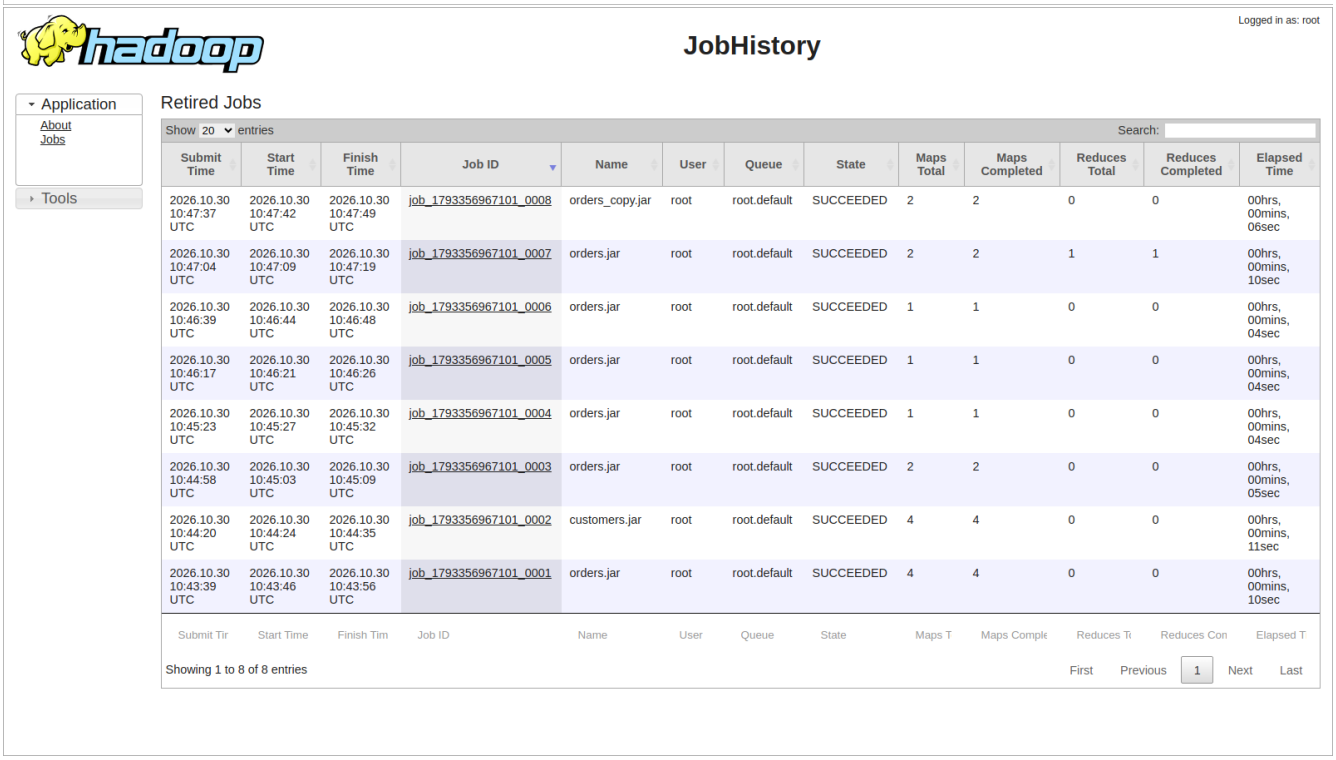
```
### 1.3 incremental append (initial load, --last-value 0)
2026-10-30 10:44:51,951 INFO tool.ImportTool: Lower bound value: 0
2026-10-30 10:44:51,951 INFO tool.ImportTool: Upper bound value: 5000
2026-10-30 10:45:10,960 INFO mapreduce.ImportJobBase: Retrieved 5000 records.
2026-10-30 10:45:10,989 INFO tool.ImportTool:   --incremental append
2026-10-30 10:45:10,989 INFO tool.ImportTool:    --check-column order_seq
2026-10-30 10:45:10,989 INFO tool.ImportTool:    --last-value 5000
2026-10-30 10:45:10,989 INFO tool.ImportTool: (Consider saving this with 'sqoop job --create')
### 1.5a incremental lastmodified baseline
2026-10-30 10:45:15,734 INFO tool.ImportTool: Lower bound value: '1970-01-01 00:00:00'
2026-10-30 10:45:15,734 INFO tool.ImportTool: Upper bound value: '2026-10-30 10:45:15.732385'
2026-10-30 10:45:35,734 INFO mapreduce.ImportJobBase: Retrieved 5000 records.
2026-10-30 10:45:35,746 INFO tool.ImportTool:    --last-value 2026-10-30 10:45:15.732385
next --last-value: 2026-10-30 10:45:15.732385
### 1.4 saved job orders_incr
$ sqoop job --list
Available jobs:
  orders_incr
$ sqoop job --show orders_incr | grep incremental
incremental.last.value = 5000
incremental.col = order_seq
incremental.mode = AppendRows
```

L04 — simulate_changes → saved job exec, lastmodified+merge, export

```
$ psql -f sql/simulate_changes.sql   (#1, before Sqoop part 2)
INSERT 0 3
UPDATE 2
 order_seq |             order_id             | order_status |         updated_at
-----------+----------------------------------+--------------+----------------------------
      5001 | b66d7d8bbcad4568830448d9d663fac5 | created      | 2026-10-30 10:46:07.357187
      5002 | 0c83fcfb71e8bc5b9e8b3a73f7140d4f | created      | 2026-10-30 10:46:07.357187
      5003 | 7008b2a0e1ffae9397506c90fb4eab1b | created      | 2026-10-30 10:46:07.357187
       139 | 8a7ac9a44f356814c2311987dcd8ad10 | delivered    | 2026-10-30 10:46:07.361794
       278 | 65a11c7df0f6c77c1688232a1107044f | delivered    | 2026-10-30 10:46:07.361794
(5 rows)


### 4b. exec the saved job -> only the NEW orders are appended
2026-10-30 10:46:10,820 INFO tool.ImportTool: Lower bound value: 5000
2026-10-30 10:46:10,821 INFO tool.ImportTool: Upper bound value: 5003
2026-10-30 10:46:28,335 INFO mapreduce.ImportJobBase: Retrieved 3 records.
incremental.last.value = 5003
Found 3 items
-rw-r--r--   1 root supergroup     528815 2026-10-30 10:45 /data/olist/sqoop/orders_append/part-m-00000
-rw-r--r--   1 root supergroup     529172 2026-10-30 10:45 /data/olist/sqoop/orders_append/part-m-00001
-rw-r--r--   1 root supergroup        420 2026-10-30 10:46 /data/olist/sqoop/orders_append/part-m-00002
rows in append dir: 5003
### 5. incremental LASTMODIFIED + --merge-key: new AND updated rows, merged into the snapshot
using --last-value '2026-10-30 10:45:15.732385'
2026-10-30 10:46:35,899 INFO tool.ImportTool: Lower bound value: '2026-10-30 10:45:15.732385'
2026-10-30 10:46:35,899 INFO tool.ImportTool: Upper bound value: '2026-10-30 10:46:35.897666'
                Merged Map outputs=0
2026-10-30 10:46:50,942 INFO mapreduce.ImportJobBase: Retrieved 5 records.
2026-10-30 10:46:50,945 INFO tool.ImportTool: Final destination exists, will run merge job.
                Merged Map outputs=2
2026-10-30 10:47:21,419 INFO tool.ImportTool:   --last-value 2026-10-30 10:46:35.897666
Found 2 items
-rw-r--r--   1 root supergroup          0 2026-10-30 10:47 /data/olist/sqoop/orders_lastmod/_SUCCESS
-rw-r--r--   1 root supergroup    1058459 2026-10-30 10:47 /data/olist/sqoop/orders_lastmod/part-r-00000
rows in lastmod dir: 5003   (= baseline + the new orders; updated orders were replaced in place, not duplicated)
0c83fcfb71e8bc5b9e8b3a73f7140d4f,201747a216850606f49c716719a837ad,created,2026-10-30 10:46:07.0,\N,\N,\N,\N,5002,2026-10-30 10:46:07.357187
65a11c7df0f6c77c1688232a1107044f,d51eb5503b9ff6e63bb39685d32577a2,delivered,2017-03-03 11:33:42.0,2017-03-03 11:56:42.0,2017-03-04 15:56:42.0,2026-10-30 10:46:07.0,2017-03-23
  00:00:00.0,278,2026-10-30 10:46:07.361794
7008b2a0e1ffae9397506c90fb4eab1b,af3b0010d669e4b210716eae69c81c2d,created,2026-10-30 10:46:07.0,\N,\N,\N,\N,5003,2026-10-30 10:46:07.357187
8a7ac9a44f356814c2311987dcd8ad10,155e17e9b120e25f6a64f2c8efe2a650,delivered,2017-02-03 10:49:42.0,2017-02-03 10:54:42.0,2017-02-09 09:54:42.0,2026-10-30 10:46:07.0,2017-02-23
  00:00:00.0,139,2026-10-30 10:46:07.361794
b66d7d8bbcad4568830448d9d663fac5,a26322435a248248b86845ebe0587466,created,2026-10-30 10:46:07.0,\N,\N,\N,\N,5001,2026-10-30 10:46:07.357187
### 6. export: HDFS -> new Postgres table orders_copy
2026-10-30 10:47:51,093 INFO mapreduce.ExportJobBase: Transferred 1.0094 MB in 18.4353 seconds (56.0684 KB/sec)
2026-10-30 10:47:51,095 INFO mapreduce.ExportJobBase: Exported 5000 records.
-----------------------
| rows_exported       |
-----------------------
| 5000                |
-----------------------
```





4. Spark JDBC: đọc song song và ghi Parquet (TODO 1–3)

Cách đọc	Số partition	Thời gian `count()`	Phân bố dòng
Naive (không có `partitionColumn`)	1	1.17 s	tất cả 5,003 dòng trong 1 task
`partitionColumn=order_seq`, bounds 1..5003, 4 partitions	4	0.22 s	1,251 / 1,250 / 1,250 / 1,252
`partitionColumn=order_purchase_timestamp` (2017-01-01 → 2018-09-01)	4	—	782 / 1,116 / 1,419 / 1,686

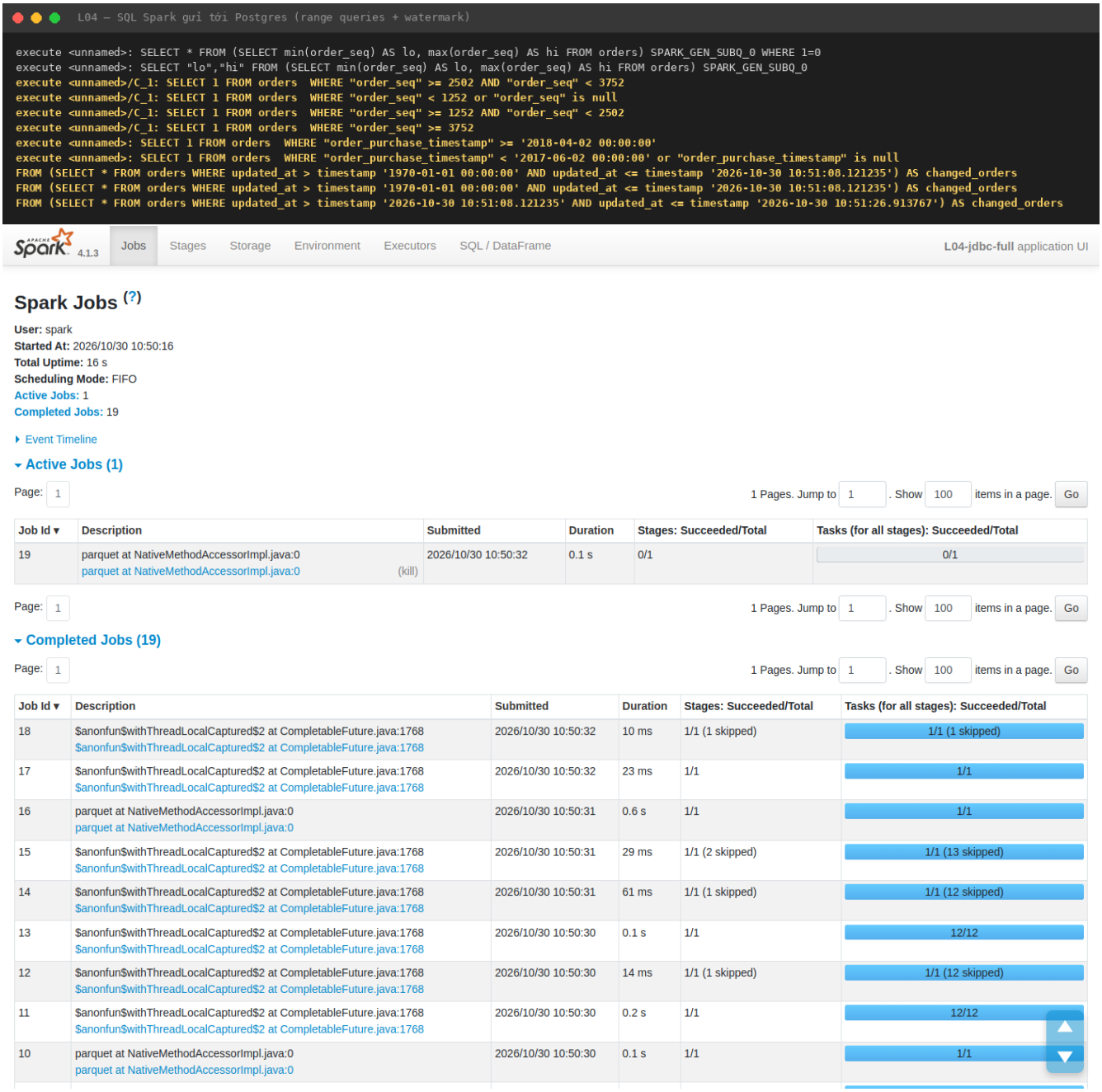
- Spark tự sinh 4 câu SQL (log PostgreSQL): `WHERE "order_seq" < 1252 or "order_seq" is null`, … , `WHERE "order_seq" >= 3752`. Partition đầu và cuối để **hở**, nên mọi dòng ngoài bounds vẫn được đọc.
- Split theo timestamp bị lệch vì số đơn tăng dần theo thời gian.
- Ghi Parquet `partitionBy("purchase_month")`:
  - mất 3.10 s, ra 21 thư mục tháng, đủ 5,003 dòng;
  - query lọc `purchase_month = '2018-08'` cho thấy `PartitionFilters` trong physical plan, tức là có partition pruning.
- Các bảng khác:

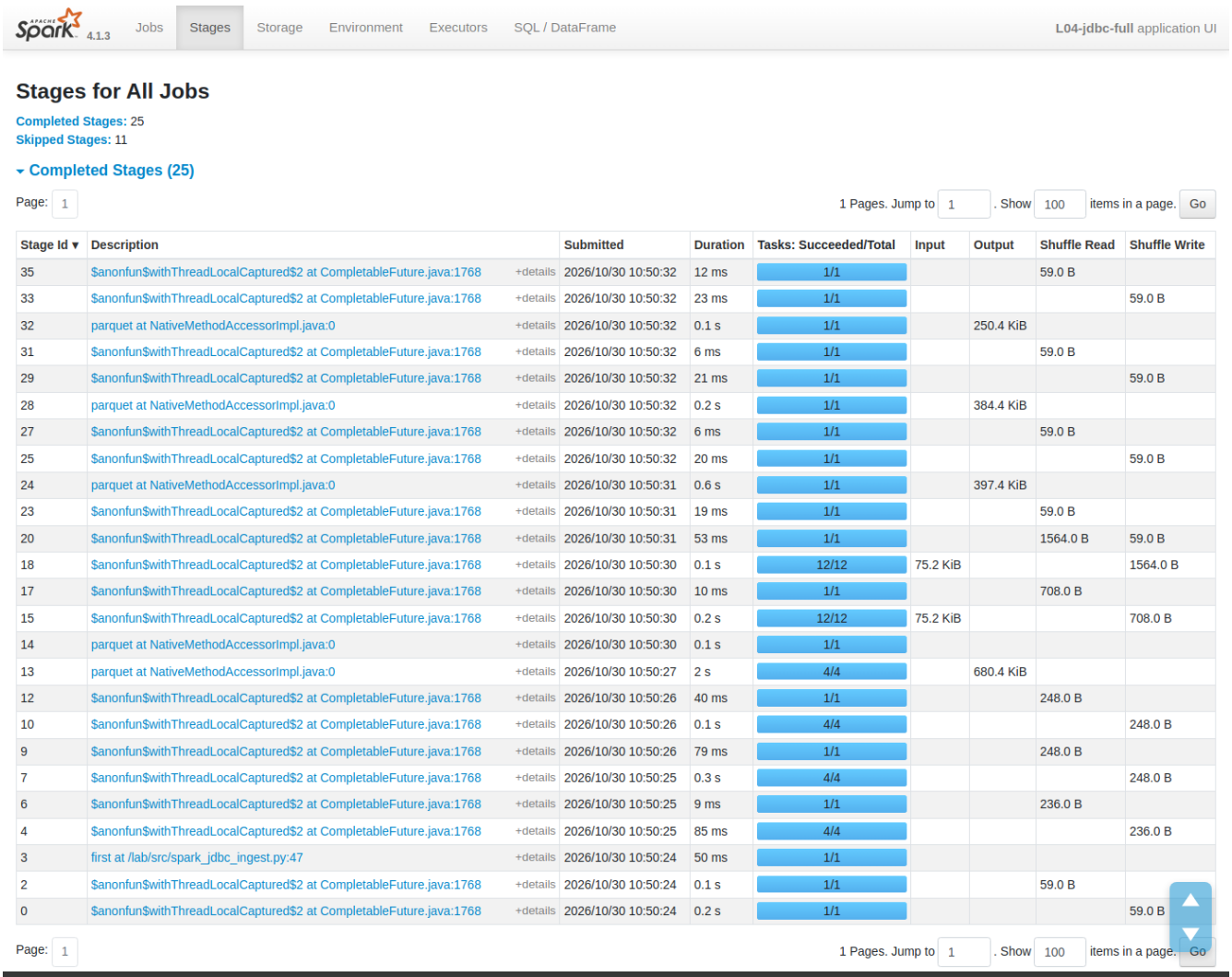
Bảng	Số dòng
customers	5,003
order_items	5,852
order_payments	5,472
products	1,500
sellers	312

- Toàn bộ `spark-submit ... full` chạy hết 21.2 s.

```
L04 — spark-submit spark_jdbc_ingest.py --step full

naive partitions: 1
[time] naive count: 1.17 s
order_seq bounds: 1 .. 5003
parallel partitions: 4
[time] parallel count: 0.22 s
+---------+-----+
|partition|count|
+---------+-----+
|        0| 1251|
|        1| 1250|
|        2| 1250|
|        3| 1252|
+---------+-----+
rows per partition when splitting on order_purchase_timestamp (volume grows over time):
+---------+-----+
|partition|count|
+---------+-----+
|        0|  782|
|        1| 1116|
|        2| 1419|
|        3| 1686|
+---------+-----+
[time] write parquet: 3.10 s
rows written: 5003 | months: 21
== Physical Plan ==
*(1) ColumnarToRow
+- FileScan parquet [order_id#108,customer_id#109,order_status#110,order_purchase_timestamp#111,order_approved_at#112,order_delivered_carrier_date#113,order_delivered_customer_date
  #114,order_estimated_delivery_date#115,order_seq#116L,updated_at#117,purchase_mo
customers: 5003 rows
order_items: 5852 rows
order_payments: 5472 rows
products: 1500 rows
sellers: 312 rows
WALL spark-submit --step full (incl. JVM start): 21.2 s
```


L04 – SQL Spark gửi tới Postgres (range queries + watermark)
execute <unnamed>: SELECT * FROM (SELECT min(order_seq) AS lo, max(order_seq) AS hi FROM orders) SPARK_GEN_SUBQ_0 WHERE 1=0
execute <unnamed>: SELECT "lo","hi" FROM (SELECT min(order_seq) AS lo, max(order_seq) AS hi FROM orders) SPARK_GEN_SUBQ_0
execute <unnamed>/C_1: SELECT 1 FROM orders WHERE "order_seq" >= 2502 AND "order_seq" < 3752
execute <unnamed>/C_1: SELECT 1 FROM orders WHERE "order_seq" < 1252 or "order_seq" is null
execute <unnamed>/C_1: SELECT 1 FROM orders WHERE "order_seq" >= 1252 AND "order_seq" < 2502
execute <unnamed>/C_1: SELECT 1 FROM orders WHERE "order_seq" >= 3752
execute <unnamed>: SELECT 1 FROM orders WHERE "order_purchase_timestamp" >= '2018-04-02 00:00:00'
execute <unnamed>: SELECT 1 FROM orders WHERE "order_purchase_timestamp" < '2017-06-02 00:00:00' or "order_purchase_timestamp" is null
FROM (SELECT * FROM orders WHERE updated_at > timestamp '1970-01-01 00:00:00' AND updated_at <= timestamp '2026-10-30 10:51:08.121235') AS changed_orders
FROM (SELECT * FROM orders WHERE updated_at > timestamp '1970-01-01 00:00:00' AND updated_at <= timestamp '2026-10-30 10:51:08.121235') AS changed_orders
FROM (SELECT * FROM orders WHERE updated_at > timestamp '2026-10-30 10:51:08.121235' AND updated_at <= timestamp '2026-10-30 10:51:26.913767') AS changed_orders
Spark 4.1.3
Jobs
Stages
Storage
Environment
Executors
SQL / DataFrame
L04-jdbc-full application UI
Spark Jobs (?)
User: spark
Started At: 2026/10/30 10:50:16
Total Uptime: 16 s
Scheduling Mode: FIFO
Active Jobs: 1
Completed Jobs: 19
Event Timeline
Active Jobs (1)
Page: 1
1 Pages. Jump to 1 . Show 100 items in a page. Go
Job Id ▾ | Description | Submitted | Duration | Stages: Succeeded/Total | Tasks (for all stages): Succeeded/Total
19 | parquet at NativeMethodAccessorImpl.java:0 parquet at NativeMethodAccessorImpl.java:0 (kill) | 2026/10/30 10:50:32 | 0.1 s | 0/1 | 0/1
Page: 1
1 Pages. Jump to 1 . Show 100 items in a page. Go
Completed Jobs (19)
Page: 1
1 Pages. Jump to 1 . Show 100 items in a page. Go
Job Id ▾ | Description | Submitted | Duration | Stages: Succeeded/Total | Tasks (for all stages): Succeeded/Total
18 | $anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 $anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 | 2026/10/30 10:50:32 | 10 ms | 1/1 (1 skipped) | 1/1 (1 skipped)
17 | $anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 $anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 | 2026/10/30 10:50:32 | 23 ms | 1/1 | 1/1
16 | parquet at NativeMethodAccessorImpl.java:0 parquet at NativeMethodAccessorImpl.java:0 | 2026/10/30 10:50:31 | 0.6 s | 1/1 | 1/1
15 | $anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 $anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 | 2026/10/30 10:50:31 | 29 ms | 1/1 (2 skipped) | 1/1 (13 skipped)
14 | $anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 $anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 | 2026/10/30 10:50:31 | 61 ms | 1/1 (1 skipped) | 1/1 (12 skipped)
13 | $anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 $anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 | 2026/10/30 10:50:30 | 0.1 s | 1/1 | 12/12
12 | $anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 $anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 | 2026/10/30 10:50:30 | 14 ms | 1/1 (1 skipped) | 1/1 (12 skipped)
11 | $anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 $anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 | 2026/10/30 10:50:30 | 0.2 s | 1/1 | 12/12
10 | parquet at NativeMethodAccessorImpl.java:0 parquet at NativeMethodAccessorImpl.java:0 | 2026/10/30 10:50:30 | 0.1 s | 1/1 | 1/1




```
L04 — Parquet partitionBy(purchase_month) trên HDFS

$ hdfs dfs -ls /data/olist/spark/orders | head
Found 22 items
-rw-r--r--   1 spark supergroup          0 2026-10-30 10:50 /data/olist/spark/orders/_SUCCESS
drwxr-xr-x   - spark supergroup          0 2026-10-30 10:50 /data/olist/spark/orders/purchase_month=2017-01
drwxr-xr-x   - spark supergroup          0 2026-10-30 10:50 /data/olist/spark/orders/purchase_month=2017-02
drwxr-xr-x   - spark supergroup          0 2026-10-30 10:50 /data/olist/spark/orders/purchase_month=2017-03
drwxr-xr-x   - spark supergroup          0 2026-10-30 10:50 /data/olist/spark/orders/purchase_month=2017-04
drwxr-xr-x   - spark supergroup          0 2026-10-30 10:50 /data/olist/spark/orders/purchase_month=2017-05
drwxr-xr-x   - spark supergroup          0 2026-10-30 10:50 /data/olist/spark/orders/purchase_month=2017-06
...
$ hdfs dfs -du -s -h /data/olist/spark/orders /data/olist/sqoop/orders_full
680.4 K  680.4 K  /data/olist/spark/orders
1.0 M    1.0 M    /data/olist/sqoop/orders_full
```

So sánh Sqoop và Spark JDBC (orders)

	Sqoop (MapReduce)	Spark JDBC
Song song	`-m 4`, chia theo MIN/MAX của `--split-by`	`numPartitions=4`, chia theo `lowerBound`/`upperBound` do mình truyền vào

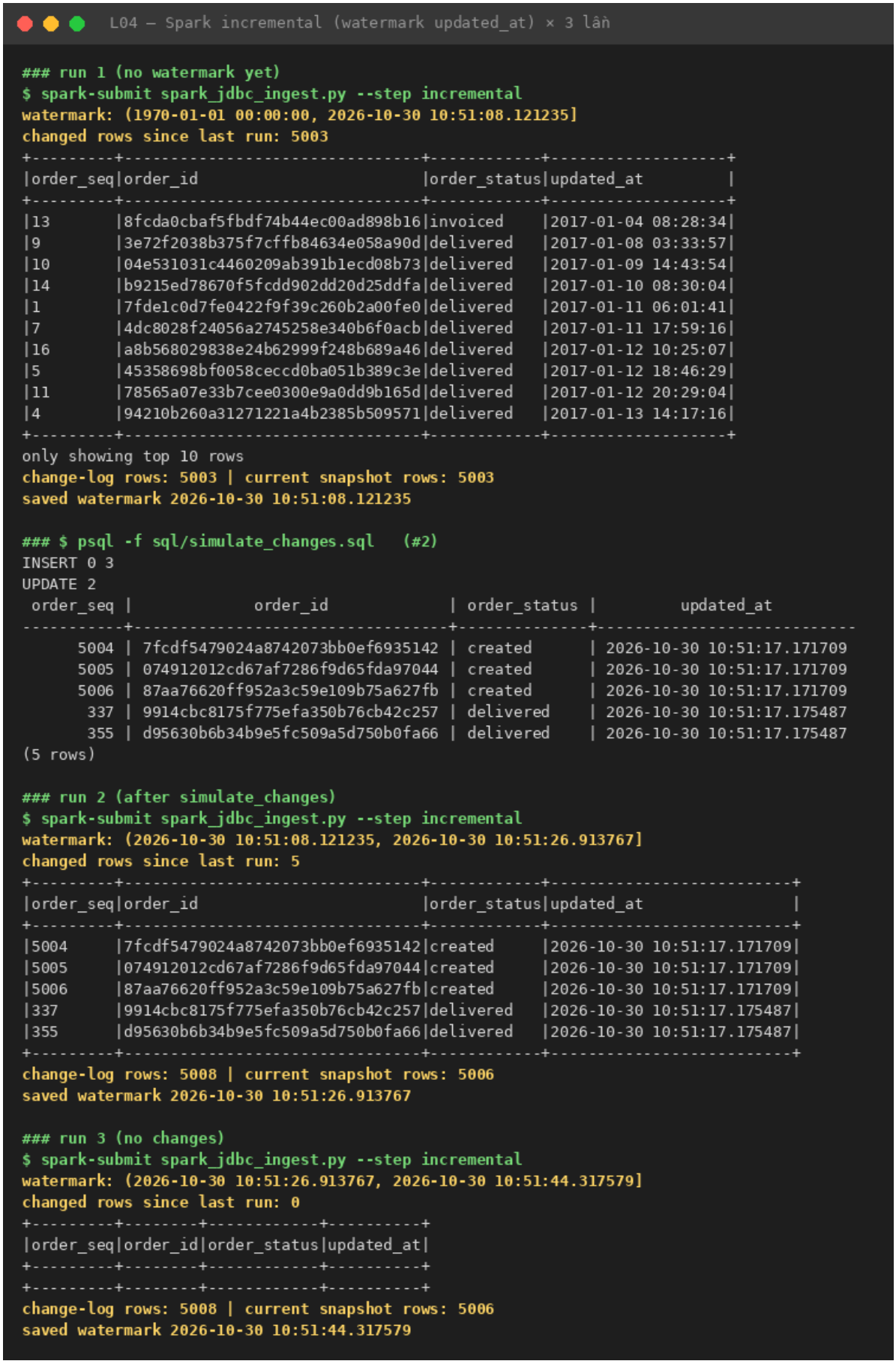
	Sqoop (MapReduce)	Spark JDBC
Thời gian	25.2 s MR job (28.5 s cả lệnh)	3.10 s ghi Parquet (21.2 s cả `spark-submit` 7 bảng, tính cả khởi động JVM/YARN)
Định dạng ra	Text CSV, 1.0 MB	Parquet Snappy phân vùng theo tháng, 680.4 KB
Incremental	Có sẵn (`--incremental`, saved job, merge)	Phải tự viết: watermark, change log, dedup
Biến đổi trong lúc nạp	Không	Có (`withColumn`, `partitionBy`, …)

Sqoop chậm chủ yếu vì phải khởi động một MR job riêng cho mỗi bảng. Spark dùng chung một SparkSession cho cả 7 bảng và ghi thẳng ra định dạng cột. Sqoop 1 cũng đã ngừng phát triển (Apache attic), nên ngày nay thường dùng Spark JDBC hoặc CDC.

5. Spark incremental: watermark, change log, snapshot (TODO 4–5)

Lần chạy	Cửa sổ watermark	Dòng đọc được	Change log	Snapshot
1 (lần đầu)	`(1970-01-01, 2026-10-30 10:51:08.121235]`	5,003	5,003	5,003
sau `simulate_changes` #2 (tạo 5004–5006, cập nhật 337 và 355)				
2	`(10:51:08.121235, now]`	**5**	5,008	**5,006**
3 (không có thay đổi)	`(…, now]`	**0**	5,008	5,006

- Change log chỉ append và giữ mọi phiên bản: 5,003 + 5 = 5,008.
- Snapshot dùng `row_number()` theo `order_id`, sắp `updated_at DESC, _ingested_at DESC`, nên mỗi đơn chỉ còn bản mới nhất: 5,006 đơn.
- Lần chạy thứ ba đọc 0 dòng, nghĩa là watermark được lưu đúng và pipeline idempotent.



Hình 21: spark incremental

Trả lời checkpoint

Q1. Trước khi import, Sqoop chạy bounding query `SELECT MIN(order_seq), MAX(order_seq) FROM orders` (thấy rõ trong log PostgreSQL), được (1, 5000). Nó chia khoảng này thành `-m 4` đoạn **bằng nhau về giá trị**, mỗi đoạn rộng (5000 − 1)/4 ≈ 1250, và mỗi map task chạy một câu `SELECT … WHERE order_seq >= lo AND order_seq < hi`. Đoạn cuối dùng `<=` để lấy cả MAX. Vì `order_seq` dày đặc từ 1 đến 5000, mỗi map nhận đúng 1,250 dòng.

Q2. Sqoop chia theo **khoảng giá trị**, không theo số dòng. Zip code của khách Olist tập trung ở São Paulo (01xxx–19xxx), nên đoạn [1014, 24536) nhận 2,450 dòng còn đoạn [48057, 71578) chỉ có 427 dòng. Task chậm nhất quyết định thời gian của cả job. `customer_id` là chuỗi hash dạng text. Sqoop mặc định từ chối split theo cột text và báo lỗi, trừ khi bật `-Dorg.apache.sqoop.splitter.allow_text_splitter=true`. Kể cả khi bật, việc chia theo thứ tự chuỗi vẫn kém tin cậy (phụ thuộc collation), và split có thể lệch hoặc rỗng. Nên chọn một cột số, phân bố đều và có index, như `order_seq`.

Q3. Có. `lowerBound`/`upperBound` chỉ dùng để tính **bước chia** (stride), **không phải bộ lọc**. Spark sinh partition đầu là `order_seq < 1252 OR order_seq IS NULL` và partition cuối là `order_seq >= 3752`, không có cận trên. Vì vậy dòng `order_seq = 9,999` vẫn được đọc và rơi vào **partition cuối**. Nếu nhiều dòng nằm ngoài bounds, partition đầu hoặc cuối sẽ phình to và gây skew. Phải lấy bounds thật (MIN/MAX) trước khi đọc, như script đã làm (`bounds 1..5003`).

Q4. - **Chốt cận trên trước khi đọc:** `wm_new = now()` được lấy trước khi đọc, và query dùng `updated_at > wm_old AND updated_at <= wm_new`. Nếu lấy `now()` sau khi đọc, những dòng được cập nhật **trong lúc** đang đọc sẽ có `updated_at` nằm trước watermark mới mà không nằm trong dữ liệu đã đọc. Lần chạy sau sẽ bỏ qua chúng vĩnh viễn: **mất dữ liệu**. Cận trên cố định cũng giúp các partition JDBC song song nhìn cùng một cửa sổ, và lần sau bắt đầu đúng từ chỗ lần này dừng. - **Chỉ lưu watermark sau khi ghi thành công:** nếu job chết giữa chừng, watermark cũ vẫn còn và lần chạy lại sẽ đọc lại cả cửa sổ. Như vậy dữ liệu được giao **at-least-once**: có thể trùng nhưng không mất. Phần trùng được dedup idempotent bằng `row_number()` theo `order_id` khi dựng snapshot. Nếu lưu watermark trước khi ghi mà ghi thất bại, cửa sổ đó sẽ bị bỏ qua mãi mãi.

Q5. `append` chỉ dựa vào `order_seq > last_value`, nên chỉ bắt được **INSERT** (lần chạy job chỉ lấy 3 dòng mới). `lastmodified` dựa vào `updated_at > last_value`, nên bắt được cả **INSERT và UPDATE** (lấy 5 dòng: 3 mới + 2 đơn 139 và 278). Cần `--merge-key` để gộp bản cũ với bản mới. **Không chế độ nào bắt được DELETE**: dòng đã xoá thì không còn trong bảng để query ra.

Q6. Watermark trên `updated_at` **không** giải quyết được: 1. **DELETE**: dòng bị xoá không bao giờ xuất hiện trong query, nên bản sao trên HDFS giữ dữ liệu "ma" (trừ khi ứng dụng dùng soft delete). 2. **Các lần UPDATE trung gian**: nếu một dòng đổi nhiều lần giữa hai lần chạy (ví dụ `shipped` → `delivered` → `canceled`), ta chỉ thấy trạng thái cuối và mất lịch sử. 3. **`updated_at` không đáng tin**: - ứng dụng quên cập nhật cột, hoặc ghi bằng giờ của app server lệch đồng hồ; - transaction dài commit sau khi watermark đã vượt qua timestamp của nó, nên dòng bị bỏ sót. 4. **Tải lên nguồn**: mỗi lần chạy là một query quét theo `updated_at` (cần index), độ trễ bằng chu kỳ poll.

CDC (Debezium, L08) không query bảng mà đọc **transaction log (WAL)** của PostgreSQL qua logical replication. Mọi INSERT, UPDATE và DELETE được phát ra thành event theo đúng thứ tự commit, kèm ảnh trước và sau. Không phụ thuộc cột `updated_at`, gần như không tạo tải query, và độ trễ chỉ vài giây.

Q7. `-m 50` hoặc `numPartitions=50` mở **50 kết nối đồng thời**, mỗi kết nối chạy một range scan.

Như vậy dễ vượt `max_connections` (PostgreSQL mặc định 100), làm ứng dụng production không lấy được kết nối. Nó cũng tranh CPU, I/O và buffer cache với tải OLTP, và với bảng nhỏ (5,000 dòng) thì mỗi task chỉ có ~100 dòng, chi phí khởi tạo lấn át lợi ích. Thực tế nên giới hạn khoảng 4–8 kết nối, đọc từ **read replica** hoặc ngoài giờ cao điểm, và đặt `fetchsize` hợp lý.

L05 — PySpark cơ bản: DataFrame, Spark SQL và Spark UI

Môi trường: Spark **4.1.3** (image `spark:4.1.3-python3`, Java 17, Python 3.10), `local[4]`, driver 2 GB, `spark.sql.shuffle.partitions=8`. Dữ liệu: `yellow_tripdata_2024-01.parquet` (2,964,624 dòng) và `taxi_zone_lookup.csv`.

Log gốc:

File	Nội dung
`01_run_no_repartition.txt`	lần chạy ghi **không** `repartition`
`03_run_with_repartition.txt`	lần chạy chính, có `repartition` và `--hold 300` để chụp UI
`02_files_no_repartition.txt`, `04_files_with_repartition.txt`	đếm file output
`05_ansi_divide_by_zero.txt`	demo ANSI mode

File nộp

`l05_basics.py`: đã hoàn thành TODO 1–7. Thêm cờ `--no-repartition` để chạy thí nghiệm TODO 7 mà không phải sửa code.

src/l05_basics.py (TODO 1-7)

```
# ------------------------------------------------------------------ 3. cleaning
banner("3. Data quality and cleaning")
# TODO 1: build ONE row with the number of NULLs per column
null_counts = trips.select([F.sum(F.col(c).isNull().cast("int")).alias(c) for c in trips.columns])
null_counts.show(vertical=True)
trips = (trips
         .withColumn("pickup_ts", F.col("tpep_pickup_datetime"))
         .withColumn("duration_min",
                     (F.unix_timestamp("tpep_dropoff_datetime") - F.unix_timestamp("tpep_pickup_datetime")) / 60.0))
months = [f.rsplit("_", 1)[1][:7] for f in files]          # ['2024-01', ...]
rules = {
    "pickup outside the file's month(s)": ~F.date_format("pickup_ts", "yyyy-MM").isin(months),
    "negative or zero fare": F.col("fare_amount") <= 0,
    # TODO 2
    "negative total": F.col("total_amount") < 0,
    "distance <= 0 or > 100 miles": (F.col("trip_distance") <= 0) | (F.col("trip_distance") > 100),
    "duration <= 0 or > 6 h": (F.col("duration_min") <= 0) | (F.col("duration_min") > 360),
}
# one pass to count every rule (conditional aggregation)
trips.select([F.sum(cond.cast("int")).alias(name) for name, cond in rules.items()]).show(vertical=True, truncate=False)
bad = None
for cond in rules.values():
    bad = cond if bad is None else (bad | cond)
clean = (trips.filter(~bad)
         .fillna({"passenger_count": 1, "RatecodeID": 1, "congestion_surcharge": 0.0, "Airport_fee": 0.0})
         .withColumn("pickup_date", F.to_date("pickup_ts"))
         .withColumn("pickup_hour", F.hour("pickup_ts"))
         # ANSI mode: plain '/' raises DIVIDE_BY_ZERO -> try_divide returns NULL instead
         .withColumn("tip_pct", F.round(F.try_divide(F.col("tip_amount"), F.col("fare_amount")) * 100, 1)))
clean = clean.cache()
total, kept = trips.count(), clean.count()
print(f"kept {kept:,} of {total:,} rows ({100 * kept / total:.2f} %)")
# ------------------------------------------------------------------ 4. joins
banner("4. Join with the zone lookup (pickup and dropoff)")
# TODO 3: join clean with zones TWICE: once on PULocationID (-> pu_borough, pu_zone) and once on
#         DOLocationID (-> do_borough, do_zone). Rename the zone columns before each join.
pu = zones.select(F.col("LocationID").alias("PULocationID"), F.col("Borough").alias("pu_borough"),
                  F.col("Zone").alias("pu_zone"))
do = zones.select(F.col("LocationID").alias("DOLocationID"), F.col("Borough").alias("do_borough"),
                  F.col("Zone").alias("do_zone"))
enriched = clean.join(pu, "PULocationID", "left").join(do, "DOLocationID", "left")
enriched.select("pickup_ts", "pu_borough", "pu_zone", "do_borough", "do_zone", "total_amount").show(5, False)
# ------------------------------------------------------------------ 5. groupBy / agg
banner("5. groupBy / agg")
# TODO 4: per pu_borough: trips (count), avg_fare, avg_tip_pct, avg_miles -- ordered by trips desc
by_borough = (enriched.groupBy("pu_borough")
              .agg(F.count("*").alias("trips"),
                   F.round(F.avg("fare_amount"), 2).alias("avg_fare"),
                   F.round(F.avg("tip_pct"), 1).alias("avg_tip_pct"),
                   F.round(F.avg("trip_distance"), 2).alias("avg_miles"))
              .orderBy(F.desc("trips")))
by_borough.show()
by_hour = enriched.groupBy("pickup_hour").agg(F.count("*").alias("trips")).orderBy("pickup_hour")
by_hour.show(24)
# ------------------------------------------------------------------ 6. window functions
banner("6. Window functions")
zone_counts = enriched.groupBy("pu_borough", "pu_zone").agg(F.count("*").alias("trips"))
# TODO 5: top-3 pickup zones per borough with dense_rank() over a window partitioned by pu_borough
w_rank = Window.partitionBy("pu_borough").orderBy(F.desc("trips"))
top3 = (zone_counts.withColumn("rank", F.dense_rank().over(w_rank))
        .filter("rank <= 3").orderBy("pu_borough", "rank"))
top3.show(30, truncate=False)
daily = enriched.groupBy("pickup_date").agg(F.count("*").alias("trips"), F.round(F.sum("total_amount")).cast("long").alias("revenue"))
w_day = Window.orderBy("pickup_date")
# TODO 6: add prev_day_trips (lag), dod_change_pct and running_revenue (sum over rows unbounded preceding)
daily = (daily.withColumn("prev_day_trips", F.lag("trips").over(w_day))
         .withColumn("dod_change_pct", F.round((F.col("trips") / F.col("prev_day_trips") - 1) * 100, 1))
         .withColumn("running_revenue", F.sum("revenue").over(w_day.rowsBetween(Window.unboundedPreceding, 0)))
         .orderBy("pickup_date"))
daily.show(10)
# ------------------------------------------------------------------ 7. Spark SQL
banner("7. Spark SQL on a temp view")
enriched.createOrReplaceTempView("trips")
sql_df = spark.sql("""
    SELECT pu_borough, count(*) AS trips, round(avg(fare_amount), 2) AS avg_fare,
           round(avg(tip_pct), 1) AS avg_tip_pct, round(avg(trip_distance), 2) AS avg_miles
    FROM trips
    GROUP BY pu_borough
    ORDER BY trips DESC
""")
sql_df.show()
same = sql_df.collect() == by_borough.collect()
print("SQL result == DataFrame result:", same)
# ------------------------------------------------------------------ 8. explain
banner("8. explain('formatted')")
by_borough.explain("formatted")
# ------------------------------------------------------------------ 9 + 10. write partitioned parquet
banner("10. Write partitioned Parquet and read it back")
# TODO 7: write enriched (without pickup_ts) as Parquet to OUTPUT, partitioned by pickup_date,
#         mode overwrite. Try it first WITHOUT and then WITH .repartition("pickup_date") -- count the files.
out = enriched.drop("pickup_ts")
if not args.no_repartition:
    out = out.repartition("pickup_date")    # one task (-> one file) per date instead of N files per date
t0 = time.time()
out.write.mode("overwrite").partitionBy("pickup_date").parquet(OUTPUT)
print(f"write (repartition={not args.no_repartition}) took {time.time() - t0:.1f} s")
back = spark.read.parquet(OUTPUT)
print("partitions (dates) written:", back.select("pickup_date").distinct().count())
one_day = back.filter(F.col("pickup_date") == "2024-01-15")
```

Hình 22: code

Kết quả chính (khớp đáp án)

Hạng mục	Kết quả
Số dòng / số partition đầu vào	2,964,624 / 4
3 transformation (filter, withColumn, select)	**0.096 s**, chưa có job nào
`count()` (action)	228,254 chuyến > 10 dặm, 0.28 s
NULL ở 5 cột `passenger_count`, `RatecodeID`, `store_and_fwd_flag`, `congestion_surcharge`, `Airport_fee`	**140,162** mỗi cột
Luật làm sạch	ngoài tháng 18 · fare ≤ 0: 38,341 · total < 0: 35,504 · distance: 60,430 · duration: 2,642
Giữ lại sau làm sạch	**2,867,786 / 2,964,624 (96.73 %)**
Manhattan	2,571,810 chuyến, fare 14.92, tip 21.6 %, 2.31 dặm
Queens	257,573 chuyến, fare 52.77, 12.76 dặm (sân bay)
Giờ đông nhất	18:00 (206,163 chuyến)
Top-3 Manhattan	Midtown Center 140,065 · UES South 140,059 · UES North 133,891
Top-3 Queens	JFK 138,296 · LaGuardia 87,657 · East Elmhurst 11,861
SQL so với DataFrame	`SQL result == DataFrame result: True`
Ghi Parquet phân vùng theo ngày	31 partition, 73 MB, 8.2 s
Đọc lại ngày 2024-01-15	74,743 chuyến, có `PartitionFilters`

1-2. Read Parquet, lazy evaluation

```
WARNING: Using incubator modules: jdk.incubator.vector

================================================================================
1. SparkSession and reading Parquet
================================================================================
Spark 4.1.3 | master local[4] | ANSI mode: true
input files: ['/datasets/data/taxi/yellow_tripdata_2024-01.parquet']
root
 |-- VendorID: integer (nullable = true)
 |-- tpep_pickup_datetime: timestamp_ntz (nullable = true)
 |-- tpep_dropoff_datetime: timestamp_ntz (nullable = true)
 |-- trip_distance: double (nullable = true)
 |-- PULocationID: integer (nullable = true)
 |-- DOLocationID: integer (nullable = true)
 |-- fare_amount: double (nullable = true)
 |-- total_amount: double (nullable = true)

rows: 2964624 | partitions: 4
+--------+--------------------+---------------------+---------------+-------------+----------+------------------+------------+------------+-----------
  -+-----------+-----+-------+----------+------------+--------------------+-----------+------------------+-----------+
|VendorID|tpep_pickup_datetime|tpep_dropoff_datetime|passenger_count|trip_distance|RatecodeID|store_and_fwd_flag|PULocationID|DOLocationID|payment_typ
  e|fare_amount|extra|mta_tax|tip_amount|tolls_amount|improvement_surcharge|total_amount|congestion_surcharge|Airport_fee|
+--------+--------------------+---------------------+---------------+-------------+----------+------------------+------------+------------+-----------
  -+-----------+-----+-------+----------+------------+--------------------+-----------+------------------+-----------+
|2       |2024-01-01 00:57:55 |2024-01-01 01:17:43  |1              |1.72         |1         |N                 |186         |79          |2
           |17.7       |1.0  |0.5    |0.0       |0.0         |1.0                 |22.7       |2.5               |0.0        |
|1       |2024-01-01 00:03:00 |2024-01-01 00:09:36  |1              |1.8          |1         |N                 |140         |236         |1
           |10.0       |3.5  |0.5    |3.75      |0.0         |1.0                 |18.75      |2.5               |0.0        |
|1       |2024-01-01 00:17:06 |2024-01-01 00:35:01  |1              |4.7          |1         |N                 |236         |79          |1
           |23.3       |3.5  |0.5    |3.0       |0.0         |1.0                 |31.3       |2.5               |0.0        |
+--------+--------------------+---------------------+---------------+-------------+----------+------------------+------------+------------+-----------
  -+-----------+-----+-------+----------+------------+--------------------+-----------+------------------+-----------+
only showing top 3 rows
root
 |-- LocationID: integer (nullable = true)
 |-- Borough: string (nullable = true)
 |-- Zone: string (nullable = true)
 |-- service_zone: string (nullable = true)


================================================================================
2. Transformations are lazy, actions run jobs
================================================================================
building 3 transformations took 0.096 s  (no Spark job yet -- check the UI)
count() = 228254 long trips, took 0.28 s  (an ACTION: a job ran)
```

```
3. TODO 1-2: NULL counts and cleaning rules

3. Data quality and cleaning
================================================================================
-RECORD 0-----------------------
 VendorID              | 0
 tpep_pickup_datetime  | 0
 tpep_dropoff_datetime | 0
 passenger_count       | 140162
 trip_distance         | 0
 RatecodeID            | 140162
 store_and_fwd_flag    | 140162
 PULocationID          | 0
 DOLocationID          | 0
 payment_type          | 0
 fare_amount           | 0
 extra                 | 0
 mta_tax               | 0
 tip_amount            | 0
 tolls_amount          | 0
 improvement_surcharge | 0
 total_amount          | 0
 congestion_surcharge  | 140162
 Airport_fee           | 140162

-RECORD 0-----------------------------------
 pickup outside the file's month(s) | 18
 negative or zero fare              | 38341
 negative total                     | 35504
 distance <= 0 or > 100 miles       | 60430
 duration <= 0 or > 6 h             | 2642

kept 2,867,786 of 2,964,624 rows (96.73 %)
```

4-5. TODO 3-4: join zones x2 + groupBy

```
+-------------------+----------+--------------------------+----------+--------------------+------------+
|pickup_ts          |pu_borough|pu_zone                   |do_borough|do_zone             |total_amount|
+-------------------+----------+--------------------------+----------+--------------------+------------+
|2024-01-01 00:57:55|Manhattan |Penn Station/Madison Sq West|Manhattan |East Village        |22.7        |
|2024-01-01 00:03:00|Manhattan |Lenox Hill East           |Manhattan |Upper East Side North|18.75       |
|2024-01-01 00:17:06|Manhattan |Upper East Side North     |Manhattan |East Village        |31.3        |
|2024-01-01 00:36:38|Manhattan |East Village              |Manhattan |SoHo                |17.0        |
|2024-01-01 00:46:51|Manhattan |SoHo                      |Manhattan |Lower East Side     |16.1        |
+-------------------+----------+--------------------------+----------+--------------------+------------+
only showing top 5 rows

================================================================================
5. groupBy / agg
================================================================================
+-------------+-------+--------+-----------+---------+
|   pu_borough|  trips|avg_fare|avg_tip_pct|avg_miles|
+-------------+-------+--------+-----------+---------+
|    Manhattan|2571810|   14.92|       21.6|     2.31|
|       Queens| 257573|   52.77|       21.8|    12.76|
|     Brooklyn|  22257|   28.77|        6.9|     6.15|
|      Unknown|   9614|   18.93|       20.2|     3.31|
|        Bronx|   5750|   32.38|        0.8|     7.75|
|          N/A|    688|   71.27|       12.4|     9.26|
|          EWR|     49|   84.37|       11.6|     5.77|
|Staten Island|     45|   46.94|        5.8|     10.2|
+-------------+-------+--------+-----------+---------+

+-----------+------+
|pickup_hour| trips|
+-----------+------+
|          0| 75156|
|          1| 50431|
|          2| 34923|
|          3| 22916|
|          4| 15252|
|          5| 17484|
|          6| 39392|
|          7| 80811|
|          8|113426|
|          9|125530|
|         10|135322|
|         11|146646|
|         12|159808|
|         13|165198|
|         14|177868|
|         15|183840|
|         16|184816|
|         17|200144|
|         18|206163|
|         19|178696|
|         20|155463|
|         21|155808|
|         22|138149|
|         23|104544|
+-----------+------+
```

```
6. TODO 5-6: dense_rank top-3, lag, running sum

6. Window functions
================================================================================
+-------------+------------------------------+------+----+
|pu_borough   |pu_zone                       |trips |rank|
+-------------+------------------------------+------+----+
|Bronx        |Mott Haven/Port Morris        |436   |1   |
|Bronx        |Co-Op City                    |362   |2   |
|Bronx        |East Concourse/Concourse Village|300   |3   |
|Brooklyn     |Downtown Brooklyn/MetroTech   |1292  |1   |
|Brooklyn     |Brooklyn Heights              |1167  |2   |
|Brooklyn     |Fort Greene                   |1048  |3   |
|EWR          |Newark Airport                |49    |1   |
|Manhattan    |Midtown Center                |140065|1   |
|Manhattan    |Upper East Side South         |140059|2   |
|Manhattan    |Upper East Side North         |133891|3   |
|N/A          |Outside of NYC                |688   |1   |
|Queens       |JFK Airport                   |138296|1   |
|Queens       |LaGuardia Airport             |87657 |2   |
|Queens       |East Elmhurst                 |11861 |3   |
|Staten Island|Arrochar/Fort Wadsworth       |18    |1   |
|Staten Island|Heartland Village/Todt Hill   |4     |2   |
|Staten Island|Saint George/New Brighton     |4     |2   |
|Staten Island|South Beach/Dongan Hills      |4     |2   |
|Staten Island|Westerleigh                   |2     |3   |
|Staten Island|New Dorp/Midland Beach        |2     |3   |
|Staten Island|Oakwood                       |2     |3   |
|Staten Island|Bloomfield/Emerson Hill       |2     |3   |
|Staten Island|Grymes Hill/Clifton           |2     |3   |
|Unknown      |N/A                           |9614  |1   |
+-------------+------------------------------+------+----+

+-----------+------+-------+-------------+-------------+---------------+
|pickup_date| trips|revenue|prev_day_trips|dod_change_pct|running_revenue|
+-----------+------+-------+-------------+-------------+---------------+
| 2024-01-01| 75440|2335972|         NULL|         NULL|        2335972|
| 2024-01-02| 72940|2251748|        75440|         -3.3|        4587720|
| 2024-01-03| 79903|2336980|        72940|          9.5|        6924700|
| 2024-01-04|100085|2779093|        79903|         25.3|        9703793|
| 2024-01-05|100073|2700263|       100085|          0.0|       12404056|
| 2024-01-06| 93682|2400380|       100073|         -6.4|       14804436|
| 2024-01-07| 65174|1870461|        93682|        -30.4|       16674897|
| 2024-01-08| 77683|2198845|        65174|         19.2|       18873742|
| 2024-01-09| 90303|2322347|        77683|         16.2|       21196089|
| 2024-01-10| 92475|2526815|        90303|          2.4|       23722904|
+-----------+------+-------+-------------+-------------+---------------+
only showing top 10 rows

26/10/30 10:56:23 WARN WindowExec: No Partition Defined for Window operation! Moving all data to a single partition, this can cause serious
  performance degradation.
```

Spark 4 bật **ANSI mode** mặc định: `1/0` báo lỗi `[DIVIDE_BY_ZERO]` thay vì trả NULL. Vì vậy script tính `tip_pct` bằng `F.try_divide`.

```
Spark 4 ANSI mode: 1/0 vs try_divide

spark.sql.ansi.enabled = true
+-----------------+
|try_divide_1_by_0|
+-----------------+
|             NULL|
+-----------------+
ArithmeticException [DIVIDE_BY_ZERO] Division by zero. Use `try_divide` to tolerate divisor being 0 and return NULL instead. If necessary set
  "spark.sql.ansi.enabled" to "false" to bypass this error. SQLSTATE: 22012
```

Hình 23: ansi

Plan và Spark UI

7-8. Spark SQL == DataFrame; explain('formatted')

```
================================================================================
7. Spark SQL on a temp view
================================================================================
26/10/30 10:56:24 WARN SparkStringUtils: Truncated the string representation of a plan since it was too large. This behavior can be adjusted by
  setting 'spark.sql.debug.maxToStringFields'.
+-------------+-------+--------+-----------+---------+
|   pu_borough|  trips|avg_fare|avg_tip_pct|avg_miles|
+-------------+-------+--------+-----------+---------+
|    Manhattan|2571810|   14.92|       21.6|     2.31|
|       Queens| 257573|   52.77|       21.8|    12.76|
|     Brooklyn|  22257|   28.77|        6.9|     6.15|
|      Unknown|   9614|   18.93|       20.2|     3.31|
|        Bronx|   5750|   32.38|        0.8|     7.75|
|          N/A|    688|   71.27|       12.4|     9.26|
|          EWR|     49|   84.37|       11.6|     5.77|
|Staten Island|     45|   46.94|        5.8|     10.2|
+-------------+-------+--------+-----------+---------+

SQL result == DataFrame result: True

================================================================================
8. explain('formatted')
================================================================================
== Physical Plan ==
AdaptiveSparkPlan (47)
+- == Final Plan ==
   ResultQueryStage (31)
   +- * Sort (30)
      +- AQEShuffleRead (29)
         +- ShuffleQueryStage (28), Statistics(sizeInBytes=464.0 B, rowCount=8)
            +- Exchange (27)
               +- * HashAggregate (26)
                  +- AQEShuffleRead (25)
                     +- ShuffleQueryStage (24), Statistics(sizeInBytes=1968.0 B, rowCount=24)
                        +- Exchange (23)
                           +- * HashAggregate (22)
                              +- * Project (21)
                                 +- * BroadcastHashJoin LeftOuter BuildRight (20)
                                    :- * Project (14)
                                    :  +- * BroadcastHashJoin LeftOuter BuildRight (13)
                                    :     :- TableCacheQueryStage (7), Statistics(sizeInBytes=462.1 MiB, rowCount=2.87E+6)
                                    :     :  +- InMemoryTableScan (1)
                                    :     :        +- InMemoryRelation (2)
                                    :     :              +- * Project (6)
                                    :     :                 +- * Filter (5)
                                    :     :                    +- * ColumnarToRow (4)
                                    :     :                       +- Scan parquet  (3)
                                    :     +- BroadcastQueryStage (12), Statistics(sizeInBytes=1026.1 KiB, rowCount=265)
                                    :        +- BroadcastExchange (11)
                                    :           +- * Project (10)
                                    :              +- * Filter (9)
                                    :                 +- Scan csv  (8)
                                    +- BroadcastQueryStage (19), Statistics(sizeInBytes=1026.1 KiB, rowCount=265)
                                       +- BroadcastExchange (18)
                                          +- * Project (17)
                                             +- * Filter (16)
                                                +- Scan csv  (15)

+- == Initial Plan ==
   Sort (46)
   +- Exchange (45)
      +- HashAggregate (44)
         +- Exchange (43)
            +- HashAggregate (42)
               +- Project (41)
                  +- BroadcastHashJoin LeftOuter BuildRight (40)
                     :- Project (36)
                     :  +- BroadcastHashJoin LeftOuter BuildRight (35)
```

8. partial HashAggregate -> Exchange -> final HashAggregate

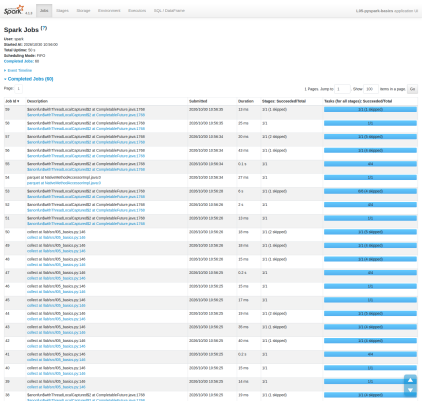
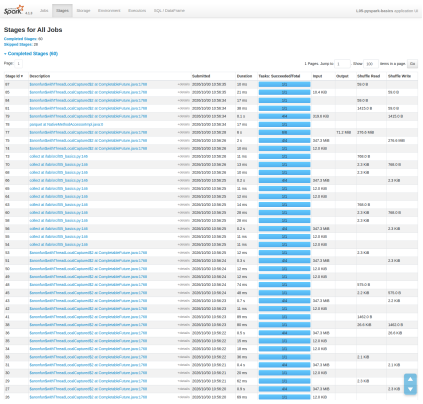
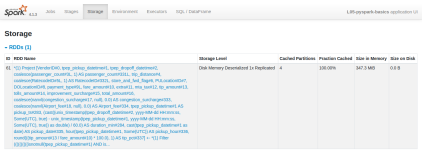
```
(22) HashAggregate [codegen id : 3]
Keys [1]: [pu_borough#1112]
Functions [4]: [partial_count(1), partial_avg(fare_amount#10), partial_avg(tip_pct#337), partial_avg(trip_distance#4)]
Aggregate Attributes [7]: [count#2026L, sum#2027, count#2028L, sum#2029, count#2030L, sum#2031, count#2032L]
Results [8]: [pu_borough#1112, count#2033L, sum#2034, count#2035L, sum#2036, count#2037L, sum#2038, count#2039L]

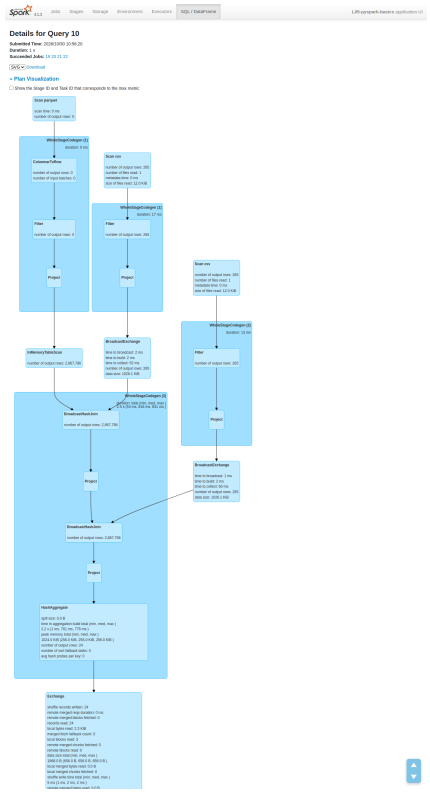
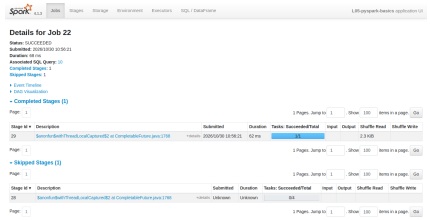
(23) Exchange
Arguments: hashpartitioning(pu_borough#1112, 8), ENSURE_REQUIREMENTS, [plan_id=2521]

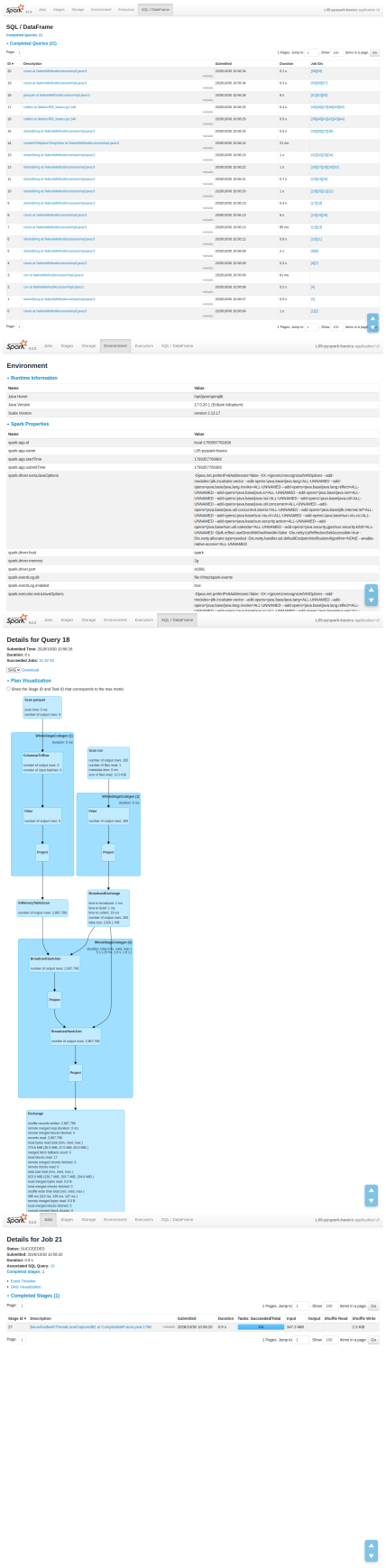
(24) ShuffleQueryStage
Output [8]: [pu_borough#1112, count#2033L, sum#2034, count#2035L, sum#2036, count#2037L, sum#2038, count#2039L]
Arguments: 3

(25) AQEShuffleRead
Arguments: coalesced

(26) HashAggregate [codegen id : 4]
Keys [1]: [pu_borough#1112]
Functions [4]: [count(1), avg(fare_amount#10), avg(tip_pct#337), avg(trip_distance#4)]
```

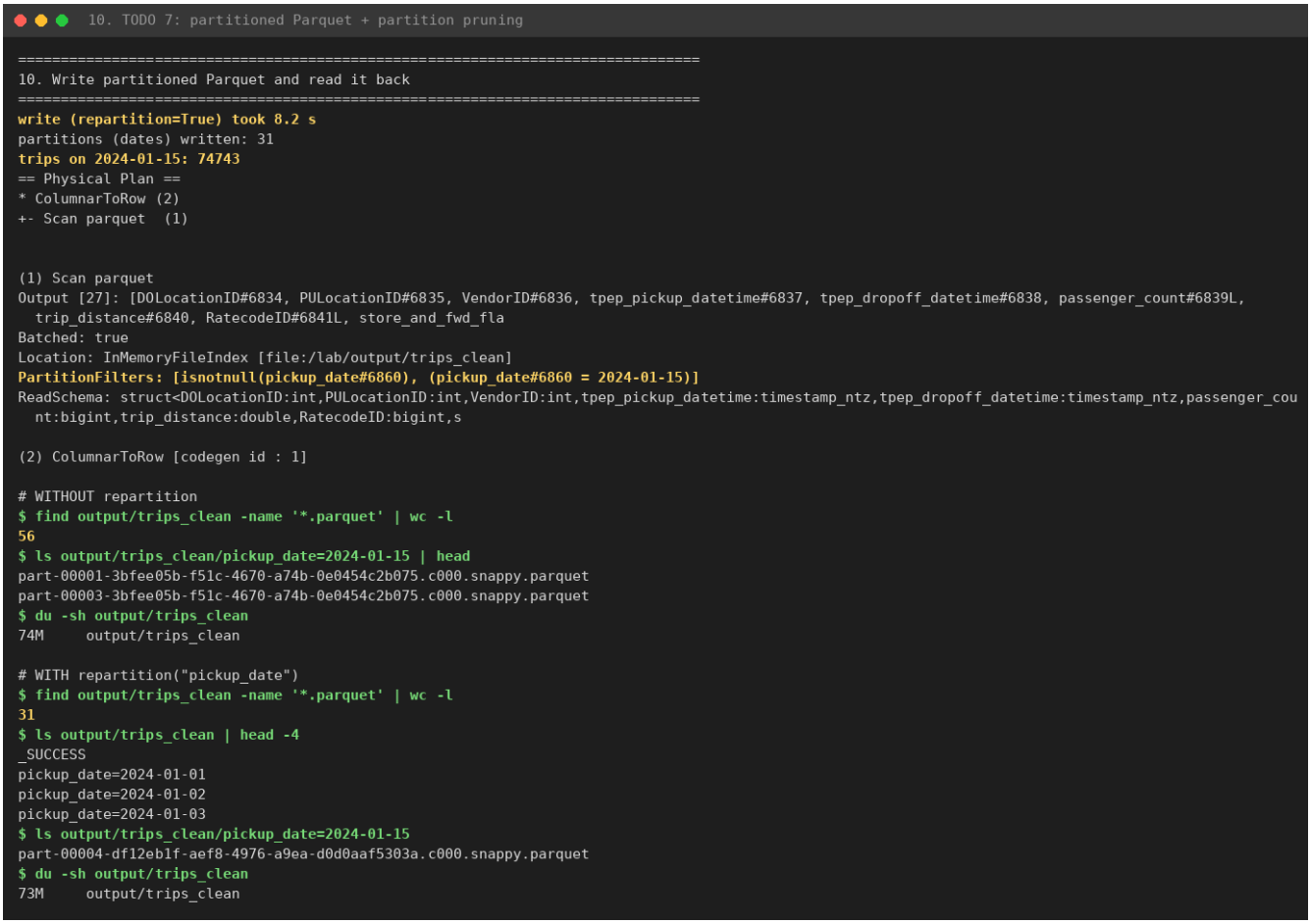
Tab	Ảnh	Nhận xét
Jobs		mỗi action (`count`, `showString`, `collect`, `parquet`) sinh ra một hoặc nhiều job
Stages		có cột Shuffle Read/Write; stage 4 task đọc từ cache
Storage		DataFrame `clean` được cache: 4/4 partition, **100 %**, **347.3 MiB** trong bộ nhớ

Tab	Ảnh	Nhận xét
SQL (query 10 = `by_borough.show()`)		`InMemoryTableScan` 2,867,786 dòng → 2 × `BroadcastHashJoin` (265 dòng) → `HashAggregate` partial: **24 dòng** → `Exchange`
Job 22		stage 28 skipped (dùng lại shuffle của job 21); stage 29 chạy 1 task, Shuffle Read 2.3 KiB

Tab	Ảnh	Nhận xét
Khác		

Ghi Parquet phân vùng (TODO 7)

	Không `repartition`	`repartition("pickup_date")`
Số file `.parquet`	**56**	**31** (1 file/ngày)
File trong `pickup_date=2024-01-15`	2	1
Dung lượng	74 MB	73 MB
Thời gian ghi	6.8 s	8.2 s (thêm 1 shuffle)



Hình 24: write + pruning

Không `repartition`, mỗi task trong 4 task ghi một file cho **mỗi ngày mà nó chứa**. Ở đây chỉ ra 56 file chứ không phải 4 × 31 = 124, vì file nguồn gần như đã sắp theo thời gian: mỗi task chỉ chứa một khoảng ngày liền nhau, và các ngày ở ranh giới bị tách thành 2 file. Với dữ liệu không có thứ tự và hàng trăm task, số file sẽ là N × 31. Khi đọc lại, `PartitionFilters: [isnotnull(pickup_date), (pickup_date = 2024-01-15)]` cho thấy Spark chỉ mở **một thư mục**.

Trả lời checkpoint

Q1. Spark chia file thành input partition dựa trên: - `spark.sql.files.maxPartitionBytes`, mặc định 128 MB; - mục tiêu có ít nhất `defaultParallelism` split (= 4 core với `local[4]`), nên kích thước split được hạ xuống khoảng 48 MB / 4.

Parquet chỉ cắt được ở ranh giới **row group**: task nào nhận được điểm bắt đầu của row group thì đọc row group đó. Vì vậy dù chỉ có một file 48 MB vẫn ra **4 partition**, và một partition có thể rỗng nếu số row group ít hơn 4.

Q2. `filter`, `withColumn` và `select` là **transformation**: chúng chỉ thêm node vào logical plan, không đụng tới dữ liệu, nên mất 0.096 s. `count()` là **action**: Catalyst tối ưu plan, DAG scheduler tạo stage và task, rồi mới đọc file (0.28 s). `show`, `collect`, `take`, `write` và `count` cũng là action. Plan dựng trong vài mili-giây vì đó chỉ là thao tác trên cây biểu thức trong bộ nhớ của driver.

Q3 (lazy evaluation và DAG). `by_borough.show()` là SQL query 10. Nó sinh ra **4 job** (19–22) với AQE:

Job	Stage	Số task	Việc làm
19	25	1	dựng broadcast bảng zone cho **pickup** (`BroadcastExchange`)
20	26	1	dựng broadcast bảng zone cho **dropoff**
21	27	**4**	đọc `InMemoryTableScan` (cache), 2 × `BroadcastHashJoin`, `HashAggregate` partial, **ghi shuffle** (`Exchange hashpartitioning(pu_borough, 8)`)
22	28 (skipped, dùng lại output của 27), **29**	1	`AQEShuffleRead coalesced` (8 → 1 partition), `HashAggregate` final, `TakeOrderedAndProject` cho `orderBy + show`

- **Ranh giới stage là các toán tử `Exchange` (shuffle).**
- Broadcast chạy thành job riêng.
- Với AQE, mỗi query stage được nộp thành một job riêng, nên stage của job trước hiện là skipped trong job sau.
- Khi chạy toàn bộ `by_borough` (ví dụ `collect`), plan còn có `Exchange rangepartitioning(trips DESC, 8)` cho `orderBy`, tức là thêm một stage sort nữa (xem `explain`).
- Không có gì chạy cho tới khi gọi `show()`. Các transformation ở mục 4–5 chỉ dựng DAG.

Q4. 5 cột có cùng 140,162 NULL vì đó là **cùng 140,162 chuyến**. Các bản ghi này được gửi lên thiếu toàn bộ nhóm trường "metadata" (thường từ một vendor hoặc từ chuyến street-hail), chứ không phải 5 lỗi độc lập. Có ba cách xử lý: - giữ lại và gán mặc định, như script làm: `passenger_count=1`, `RatecodeID=1`, phụ phí = 0; - giữ NULL và chỉ loại khỏi các chỉ số cần những cột đó; - tách ra vùng quarantine.

Cách nào cũng cần thống nhất với chủ dữ liệu và ghi lại trong tài liệu.

Q5 (đọc formatted plan). Đọc từ dưới lên: 1. `Scan parquet` với `PushedFilters` (`GreaterThan(fare_amount,0` `LessThanOrEqual(trip_distance,100.0)`...): luật làm sạch được đẩy xuống reader Parquet. 2. `InMemoryTableScan`: đọc từ cache. 3. 2 × `BroadcastExchange` và `BroadcastHashJoin LeftOuter BuildRight`. 4. `HashAggregate` (22) với `partial_count(1), partial_avg(...)`. Mỗi task **gộp trước** các dòng của nó theo `pu_borough`. `avg` được tách thành cặp `(sum, count)`, nên mỗi task chỉ phát ra khoảng 6–8 dòng; tổng cộng **24 dòng** đi qua shuffle thay vì 2.87 triệu. 5. `Exchange hashpartitioning(pu_borough, 8)`: **shuffle**, cũng là ranh giới stage. 6. `AQEShuffleRead coalesced`: AQE gộp 8 partition shuffle nhỏ lại. 7. `HashAggregate` (26) final với `count(1), avg(...)`: gộp các kết quả partial. 8. `Exchange rangepartitioning` và `Sort` cho `orderBy`.

Partial/final aggregate chính là **combiner của L02**. Tách `avg` thành `(sum, count)` cũng giống hệt cách làm ở bài stretch L02 (combiner tính giá cước trung bình).

Q6. Bảng zone chỉ có 265 dòng (12 KB, khoảng 1 MB trong bộ nhớ theo `Statistics(sizeInBytes=1026.1 KiB)`), nhỏ hơn nhiều so với `spark.sql.autoBroadcastJoinThreshold` (10 MB). Spark gửi một bản sao cho mọi task, nên 2.87 triệu chuyến **không phải shuffle**. Khi cả hai bên đều lớn hơn ngưỡng (hoặc đặt ngưỡng = -1, hoặc thống kê kích thước vượt ngưỡng), Spark shuffle cả hai bên theo key và dùng **SortMergeJoin**. L06 đo sự khác biệt này.

Q7. Sau khi `cache()`: - **Plan** đọc từ `InMemoryRelation`/`InMemoryTableScan` thay vì quét và lọc lại Parquet. - **Tab Storage** hiện RDD đã cache: 4 partition, 100 %, 347.3 MiB. - Các action sau nhanh hơn, ví dụ `clean.count()` lần đầu (materialize cache) mất khoảng 6 s, còn các query sau chỉ 0.3–1.5 s.

Cache là ý tồi khi: - dữ liệu chỉ dùng một lần; - dữ liệu không vừa bộ nhớ, gây spill, eviction và áp lực GC (347 MiB cache so với 48 MB Parquet nén là hệ số khoảng 7×); - đọc nguồn vốn đã rẻ (Parquet có column pruning có thể nhanh hơn cache dạng hàng rộng); - quên `unpersist()`, làm giữ bộ nhớ không cần thiết.

Q8. `partitionBy` ghi **một file cho mỗi task × mỗi giá trị partition** mà task đó chứa. Không `repartition`, mỗi task có thể chứa dòng của nhiều ngày, và số file có thể lên tới N task × 31 ngày. Ở đây đo được 56 file; trên cluster thật sẽ là hàng nghìn file nhỏ, gây hại cho NameNode và cho tốc độ đọc. `repartition("pickup_date")` shuffle trước để mọi dòng của một ngày về cùng một task, nên ra **đúng 1 file/ngày (31 file)**. Cái giá là thêm một shuffle (6.8 s → 8.2 s). Cần cẩn thận khi một giá trị quá lớn (skew): một task sẽ phải ghi cả file khổng lồ. Khi đó dùng `repartition(n, "pickup_date", salt)` hoặc `maxRecordsPerFile`.

L06 — Spark performance: shuffle, join, skew, AQE, partitioning, cache

Môi trường: Spark 4.1.3 `local[4]`, driver 2 GB. Dữ liệu: taxi 2024-01 (2.96 triệu dòng) và một fact tổng hợp 10 triệu dòng, trong đó 60 % có key = 0. Script được chạy **2 lần**; báo cáo dùng lần 2.

File nộp

File	Nội dung
`l06_experiments.py`	TODO 1–4: E1 AQE on, E2 hint `broadcast()`, E3 salting và AQE skew join
`results.csv`	34 dòng (2 lần chạy × 17 variant)
`RESULTS.md`	bảng lần 2, nhận xét E1–E5, khuyến nghị cho cluster 20 node
`01_run1.txt`, `02_run2.txt`	log console
`03_e3_task_quantiles.txt`	phân vị thời gian task và số bản ghi shuffle/task (REST API)

src/l06_experiments.py (TODO 1-4)

```
    run("E1", "AQE off, shuffle.partitions=200", q())
    set_conf(spark__sql__shuffle__partitions=8)
    run("E1", "AQE off, shuffle.partitions=8", q())
    set_conf(spark__sql__shuffle__partitions=1)
    run("E1", "AQE off, shuffle.partitions=1", q())
    # TODO 1: turn AQE back ON (keep shuffle.partitions=200) and run the same query as
    #         variant "AQE on,  shuffle.partitions=200". What does AQEShuffleRead do?
    set_conf(spark__sql__adaptive__enabled=True, spark__sql__shuffle__partitions=200)
    run("E1", "AQE on,  shuffle.partitions=200", q(), "AQE coalesces small shuffle partitions")
def e2_broadcast_vs_smj():
    q = lambda: (trips_df().join(zones_df(), F.col("PULocationID") == F.col("LocationID"))
                 .groupBy("Borough").agg(F.count("*").alias("trips")))
    reset_conf()
    set_conf(spark__sql__adaptive__enabled=False)
    run("E2", "AQE off, broadcast (threshold 10MB)", q())
    set_conf(spark__sql__autoBroadcastJoinThreshold=-1)
    run("E2", "AQE off, sort-merge (threshold -1)", q())
    set_conf(spark__sql__adaptive__enabled=True, spark__sql__autoBroadcastJoinThreshold=-1)
    # TODO 2: with AQE on and the threshold still -1, force a broadcast join with the F.broadcast()
    #         hint on zones_df() and record it as variant "AQE on,  threshold -1 + broadcast() hint".
    run("E2", "AQE on,  threshold -1 + broadcast() hint",
        trips_df().join(F.broadcast(zones_df()), F.col("PULocationID") == F.col("LocationID"))
        .groupBy("Borough").agg(F.count("*").alias("trips")), "an explicit hint beats the threshold")
def skew_inputs():
    fact = (spark.range(N_FACT)
            .withColumn("key", F.when(F.rand(seed=7) < HOT_SHARE, F.lit(0))
                        .otherwise((F.rand(seed=11) * N_KEYS).cast("long")))
            .withColumn("amount", (F.rand(seed=3) * 100).cast("double")))
    dim = (spark.range(N_KEYS).withColumnRenamed("id", "key")
           .withColumn("segment", F.concat(F.lit("seg_"), (F.col("key") % 50).cast("string"))))
    return fact, dim
def e3_skew():
    reset_conf()
    fact, dim = skew_inputs()
    fact.groupBy((F.col("key") == 0).alias("is_hot_key")).count().show()
    # force a shuffle join (dim is 200k rows -> could be broadcast, which would hide the skew)
    set_conf(spark__sql__autoBroadcastJoinThreshold=-1, spark__sql__adaptive__enabled=False,
             spark__sql__shuffle__partitions=16)
    joined = fact.join(dim, "key").groupBy("segment").agg(F.sum("amount").alias("amount"))
    base = run("E3", "skewed SMJ, AQE off", joined)
    # Fix 1: SALTING -- spread the hot key over SALT sub-keys. Only the HOT key is salted (random
    # salt 0..SALT-1); every other key keeps salt 0. The dimension row of the hot key is replicated
    # SALT times, all other dimension rows once -> almost no extra data.
    SALT = 64   # >> shuffle partitions, so hash collisions of (key, salt) even out
    # TODO 3: SALTING of the hot key only.
    #   fact_s: new int column "salt" = random 0..SALT-1 when key == 0, else 0
    #   dim_s : dim rows with key != 0 get salt 0; the dim row with key == 0 is cross-joined with
    #           spark.range(SALT) (as int column "salt") -> SALT copies
    #   salted: fact_s.join(dim_s, ["key", "salt"]) then the same groupBy/agg as `joined`
    hot = F.col("key") == 0
    fact_s = fact.withColumn("salt", F.when(hot, (F.rand(seed=5) * SALT).cast("int")).otherwise(F.lit(0)))
    salts = spark.range(SALT).select(F.col("id").cast("int").alias("salt"))
    dim_s = (dim.filter(~hot).withColumn("salt", F.lit(0))
             .unionByName(dim.filter(hot).crossJoin(salts)))
    salted = fact_s.join(dim_s, ["key", "salt"]).groupBy("segment").agg(F.sum("amount").alias("amount"))
    run("E3", f"salted SMJ (salt={SALT}), AQE off", salted)
    # Fix 2: AQE skew join -- split the skewed shuffle partition at runtime
    # TODO 4: enable AQE + skew-join handling. Our data is small, so lower the thresholds:
    #         advisoryPartitionSizeInBytes=4m, skewJoin.skewedPartitionThresholdInBytes=16m,
    #         skewJoin.skewedPartitionFactor=3, and disable coalescePartitions for a clean comparison.
    set_conf(spark__sql__adaptive__enabled=True, spark__sql__adaptive__skewJoin__enabled=True,
             spark__sql__adaptive__coalescePartitions__enabled=False,
             spark__sql__adaptive__advisoryPartitionSizeInBytes="4m",
             spark__sql__adaptive__skewJoin__skewedPartitionThresholdInBytes="16m",
             spark__sql__adaptive__skewJoin__skewedPartitionFactor=3)
```

Hình 25: code

Bảng kết quả (lần 2)

```
spark-submit l06_experiments.py (run 2)

Spark 4.1.3 | cores 4 | input ['/datasets/data/taxi/yellow_tripdata_2024-01.parquet']

===== E1: e1_shuffle_partitions
[E1] AQE off, shuffle.partitions=200           2.20 s | tasks=204 skewiest stage: median=6.0 ms max=60.0 ms | Exchange
[E1] AQE off, shuffle.partitions=8             0.33 s | tasks=12 skewiest stage: median=177.0 ms max=178.0 ms | Exchange
[E1] AQE off, shuffle.partitions=1             0.44 s | tasks=5 skewiest stage: median=232.0 ms max=233.0 ms | Exchange
[E1] AQE on,  shuffle.partitions=200           0.53 s | tasks=5 skewiest stage: median=271.0 ms max=274.0 ms | AQEShuffleRead Exchange

===== E2: e2_broadcast_vs_smj
[E2] AQE off, broadcast (threshold 10MB)       1.20 s | tasks=13 skewiest stage: median=626.0 ms max=635.0 ms | BroadcastExchange BroadcastHashJoin
  Exchange
[E2] AQE off, sort-merge (threshold -1)        1.51 s | tasks=21 skewiest stage: median=386.0 ms max=591.0 ms | Exchange SortMergeJoin
[E2] AQE on,  threshold -1 + broadcast() hint  0.40 s | tasks=6 skewiest stage: median=157.0 ms max=168.0 ms | AQEShuffleRead BroadcastExchange
  BroadcastHashJoin Exchange

===== E3: e3_skew
+----------+-------+
|is_hot_key|  count|
+----------+-------+
|      true|5998366|
|     false|4001634|
+----------+-------+

[E3] skewed SMJ, AQE off                       4.10 s | tasks=40 skewiest stage: median=165.0 ms max=2366.0 ms | Exchange SortMergeJoin
[E3] salted SMJ (salt=64), AQE off             4.01 s | tasks=56 skewiest stage: median=590.0 ms max=1197.0 ms | Exchange SortMergeJoin
[E3] skewed SMJ, AQE skewJoin on               2.22 s | tasks=27 skewiest stage: median=186.0 ms max=682.0 ms | AQEShuffleRead Exchange
  SortMergeJoin(skew=true)
same result with and without salting: True (500061629.0)

===== E4: e4_repartition_vs_coalesce
[E4] repartition(64)                          7.25 s | files=64 | tasks=68 | Exchange
[E4] repartition(4)                           4.40 s | files=4 | tasks=8 | Exchange
[E4] coalesce(4)                              3.12 s | files=3 | tasks=4 | Coalesce
[E4] coalesce(1)                              6.95 s | files=1 | tasks=1 | Coalesce

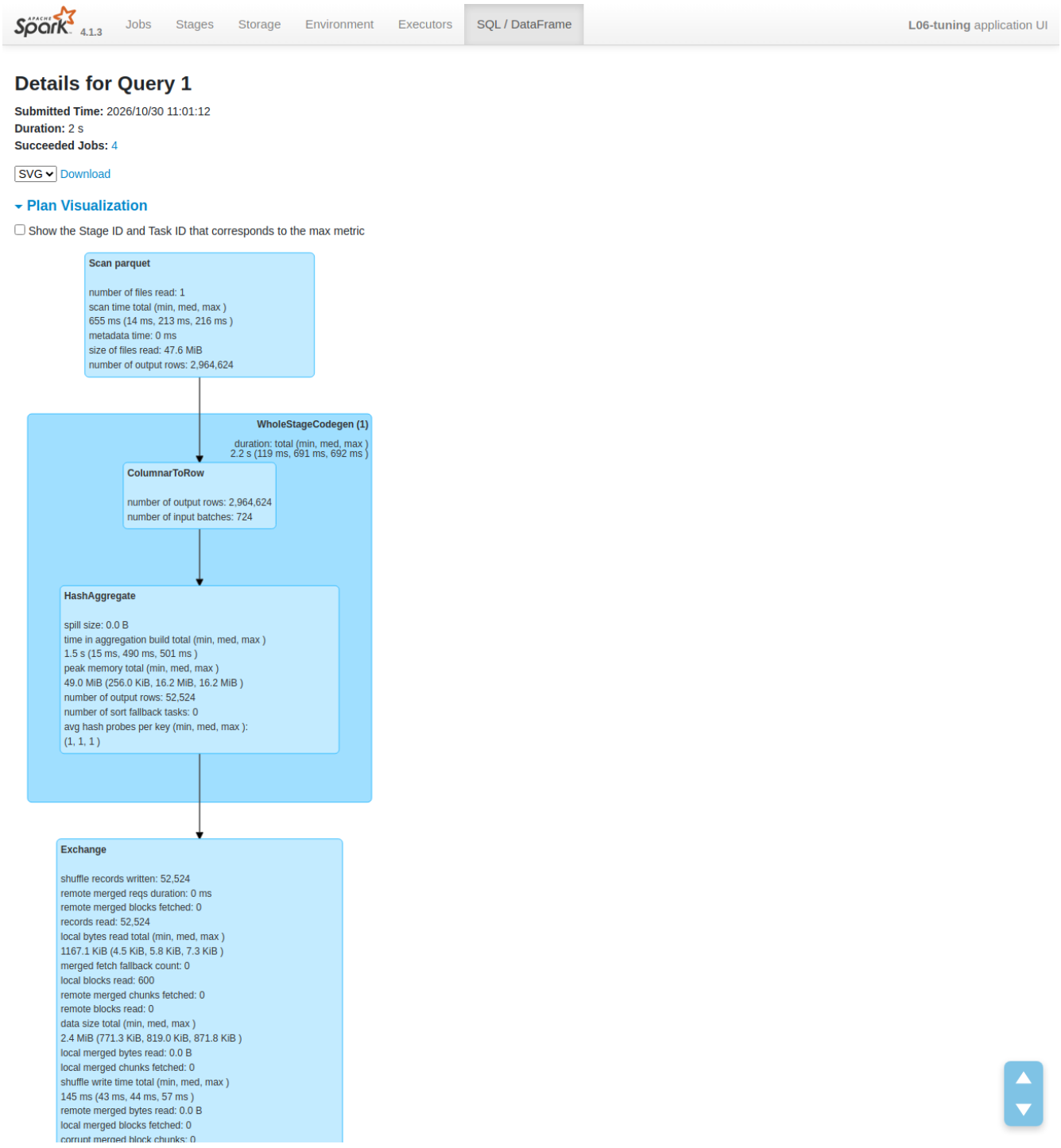
===== E5: e5_cache
[E5] no cache                                 6.40 s
[E5] with cache (MEMORY_AND_DISK)             0.99 s
```

```
results table (run 2)

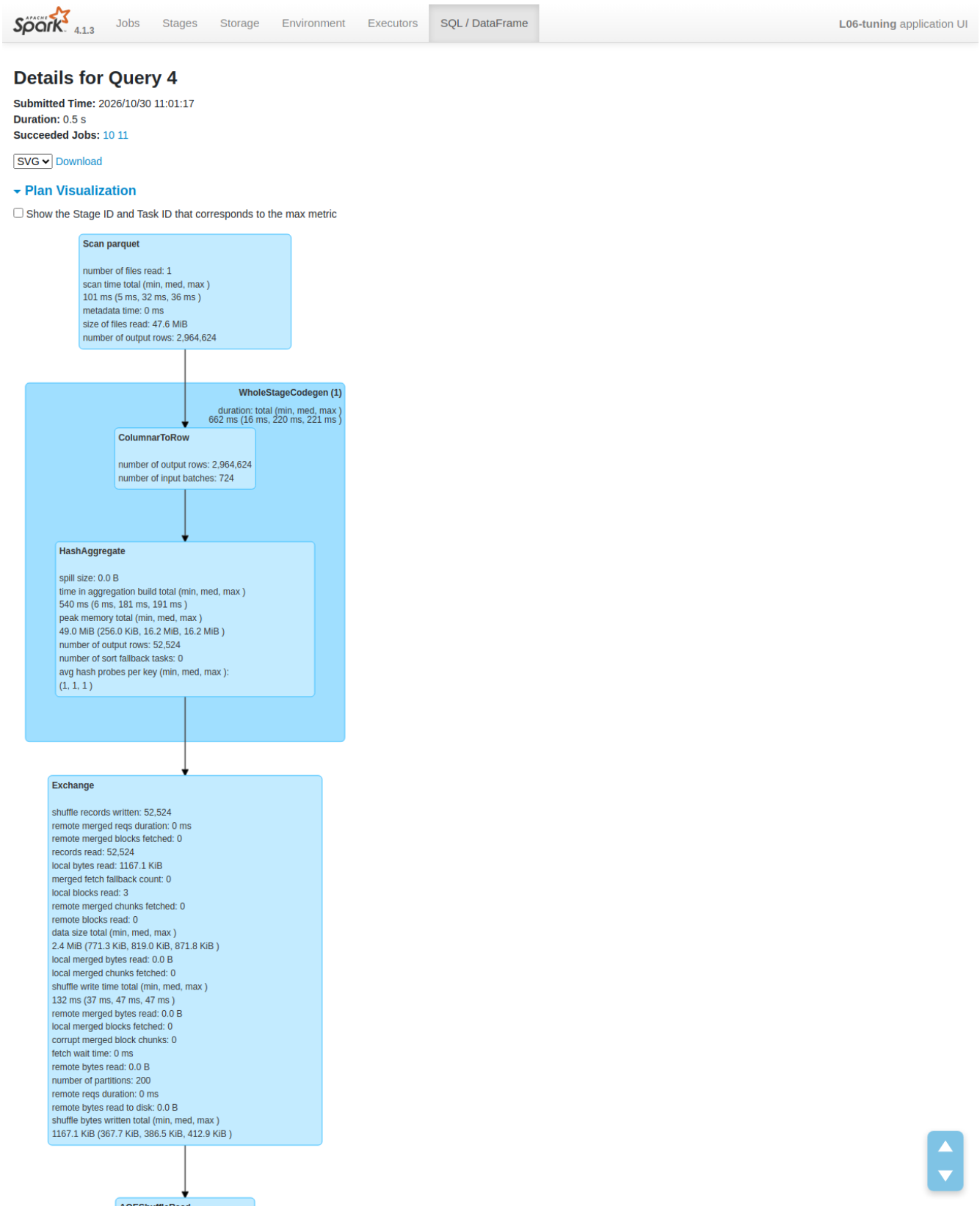
 experiment | variant | seconds | total tasks | task median ms* | task max ms* | plan (final, after AQE)
---|---|---|---|---|---|---
 E1 | AQE off, shuffle.partitions=200 | 2.2 | 204 | 6.0 | 60.0 | Exchange
 E1 | AQE off, shuffle.partitions=8 | 0.33 | 12 | 177.0 | 178.0 | Exchange
 E1 | AQE off, shuffle.partitions=1 | 0.44 | 5 | 232.0 | 233.0 | Exchange
 E1 | AQE on,  shuffle.partitions=200 | 0.53 | 5 | 271.0 | 274.0 | AQEShuffleRead Exchange
 E2 | AQE off, broadcast (threshold 10MB) | 1.2 | 13 | 626.0 | 635.0 | BroadcastExchange BroadcastHashJoin Exchange
 E2 | AQE off, sort-merge (threshold -1) | 1.51 | 21 | 386.0 | 591.0 | Exchange SortMergeJoin
 E2 | AQE on,  threshold -1 + broadcast() hint | 0.4 | 6 | 157.0 | 168.0 | AQEShuffleRead BroadcastExchange BroadcastHashJoin Exchange
 E3 | skewed SMJ, AQE off | 4.1 | 40 | 165.0 | 2366.0 | Exchange SortMergeJoin
 E3 | salted SMJ (salt=64), AQE off | 4.01 | 56 | 590.0 | 1197.0 | Exchange SortMergeJoin
 E3 | skewed SMJ, AQE skewJoin on | 2.22 | 27 | 186.0 | 682.0 | AQEShuffleRead Exchange SortMergeJoin(skew=true)
 E4 | repartition(64) | 7.25 | 68 | 152.0 | 980.0 | Exchange
 E4 | repartition(4) | 4.4 | 8 | 2185.0 | 2210.0 | Exchange
 E4 | coalesce(4) | 3.12 | 4 | 2907.0 | 2970.0 | Coalesce
 E4 | coalesce(1) | 6.95 | 1 | 6800.0 | 6800.0 | Coalesce
 E5 | no cache | 6.4 |  |  |  |
 E5 | cache materialisation (count) | 3.42 |  |  |  |
 E5 | with cache (MEMORY_AND_DISK) | 0.99 |  |  |  |
* median / max task run time of the most skewed stage of that query
```

Bảng đầy đủ, nhận xét từng thí nghiệm và khuyến nghị nằm trong `RESULTS.md`.

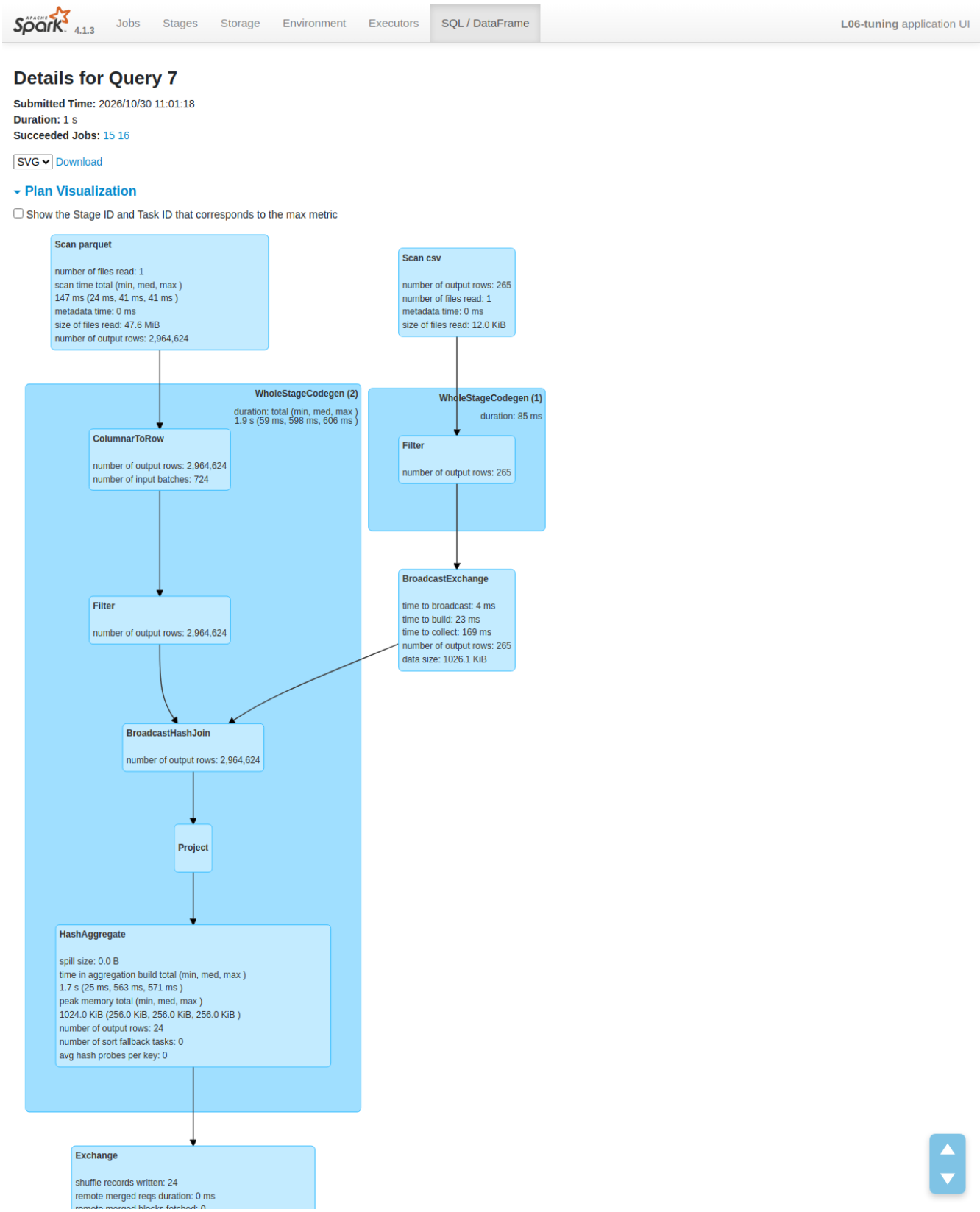
Bằng chứng trên Spark UI



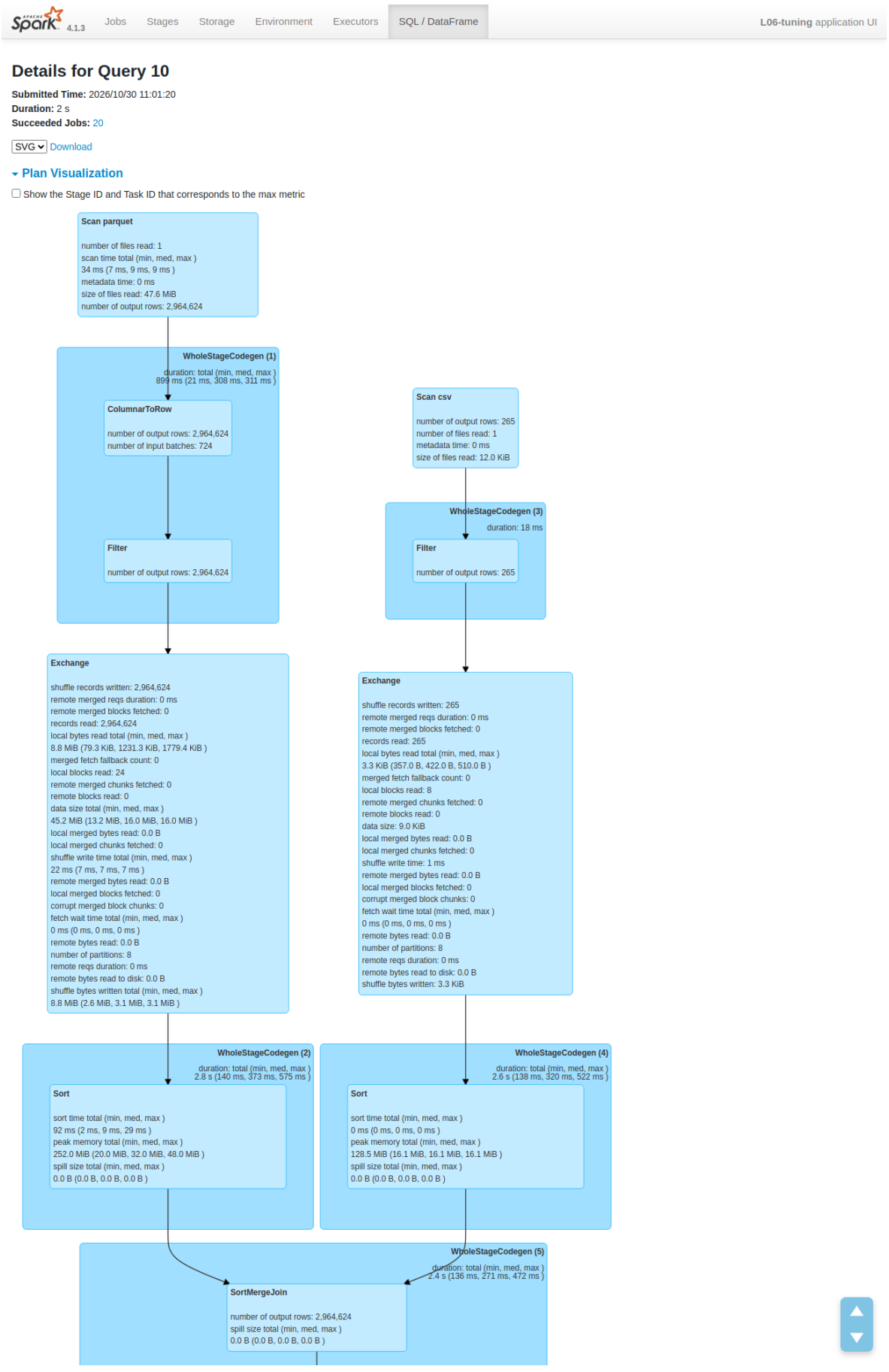
Hình 26: **E1**: AQE off, 200 partition → Exchange 200 partition



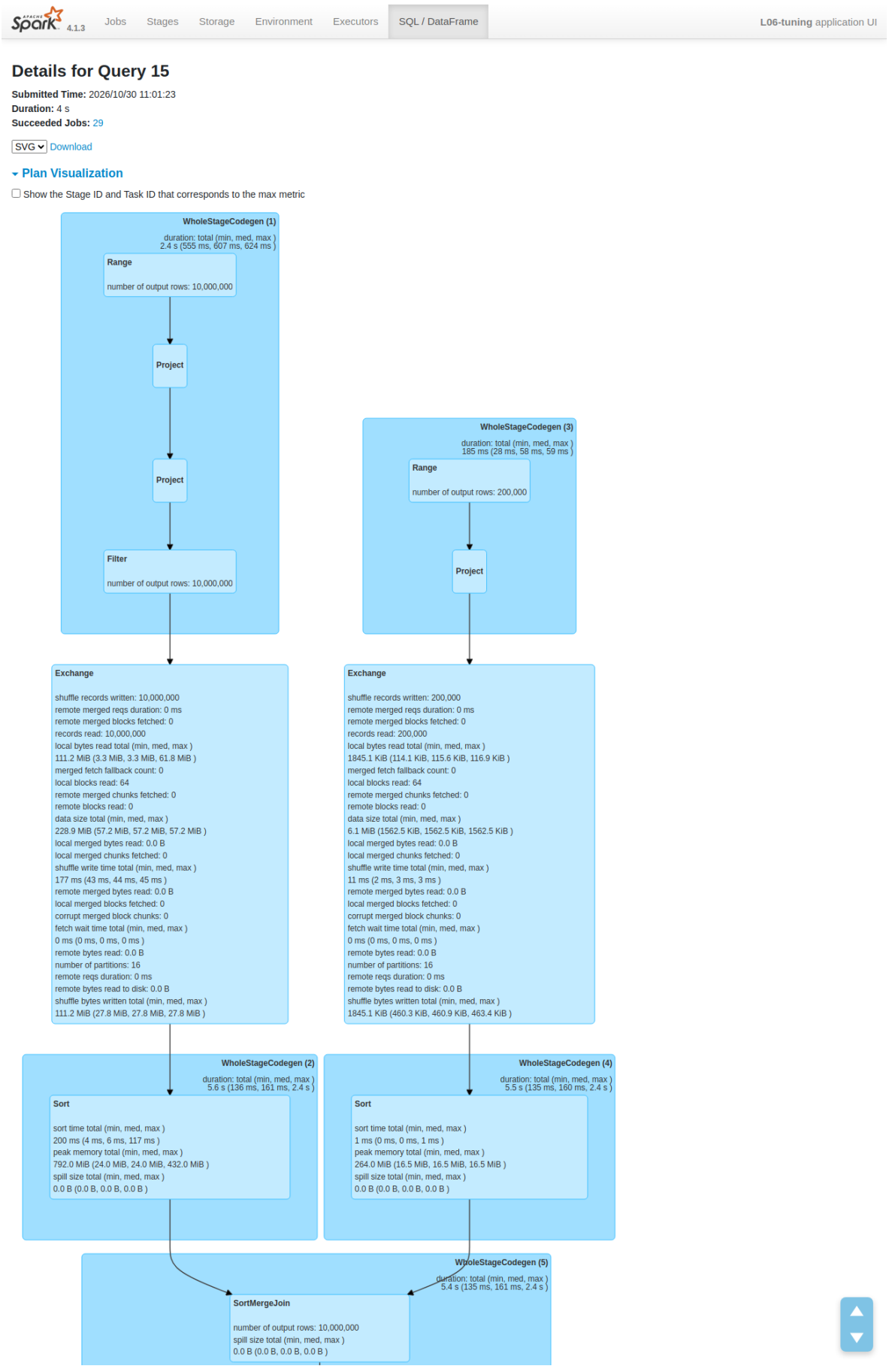
Hình 27: **E1**: AQE on → `AQEShuffleRead` coalesced



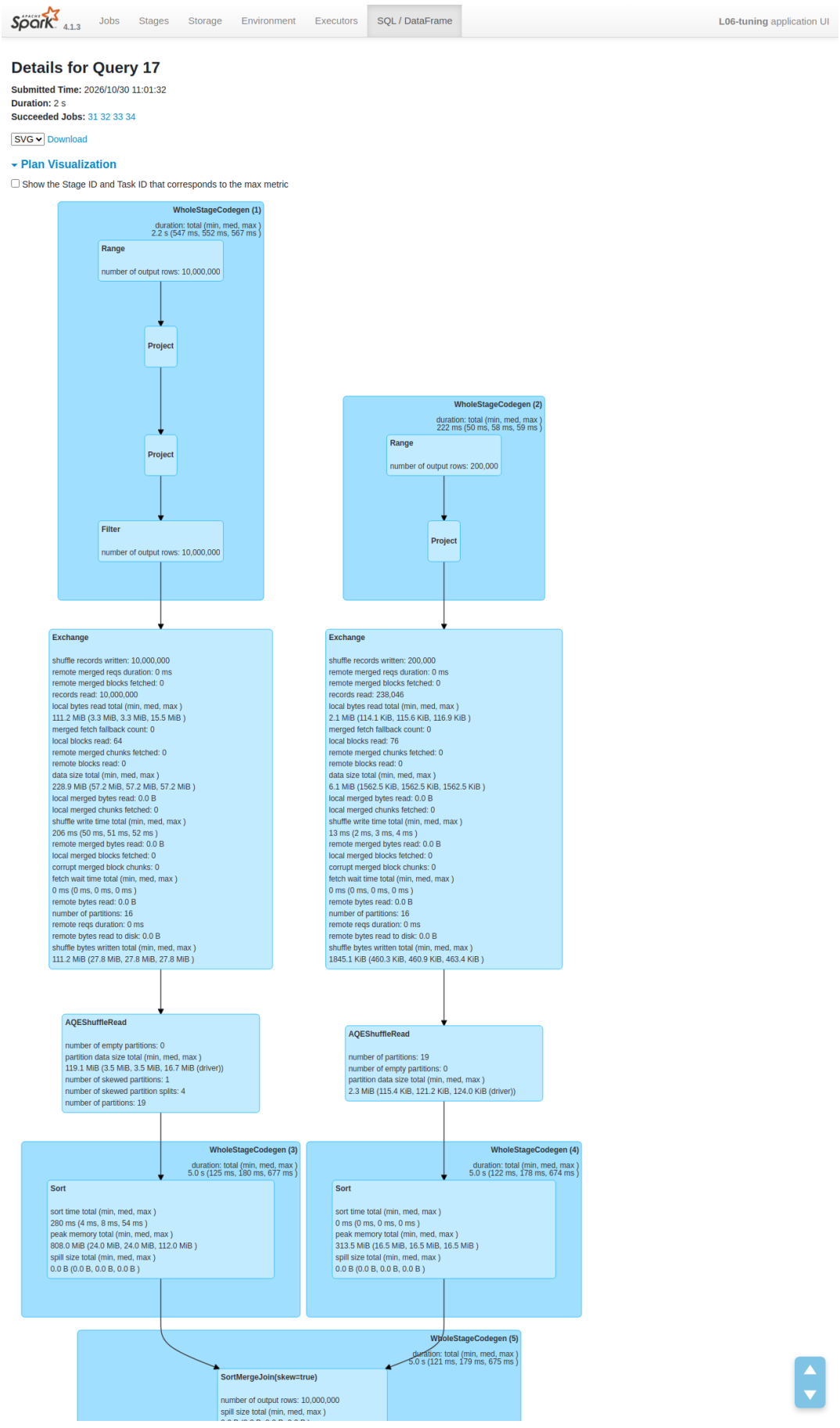
Hình 28: **E2**: `BroadcastExchange` + `BroadcastHashJoin`



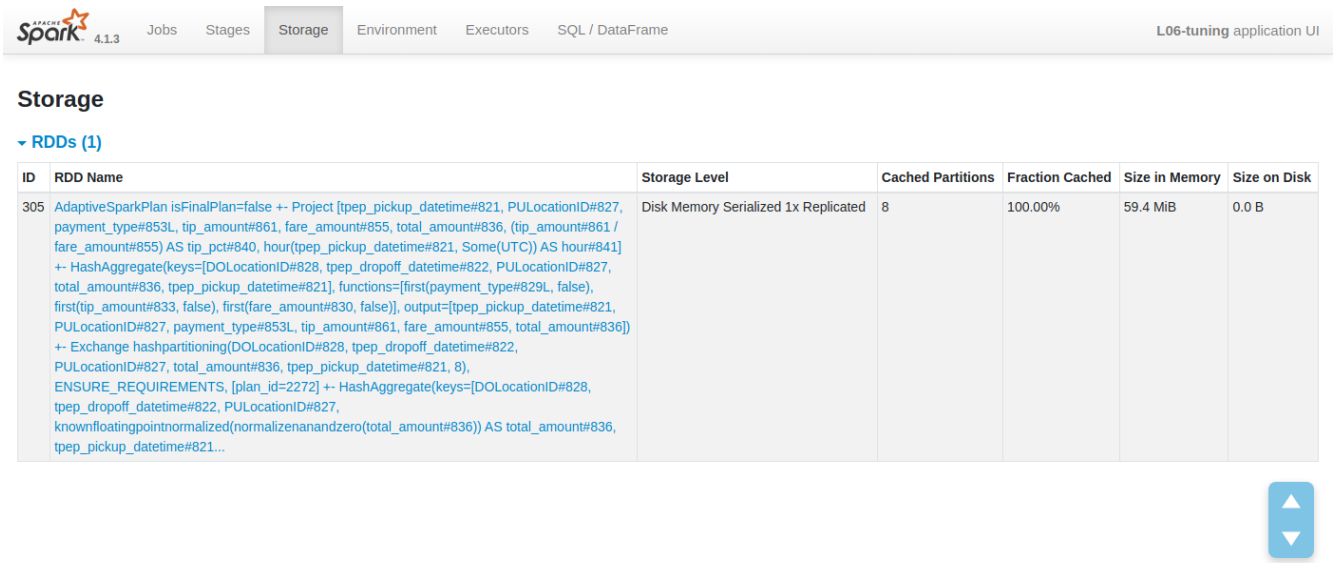
Hình 29: **E2**: threshold -1 → `Exchange` + `Sort` hai phía + `SortMergeJoin`



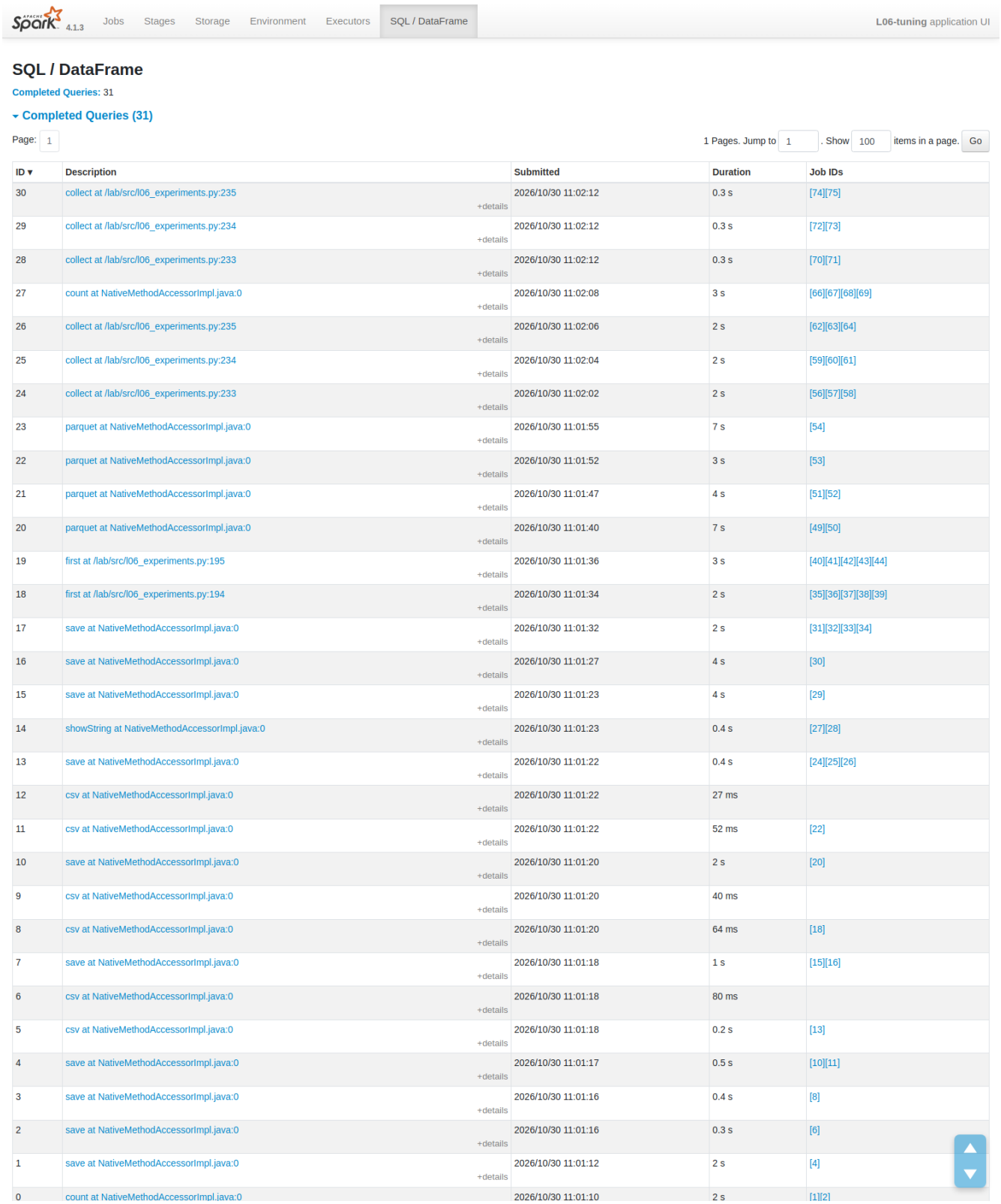
Hình 30: **E3**: baseline SMJ



Hình 31: **E3**: AQE → `AQEShuffleRead` (skewed partitions: 1, splits: 4, partitions: 19) + `SortMergeJoin(skew=true)`



Hình 32: **E5**: Storage tab, `cleaned` cache 8/8 partition, 100 %, 59.4 MiB



Hình 33: Danh sách SQL query

Skew: Summary Metrics của stage join

Stage	Task time min / median / max	Shuffle records median / max
42: baseline, AQE off	140 / 165 / **2,366 ms**	262,619 / **6,264,438** (61.9 MiB, so với 3.4 MiB)
46: salting 64	308 / 590 / 1,197 ms	638,133 / 917,267
52: AQE skew join	129 / 186 / 682 ms	262,677 / 1,576,365 (≈ 6.26 M / 4)

E3: task time and shuffle records per task

```
# task quantiles [min, p25, median, p75, max] from /api/v1/.../taskSummary
stage 42: runTime [140.0, 157.0, 165.0, 379.0, 2366.0] shuffleReadRecords [260166.0, 261225.0, 262619.0, 264416.0, 6264438.0]
stage 46: runTime [308.0, 484.0, 590.0, 901.0, 1197.0] shuffleReadRecords [359285.0, 543450.0, 638133.0, 823338.0, 917267.0]
stage 52: runTime [129.0, 144.0, 186.0, 199.0, 682.0] shuffleReadRecords [260166.0, 261225.0, 262677.0, 265121.0, 1576365.0]
```

Details for Stage 42 (Attempt 0)

Submitted: 2026/10/30 11:01:24
Duration 3 s
Resource Profile Id: 0
Total Time Across All Tasks: 6 s
Locality Level Summary: Node local: 16
Shuffle Read Size / Records: 113.0 MiB / 10200000
Shuffle Write Size / Records: 27.1 KiB / 800
Associated Job Ids: 29

- DAG Visualization
- Show Additional Metrics
- Event Timeline

Summary Metrics for 16 Completed Tasks

Metric	Min	25th percentile	Median	75th percentile	Max
Duration	0.1 s	0.2 s	0.2 s	0.4 s	2 s
GC Time	0.0 ms	3.0 ms	3.0 ms	6.0 ms	25.0 ms
Shuffle Read Size / Records	3.4 MiB / 260166	3.4 MiB / 261225	3.4 MiB / 262619	3.4 MiB / 264416	61.9 MiB / 6264438
Shuffle Write Size / Records	1.7 KiB / 50	1.7 KiB / 50	1.7 KiB / 50	1.7 KiB / 50	1.7 KiB / 50

- Aggregated Metrics by Executor

Tasks (16)

Show 20 entries Search:

Index	Task ID	Attempt	Status	Locality level	Executor ID	Host	Logs	Launch Time	Duration	GC Time	Shuffle Write Size / Records	Shuffle Read Size / Records	Errors
0	298	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:24	0.4 s	6.0 ms	1.7 KiB / 50	3.4 MiB / 265121	
1	299	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:24	0.4 s	6.0 ms	1.7 KiB / 50	3.4 MiB / 264416	
2	300	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:24	0.4 s	6.0 ms	1.7 KiB / 50	3.4 MiB / 260569	
3	301	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:24	0.4 s	6.0 ms	1.7 KiB / 50	3.4 MiB / 262353	
4	302	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:24	0.2 s		1.7 KiB / 50	3.4 MiB / 262677	
5	303	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:24	0.2 s		1.7 KiB / 50	3.4 MiB / 260630	
6	304	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:24	0.2 s		1.7 KiB / 50	3.4 MiB / 262914	
7	305	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:24	0.2 s		1.7 KiB / 50	3.4 MiB / 264774	
8	306	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:25	0.2 s	3.0 ms	1.7 KiB / 50	3.4 MiB / 260412	
9	307	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:25	0.2 s	3.0 ms	1.7 KiB / 50	3.4 MiB / 261225	
10	308	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:25	0.2 s	3.0 ms	1.7 KiB / 50	3.4 MiB / 262619	
11	309	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:25	0.2 s	3.0 ms	1.7 KiB / 50	3.4 MiB / 264108	
12	310	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:25	0.2 s	3.0 ms	1.7 KiB / 50	3.4 MiB / 262023	
13	311	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:25	2 s	25.0 ms	1.7 KiB / 50	61.9 MiB / 6264438	
14	312	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:25	0.1 s	3.0 ms	1.7 KiB / 50	3.4 MiB / 261555	
15	313	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:25	0.2 s	3.0 ms	1.7 KiB / 50	3.4 MiB / 260166	

Showing 1 to 16 of 16 entries Previous 1 Next

Details for Stage 46 (Attempt 0)

Submitted: 2026/10/30 11:01:29
Duration 3 s
Resource Profile Id: 0
Total Time Across All Tasks: 11 s
Locality Level Summary: Node local: 16
Shuffle Read Size / Records: 127.0 MiB / 10200063
Shuffle Write Size / Records: 27.1 KiB / 800
Associated Job Ids: 30

- DAG Visualization
- Show Additional Metrics
- Event Timeline

Summary Metrics for 16 Completed Tasks

Metric	Min	25th percentile	Median	75th percentile	Max
Duration	0.3 s	0.5 s	0.6 s	0.9 s	1 s
GC Time	0.0 ms	0.0 ms	5.0 ms	7.0 ms	12.0 ms
Shuffle Read Size / Records	4.9 MiB / 359285	6.9 MiB / 543450	8 MiB / 638133	10 MiB / 823338	11 MiB / 917267
Shuffle Write Size / Records	1.7 KiB / 50	1.7 KiB / 50	1.7 KiB / 50	1.7 KiB / 50	1.7 KiB / 50

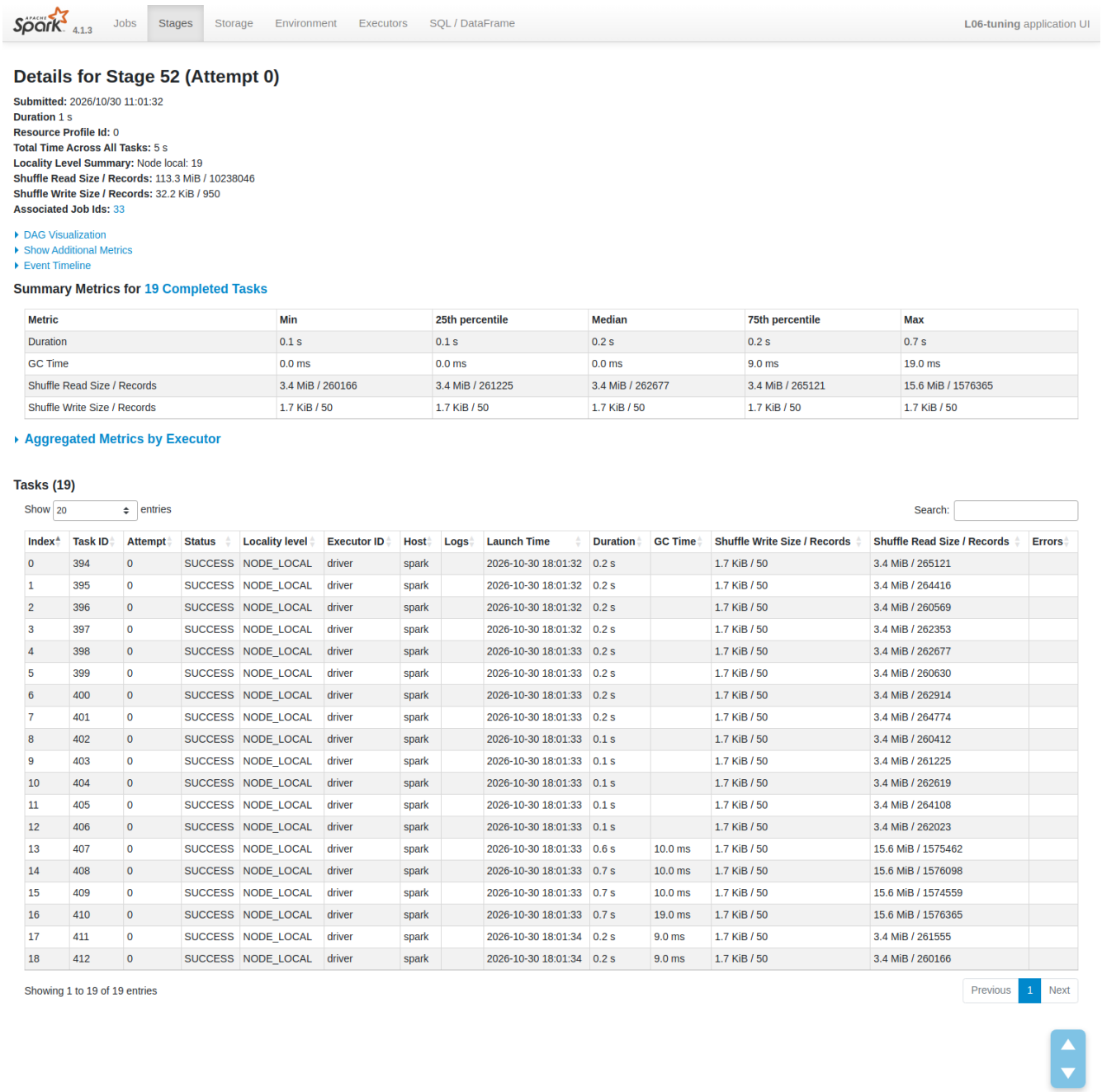
- Aggregated Metrics by Executor

Tasks (16)

Show 20 entries Search:

Index	Task ID	Attempt	Status	Locality level	Executor ID	Host	Logs	Launch Time	Duration	GC Time	Shuffle Write Size / Records	Shuffle Read Size / Records	Errors
0	354	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:29	1.0 s	5.0 ms	1.7 KiB / 50	9 MiB / 733891	
1	355	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:29	0.9 s	5.0 ms	1.7 KiB / 50	7.9 MiB / 637285	
2	356	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:29	1 s	12.0 ms	1.7 KiB / 50	10 MiB / 824727	
3	357	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:29	1 s	12.0 ms	1.7 KiB / 50	10 MiB / 825508	
4	358	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:30	0.8 s	10.0 ms	1.7 KiB / 50	10 MiB / 823338	
5	359	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:30	0.6 s	7.0 ms	1.7 KiB / 50	8 MiB / 638553	
6	360	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:30	0.6 s	3.0 ms	1.7 KiB / 50	8 MiB / 637050	
7	361	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:30	0.4 s		1.7 KiB / 50	4.9 MiB / 359285	
8	362	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:30	0.8 s	7.0 ms	1.7 KiB / 50	11 MiB / 917267	
9	363	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:30	0.3 s	7.0 ms	1.7 KiB / 50	4.9 MiB / 361145	
10	364	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:30	0.5 s	4.0 ms	1.7 KiB / 50	8 MiB / 638133	
11	365	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:30	0.5 s	4.0 ms	1.7 KiB / 50	6.9 MiB / 543450	
12	366	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:31	0.5 s		1.7 KiB / 50	6.9 MiB / 539233	
13	367	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:31	0.6 s		1.7 KiB / 50	8.9 MiB / 727672	
14	368	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:31	0.4 s		1.7 KiB / 50	5.9 MiB / 448281	
15	369	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:31	0.4 s		1.7 KiB / 50	7 MiB / 545245	

Showing 1 to 16 of 16 entries Previous 1 Next



Trả lời checkpoint

Q1. Kết quả `groupBy(PU, DO)` chỉ có vài MB, khoảng 20 nghìn nhóm. 200 partition nghĩa là 204 task, mỗi task chạy 6 ms; chi phí lập lịch, serialize và khởi tạo task áp đảo, nên mất 2.2 s so với 0.33 s khi dùng 8 partition. `shuffle.partitions=1` nhanh ở đây chỉ vì dữ liệu shuffle tí hon. Trên dữ liệu thật, một reducer phải xử lý **toàn bộ** shuffle: không có song song, spill ra đĩa, dễ OOM. Quy tắc: khoảng 100–200 MB mỗi partition shuffle, hoặc để AQE tự quyết.

Q2. Sau khi stage map của shuffle chạy xong, AQE có **thống kê thật** về kích thước từng partition shuffle (map output statistics). `AQEShuffleRead coalesced` cho một reducer đọc gộp nhiều partition nhỏ liền kề cho tới khi đạt `advisoryPartitionSizeInBytes`. Nhờ đó 200 partition chỉ còn 1 task reduce (tổng 5 task, thay vì 204). Planner tĩnh chỉ có **ước lượng** (kích thước

file), và ước lượng thường sai sau filter, join hay aggregate.

Q3. Con số chứng minh skew là **task max** ≫ **median** ở stage join: 2,366 ms so với 165 ms (khoảng 14×). Task đó đọc 6.26 triệu bản ghi trong khi median chỉ 262 nghìn. Salting giảm straggler (max 1,197 ms) nhưng **wall-clock gần như không đổi** (4.10 → 4.01 s). Lý do: - Nó thêm việc ở mọi nơi: `rand()` cho mỗi dòng, thêm cột join, 64 bản sao dim. - 6 triệu dòng hot vẫn phải chia trên 16 partition × 4 core, nên median tăng từ 165 lên 590 ms. Với 4 core, tổng công việc là như nhau, chỉ phân bổ lại.

Salting thắng rõ khi có **nhiều core và partition**: straggler sẽ giữ hàng trăm executor ngồi chờ. Nó cũng thắng khi công việc trên mỗi dòng nặng (dòng rộng, join many-to-many, spill), hoặc khi skew nặng hơn (ví dụ 90 % hot).

Q4. Một partition bị coi là skew khi kích thước của nó **lớn hơn** **`skewedPartitionFactor` × median và** lớn hơn `skewedPartitionThresholdInBytes`. - Mặc định là 5× và 256 MB; lab hạ xuống 3× và 16 MB. - Ở đây partition hot 61.9 MiB so với median 3.4 MiB.

AQE **tách** partition đó thành nhiều task theo khoảng map output (ở đây 4 split, nên 16 → 19 partition), rồi **nhân bản** partition tương ứng của phía bên kia. Plan khi đó hiện `SortMergeJoin(skew=true)`.

Giới hạn của AQE so với salting: - chỉ áp dụng cho join có shuffle ở cả hai phía (SMJ hoặc shuffled hash); - không áp dụng cho mọi kiểu join (không tách được phía cần giữ nguyên của outer join); - cần ngưỡng hợp với dữ liệu; - **không xử lý skew trong** **`groupBy`** hay skew sau join. Salting, hay aggregate hai pha, xử lý được những trường hợp này.

Q5. Không nên broadcast dù một bên "nhỏ" khi: - bảng đó vẫn lớn trong bộ nhớ: driver phải collect trước rồi gửi bản sao tới mọi executor, gây OOM driver hoặc broadcast timeout; - thống kê **đánh giá thấp** kích thước, ví dụ sau filter hay UDF; - là **full outer join** (không hỗ trợ), hoặc phía cần broadcast là phía được giữ của outer join; - join chạy lặp lại nhiều lần trên dữ liệu thay đổi, nên mỗi lần phải dựng lại broadcast.

Q6. - `repartition(n)` **luôn shuffle**: partition cân bằng (4 file đều nhau), và stage trước giữ nguyên song song. Cái giá là thêm một shuffle (4.40 s). - `coalesce(n)` gộp partition sẵn có **không shuffle**: rẻ (3.12 s), nhưng **có thể tạo file lệch**. Ở đây chỉ ra 3 file vì input chỉ có 3 row group, nên một partition rỗng. - Vì `coalesce` là narrow dependency, nó **giảm song song của cả stage trước**: `coalesce(1)` chạy scan và ghi trong **1 task** (6,800 ms).

Q7. DataFrame được cache là kết quả của một bước **đắt** (`dropDuplicates`, shuffle toàn bộ khoảng 3 triệu dòng) và được dùng lại 3 lần. - Không cache: dedup chạy 3 lần, 6.40 s. - Có cache: dedup chạy 1 lần (3.42 s) rồi 3 query chỉ còn 0.99 s.

Một scan Parquet thuần, đã prune cột, thì vốn rẻ: đọc vectorized, có column pruning và predicate pushdown. Cache khi đó chỉ tốn bộ nhớ và thời gian materialize, gần như không lợi. Quy tắc: cache cái **đắt và được dùng lại**, rồi `unpersist()`.

Q8. Checklist trước khi mua thêm phần cứng: 1. **Đọc plan và Spark UI**: tìm stage chậm nhất, spill, GC, và tỉ lệ task max/median để phát hiện skew. 2. **Lọc và chọn cột sớm**: partition pruning, predicate/column pushdown, dùng định dạng cột (Parquet/ORC). 3. **Kích thước partition shuffle hợp lý**: bật AQE (coalesce và skew join), khoảng 100–200 MB mỗi partition. 4. **Chọn đúng chiến lược join**: broadcast bảng nhỏ; với skew thì dùng AQE skew join, salting, hoặc tách hot key. 5. **Tránh UDF Python** (dùng hàm built-in hoặc pandas UDF), **cache có chọn lọc**, và **tránh file nhỏ** khi ghi (repartition hoặc compaction).